4D SUMMIT 2012

Pre-class

Extending your 4D apps to Web and Mobile

Presented by:

Milan Adamov

Angelo Caroprese

Simil Guetta

Philip Keth

October 23rd, 2012

TABLE OF CONTENTS

INTRODUCTION **7**

FUNDAMENTALS **7**

- **HTTP** **7**
- **HTML** **7**
- **CSS** **7**
- **4D TAGS** **7**
- **4D commands for the web** **8**
- **JavaScript** **8**
- **JQuery** **8**
- **XML** **8**
- **JSON** **8**

COMMUNICATION BETWEEN THE BROWSER AND THE WEB SERVER **8**

NEW WEB SERVER IN 4D V13 ..... **9**

WEB SETTINGS **9**

- **Configuration page** **9**
  - SSL 10
  - File architecture 10
- **Options** **10**
  - Web cache 11
  - Web Process 11
  - Web Passwords 12
  - Send Extended Characters Directly 12
  - Standard Set 12
  - Use Keep-Alive Connections 12
- **Log type** **13**
- **Log backup** **14**
- **Setting the web options programmatically** **15**

STATIC WEB **15**

- **Launch the 4D Web Server** **15**
- **Serving a static document** **16**
- **Creating a static document with HTML** **16**

Serving pictures **17**
Path referencing **17**
Absolute path 18
Relative path to the current folder 18
Relative path to the web root folder 18
Compression **18**
DYNAMIC WEB **19**
4D TAGS **19**
4DTEXT 19
4DHTML 19
4DSCRIPT 20
4DINCLUDE 20
4DIF, 4DELSE,4DELSEIF and 4DENDIF 20
4DLOOP 20
4D Tags at work **21**
4DACTION **23**
Commands that send responses **24**
WEB SEND TEXT 24
WEB SEND BLOB 24
WEB SEND FILE 25
WEB SEND RAW DATA 25
WEB SEND HTTP REDIRECT 25
On Web Connection **26**
SESSION MANAGEMENT **27**
HTTP Overview **27**
What Session Management tries to address 28
Client / Server Analogy 28
Web Application Analogy 28
Session Management Implementation **28**
New HTTP session management **30**
Session ID 30
Session Validation 31
Determining the current session ID 32
Renaming the Session Cookie 32

Set maximum number of concurrent sessions 32
Set Session Timeout 32
Session cookie timeout 32
Session process timeout 33
Deactivate 4Dv13 HTTP SessionManagement 33
Force Web process termination 33
Maintaining critical states or information 33
CSS - CASCADING STYLE SHEETS 38
CSS Types 38
Inline CSS 38
Internal CSS 38
External CSS file 38
How does it look like? 39
CSS3 39
Some CSS3 Examples 39
JAVASCRIPT 40
What does JavaScript do? 40
JavaScript Libraries 40
Learn JavaScript 41
A WEB DEVELOPMENT TALE - EPISODE 1 - GO WEB! 42
User-stories 42
Database example 43
Step1: Contacts List 45
Analyzing the communication 45
Building the Contacts html list 46
Using 4D Tags for dynamic pages 47
Introducing jQuery 49
Creating On Web Connection handler 51
Client-side debugging 52
Step 2: Displaying Contact detail 55
Choosing an approach 55
Detecting contact selection 55
Requesting Contact Detail information 57
Structured format = JSON 58

Displaying the detail 58
Display photo 61
Step 3 - Update contact 62
Submit Contact 62
Update on Server-side 63
Validate jQuery plugin 64
Required 66
Switch to Task management 66
A WEB DEVELOPMENT TALE - EPISODE 2 - WHERE ARE WE? 67
Step 4 – Task #7 Single-Page 68
Organizing the page using DIV blocks 68
Adding the tabs 70
Listing the categories 74
Step 5 – Task #2 - CRUD Tasks 76
Setting up the HTML 76
Tasks grid 77
Task detail 80
Instant updating 83
Improving sending JSON 85
A WEB DEVELOPMENT TALE - EPISODE 3 - FINALE 86
Step 6 – Task #6 User authentication 86
Login Form 86
Login Sequence 87
Session management in 4Dv13 90
Logout 91
Improving security 92
More on session management 92
When to use sessions? 93
Filtering items per user 93
Step 7 – Task #8 Filtering by categories. 94
End of the day 95
MOBILE DEVELOPMENT 97
Mobile Architectures 97
Native Applications 97

Web-Based Applications 97
Hybrid Applications 98
Choosing an Architecture 98
Let the Requirements Dictate the Architecture 98
Some Questions to Help Guide you 99
Mobile Interface Design 99
Desktop to Mobile example 99
Pre-Class Demo to Mobile interface 102
Under the Hood of our Mobile Demo 104
Structure of the Mobile Web Interface 106
How JQuery-Mobile works 109
Loading Content from the Demo Database 110
Manipulating Objects in the DOM 110
Conclusion on Mobile Development 113
REFERENCES 113

Extending your 4D apps to Web and Mobile

Presented by:

Milan Adamov

Angelo Caroprese

Simil Guetta

Philip Keth

INTRODUCTION

Ever since the Internet massification, companies started asking themselves whether it is important to deploy their applications on the Web. Maybe at the beginning, the answer was no, because the true potential of the web had not been discovered yet, but year after year companies realized the importance of deploying their applications, or part of them, on the web. And year after year that question is more relevant because we are at a point where having a web application is not an option but an essential requisite of the company's strategy in staying competitive.

That is why it is so important for 4D Developers to grasp the necessary knowledge to create and deploy Web Applications. And this is possible since many years ago with 4D, however with the launch of the revamped Web Server in 4D v13, 4D Developers have more power in their hands to develop competitive Web and Mobile Applications.

The aim of this pre-class is to give 4D Developers all the elements they need to extend their existing applications to the web and mobile, introducing the basic concepts of the web server, HTML, CSS, JavaScript and also the design considerations to envision and implement web and mobile applications.

FUNDAMENTALS

The following list shows the key concepts that are mentioned in the session. These concepts will not be covered in depth because each of them is a whole universe on their own.

HTTP

Hypertext Transfer Protocol. A communications protocol that is in charge of the communication between the client and the web server. It is defined by the standard RFC 2616 from the IETF (Internet Engineering Task Force)

HTML

Hypertext Markup Language is a Markup Language for displaying web pages. This language is used to describe the content and with the introduction of HTML5 it gets more power than just displaying content.

CSS

Cascading Style Sheets. Files that describe the format of HTML elements on a web page. Crucial of the look and feel on the browser side. CSS is in charge of the presentation of the content. Its latest version, CSS3 is included in the HTML5 specification and it is more powerful and versatile. For example the new definitions for transitions take a load off from JavaScript. Gradients, borders and shadows help create nice buttons where before images were needed.

4D TAGS

Set of tags in the form of HTML comments that are processed by the 4D Web Server to generate dynamic content. 4D Tags are so powerful that they are not only to generate web content but they can also be used to generate reports and nice web interfaces within 4D.

4D commands for the web

There are a series of commands for managing the operation of the web server, such as data processing, session handling, type of response and the web server administration. Some of these commands are: WEB SEND BLOB, WEB CLOSE SESSION, WEB START SERVER among others.

JavaScript

Programming language used in the front-end of web applications. JavaScript is in charge of handling the events in the web page such as click, data input, data validation, requests to the server, etc. Also with the introduction of HTML5, JavaScript has more power.

JQuery

A JavaScript library that simplifies front-end web development. It facilitates access to objects, communications with the servers and the dynamic definition of the content.

XML

eXtended Markup Language. The markup language used for data formatting used for multiple purposes. Being data interchange between applications one of them.

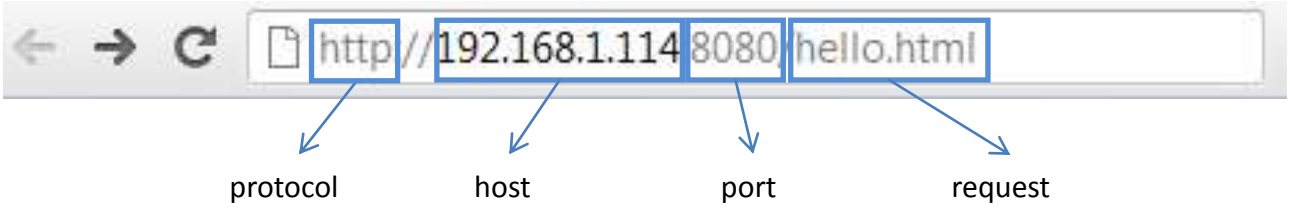
JSON

JavaScript Object Notation. Standard for data interchange between the client and the web server.

COMMUNICATION BETWEEN THE BROWSER AND THE WEB SERVER

The communication between the browser and the web server is done by requests done from the browser to the server. These requests usually are called URLs and it has 4 parts, let's analyze the following URL:

http://192.168.1.114:8080/hello.html



- **Protocol**: http or https
- **Host**: can be a domain or an IP address
  - www.greatsite.com
  - 173.194.42.52
  - 192.168.1.15
- **Port:** 80 by default, but 8080, 8081 and 8888 are widely used by intranet or extranet sites.
- **Request:** can be an HTML file, a script, an image, a PDF or any file or a customized request that will be handled by the server.

NEW WEB SERVER IN 4D V13

4D v13 introduced a new web server that is up to the crucial task of serving web applications based on 4D. The new web server is a modern web server that is Unicode, Carbon Free, Preemptive, it supports native compression for static content, new cache handling and true built-in session management system. All those features give it a powerful performance so 4D Developers can rest assured that their web applications will excel with 4D.

Many of the new features work transparently for the developer, such as the compression, the full Unicode support and the cache management. However there are two features that really facilitate the web development; these are the session management and the process management. These two features alone make that the developer has to write much less code than before.

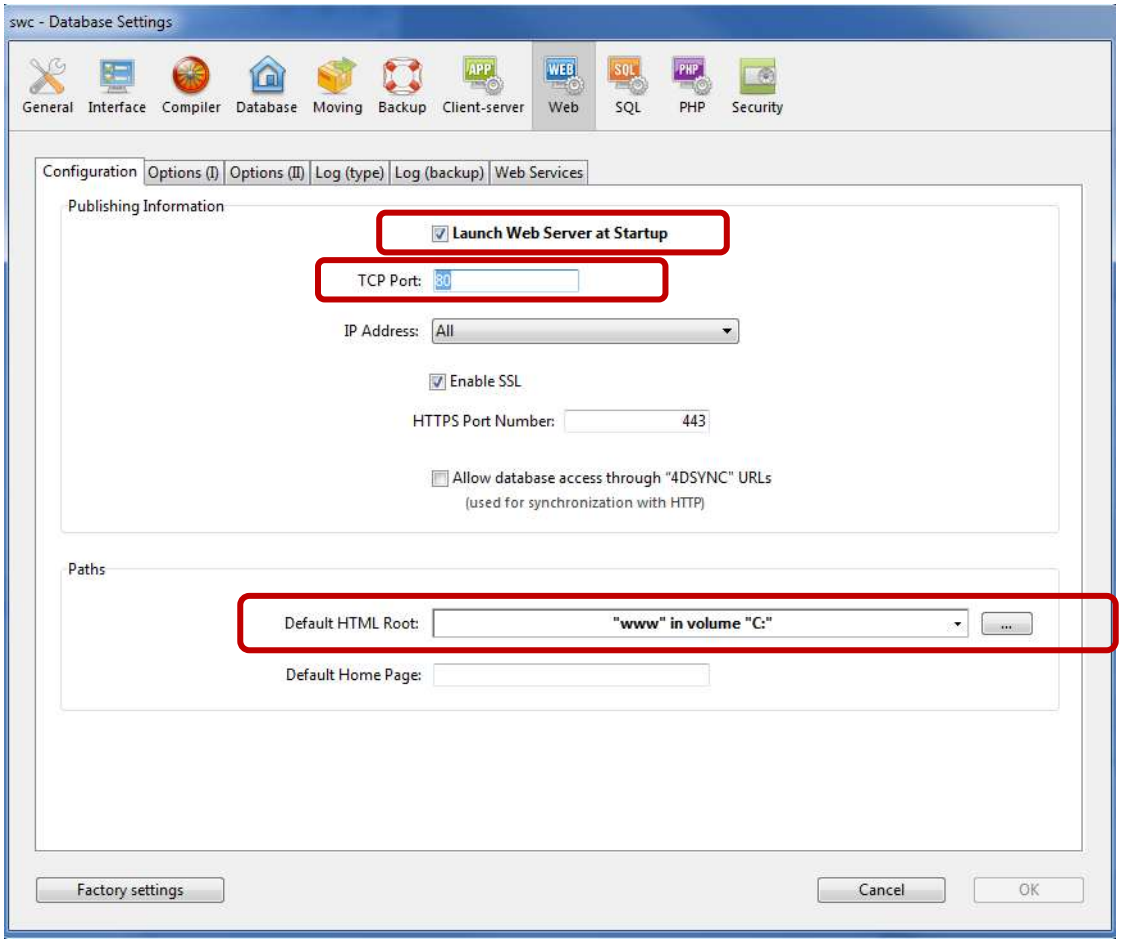
WEB SETTINGS

To configure the web there are dedicated pages to the web server in the Database Settings. These pages allow setting up the server port, the web folder, the timeouts, the web logs and also the security of the web server.

Configuration page

The main configuration of the web server is defined in this page.

- Automatic Web Server Startup
- HTTP Port
- Web folder
- SSL Enabled
- HTTPS Port



The 4D Web Server can be launched automatically at startup by checking the **Launch Web Server at Startup** checkbox in the Web section of the Database settings. In that place there are other two properties to set, the TCP port and the Web folder.

The TCP port (or HTTP port) is the port where the Web Server is going to publish its content. The value is 80 by default but other common ports used for that purpose are 8080, 8081or 8888. This is useful if port 80 is used by other applications (e.g. Apache or IIS) on the machine where the web server is.

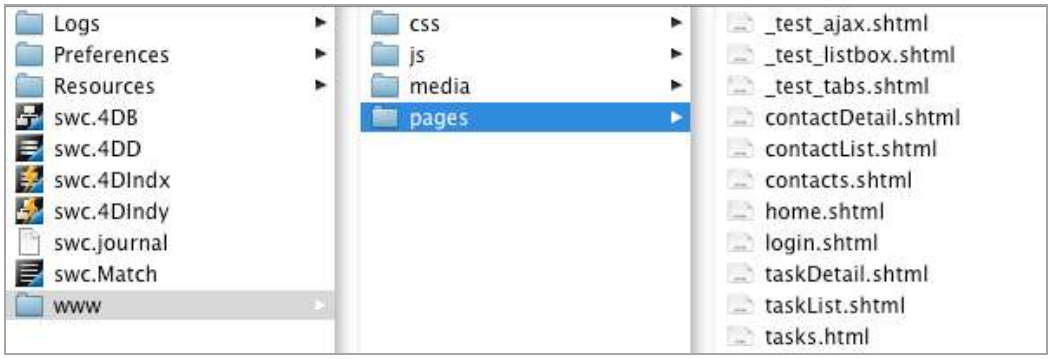
SSL

To secure the communication between the client and the server, 4D v13 supports SSL (Secure Socket Layer); this can be enabled or disabled according to the developer's needs. In order to make SSL work on the web server is necessary to have an SSL certificate installed next to the structure file. Once SSL is enabled is possible to set up the HTTPS port, usually 443.

File architecture

The **Web folder** parameter is where the content of the web server is stored; HTMLs, pictures, JavaScript files and CSS files. In this example the Web folder was called simply **www** and it was created next to 4D structure file. Once the root web folder is set we can store our resource files inside it in an organized manner, creating a folder for the scripts usually called **js**, a folder for the styles called **css**, a folder for images, video and other media called **media** and a folder to store templates called **pages**. In a static website we would have different folders according to the section of the web site, for example news, products, services and those folders would have other folders inside to allow more levels of classification or taxonomy.

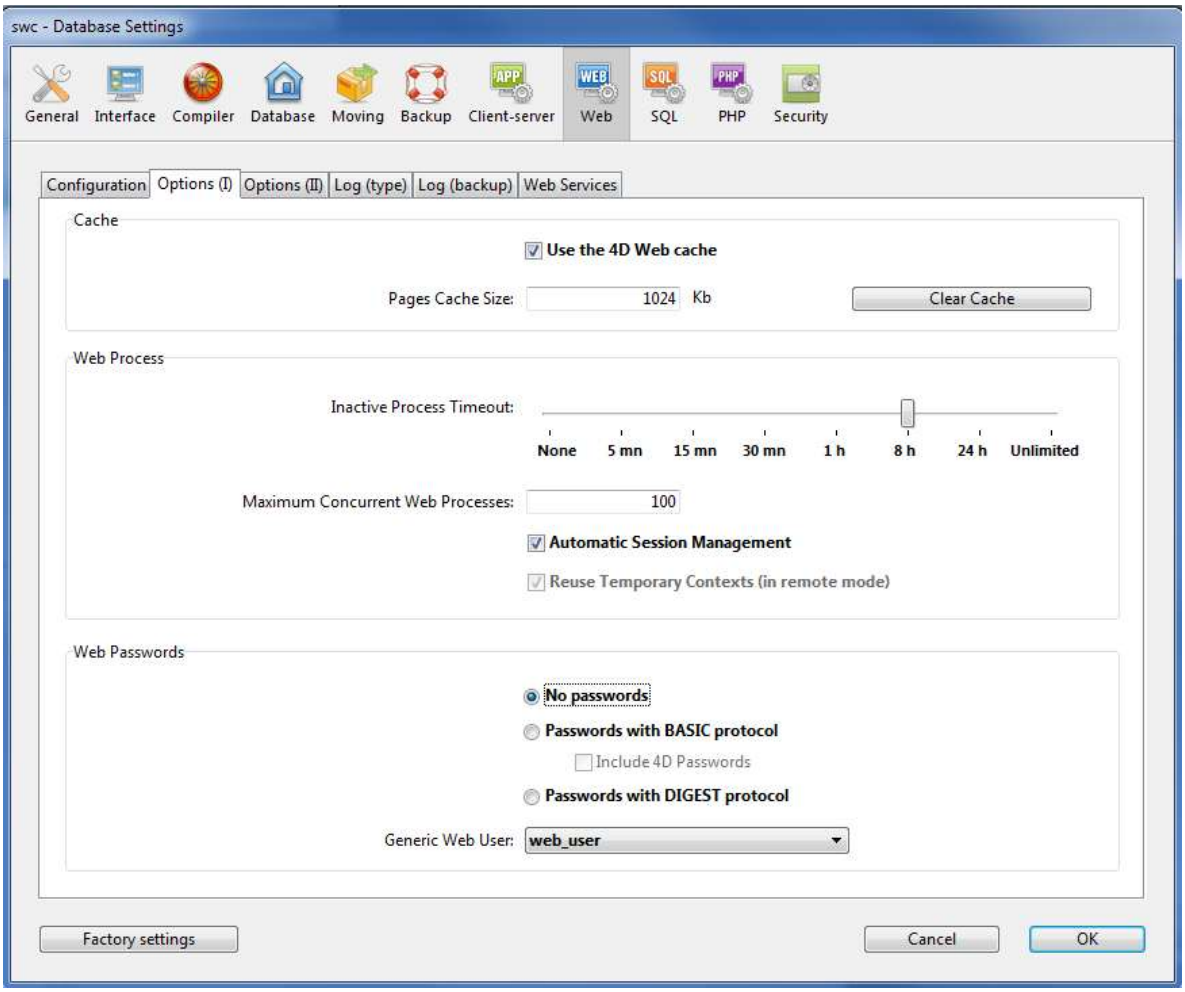
This is the file structure of this pre-class



Options

There are two Option pages. These pages define the settings for:

- Cache management
- Web processes
- Web passwords
- Character encoding
- Keep-Alive connections



Web cache

These settings allow define whether or not the cache memory is enabled and how much memory is needed to store static documents such as HTML pages, images, scripts and styles. The way the cache works is that stores the content of the static requests in order to serve them faster the next time a client needs them, in that way the web server has to access the drive fewer times, this behavior results in more performance in the web site.

The minimum value is 1024 KB and the maximum is 2 GB.

The cache statistics can be obtained with magical URL /4DSTATS or /4DHTMLSTATSor using the 4D command WEB GET STATISTICS. The cache memory can be cleared by clicking the button “Clear cache”.

Web Process

Inactive Process Timeout: defines the maximum timeout before closing for inactive Web processes on the server.

Maximum Concurrent Web Processes: this is the maximum number of web processes that can run simultaneously on the web server, thus determining the maximum number of HTTP connections. This option prevents 4D Web Server from saturation of having a high number of requests.

When that number of processes minus one is reached the web server stops ceases to create new processes and returns the HTTP status 503 Service Unavailable.

Minimum value: 10

Maximum value: 32000

This option is programmable using WEB SET OPTION.

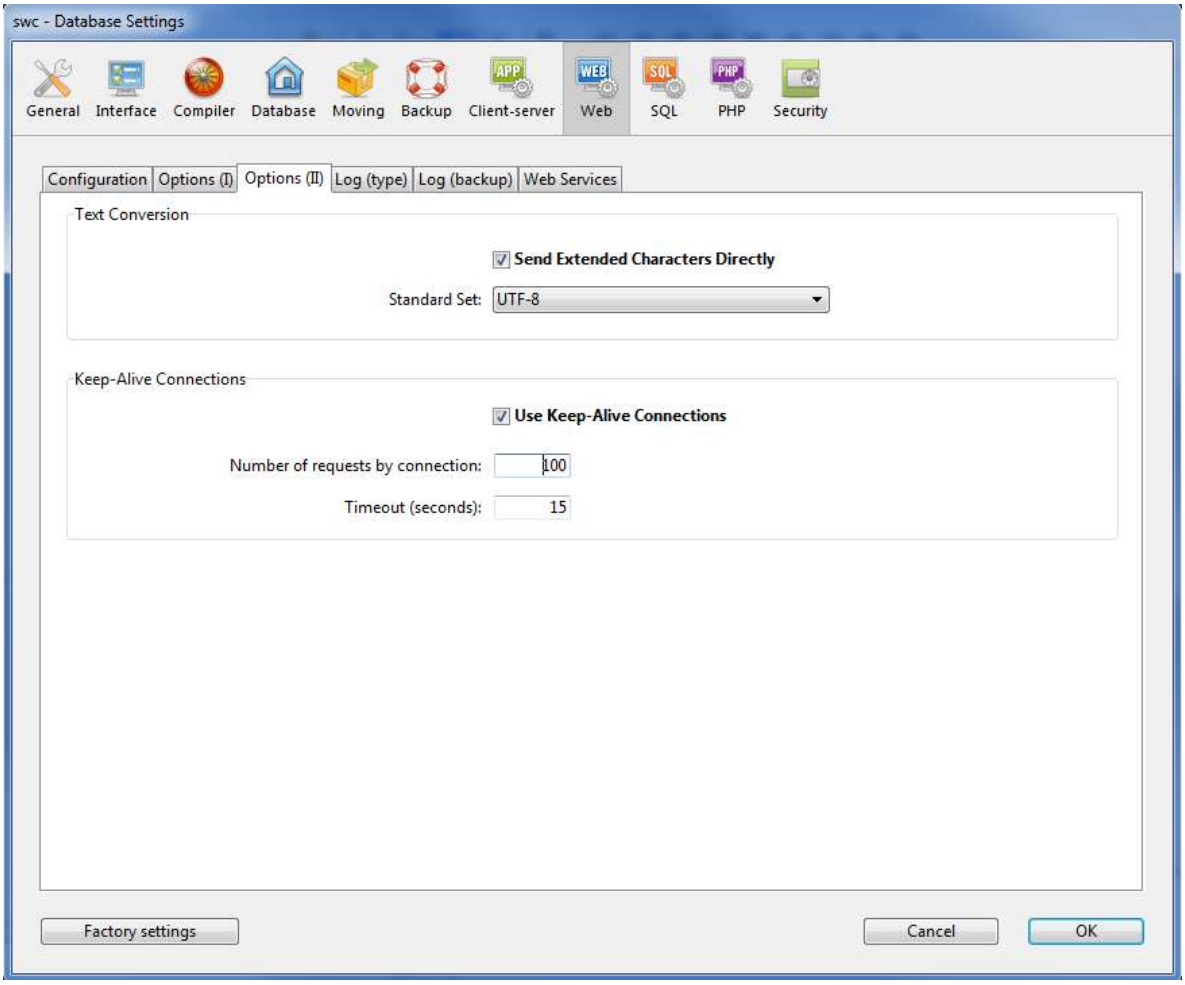
```
WEB SET OPTION(Web Max Concurrent Processes;800)
```

Automatic session management: enables or disables the built-in mechanism to manage sessions. This option is enabled by default for new databases created in 4D v13. For converted databases, this option is disabled by default.

Web Passwords

These options define the options to enhance the security of a web site done with 4D. It defines the type of passwords to be used and the generic web user i.e. the default user when doing web requests.

The second page of options defines the options for text encoding and Keep-Alive connections.



Send Extended Characters Directly

The parameter bypasses the process of converting extended characters into HTML entities and therefore sending the extended character as-is. This can improve performance.

Standard Set

It is the character set is by the web server to set text. There is a big set of options to choose from, however UTF-8 is defined by default.

Use Keep-Alive Connections

It is an efficient way to manage the communication between the client and the server because it uses the same HTTP connection to handle a series of requests and responses. It is enabled by default.

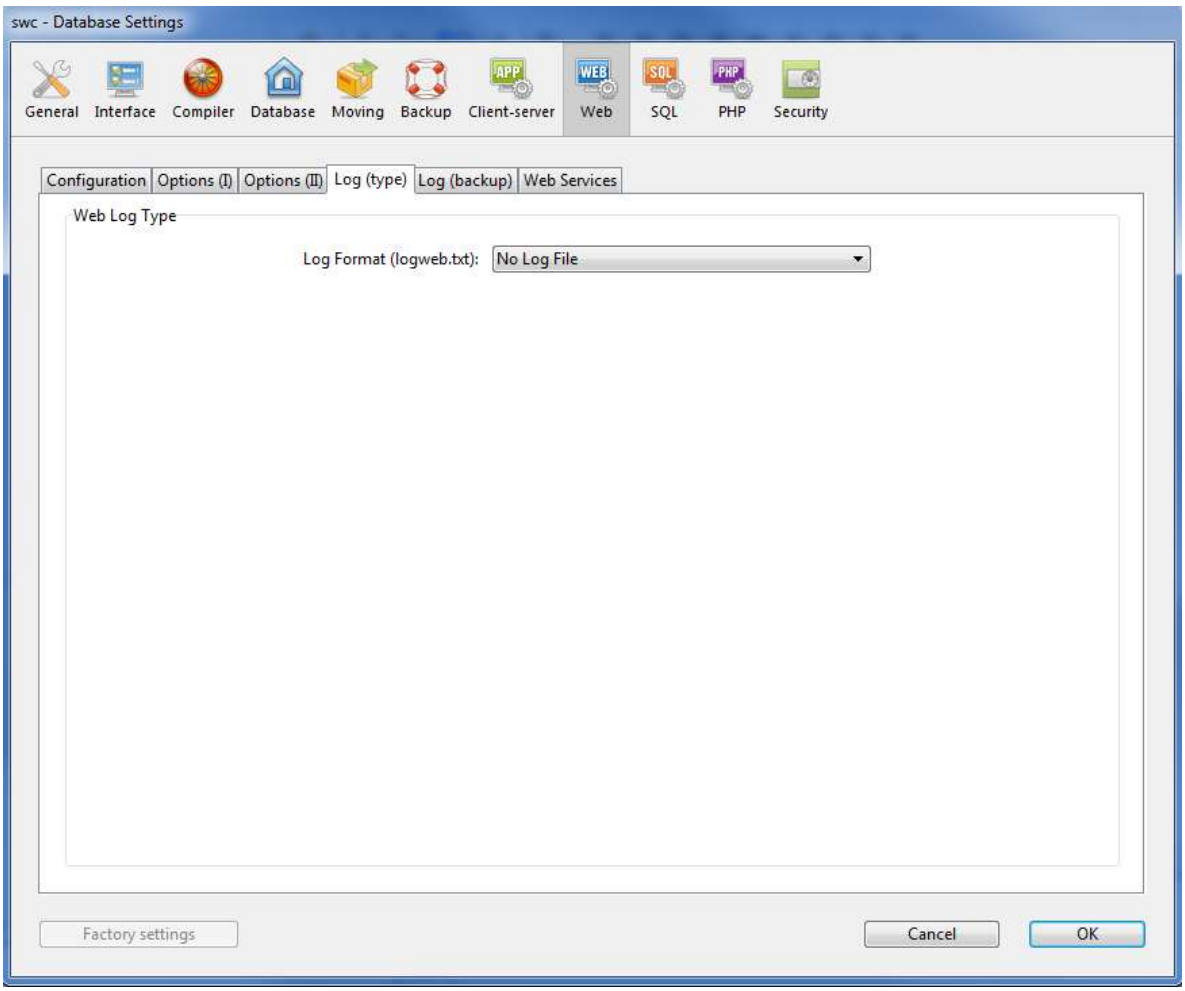
Log type

The log type option allows the developer choosing between 5 options, no log file or 4 types of log file.

The log file is an important tool to analyze the requests done from the clients to the server. There are a lot of analyzing software programs and services available for this task. AWstats, Webalizer, Urchin and Google Analytics, among others.

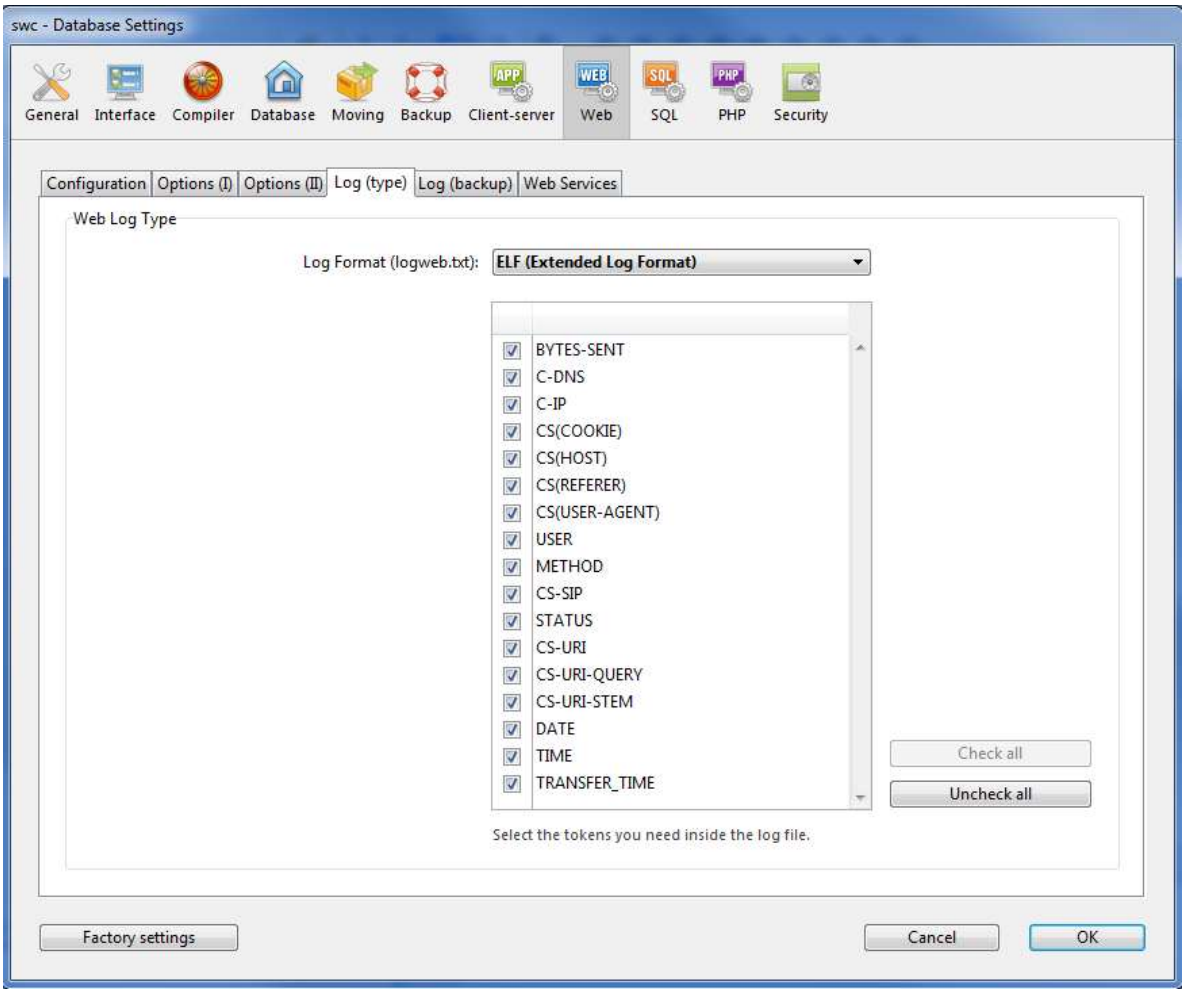
The analytics tools throws very important information about the behavior of the website, it explains how the visitors are using the website, what information they are looking for, more visited pages, which countries, what are the peak times. In the end this web log file can become into an important marketing tool that will help business people to take decisions.

The four options available are:



CLF, DLF and WLF give predefined fields of information, such as the request, data, time, user agent (browser), browser IP address, size of the response and other information.

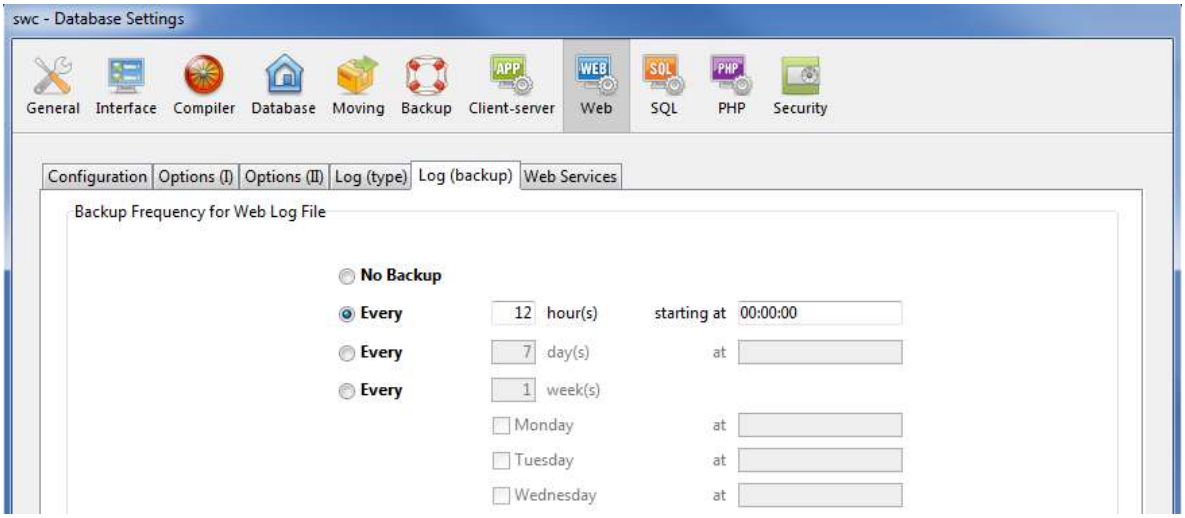
ELF gives the option to select which fields are to be included in the log file. This configuration depends on the requirements of analytics software to be used.



Log backup

A log file can grow to a huge size depending on the usage. The higher the request the higher the log. That is why it is important to have a backup of the web log file.

The log backup page configures the backup options and depending on the usage the developer can define and hourly daily or weekly backup of the web log file.



The other tab is to set up the SOAP web services, which is a that topic not covered in this pre-class.

Setting the web options programmatically

Most of the options described above can be configured using the 4D language with commands WEB SET OPTION and SET DATABASE PARAMETER.

There are other options that are not present in the database settings dialogs that can be configured using WEB SET OPTION, for example the compression options.

Examples:

```
WEB SET OPTION(Web Character set;"UTF-8")
WEB SET OPTION(Web Log Recording;2)
WEB SET OPTION(Web HTTP Compression Level;3)
SET DATABASE PARAMETER(Port ID;80)
```

Documentation

WEB SET OPTION: http://doc.4d.com/4Dv13.1/help/Command/en/page2733.html

SET DATABASE PARAMETER: http://doc.4d.com/4Dv13.1/help/Command/en/page642.html

STATIC WEB

The term static web refers to a web site that serves only files and images created before the request is done. A static web site does not do any processing in generating content because the content is already there. For that reason it is very fast.

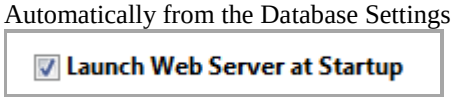
The usual content on a static web site is HTML files, images usually in JPG, GIF or PNG formats, PDF files and ZIP files.

In the case of 4D, serving static web is very efficient because it takes advantage of the cache management and the built-in compression system.

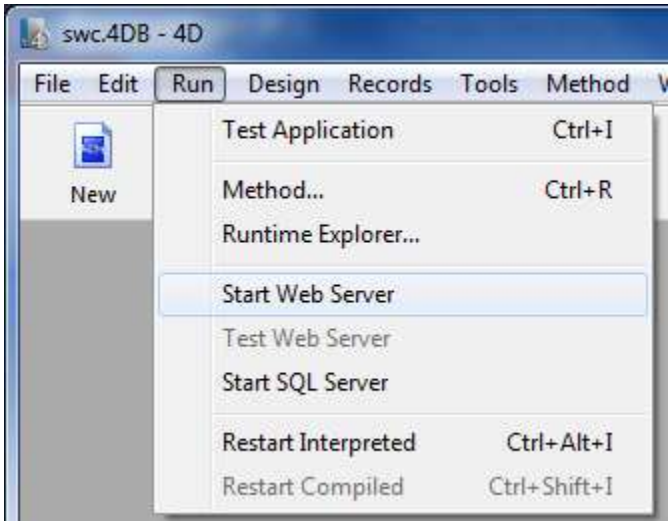
Launch the 4D Web Server

The 4D Web Server can be launched in different ways.

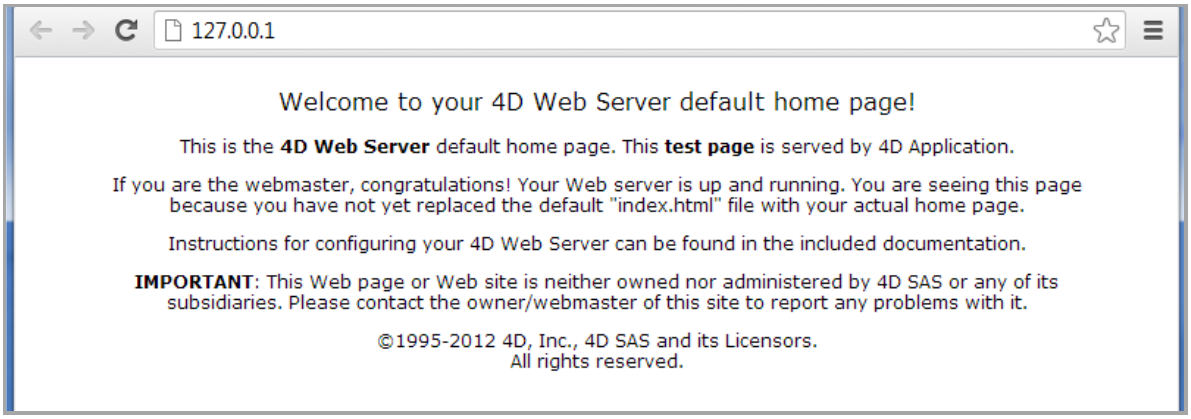
- Automatically from the Database Settings

- Using the command **WEB START SERVER**
- From the **Run** menu:



Once the Web Server is launched this can be tested from the Run menu selecting the line **Test Web Server**. If no page is defined 4D will put in place a default page.



You can test your website at one of the following URLs (considering the port is set to 80):

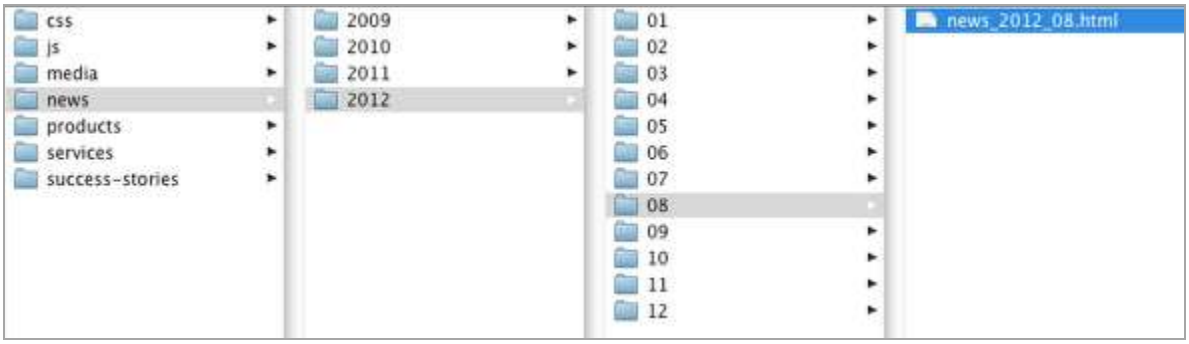
http://localhost

http://127.0.0.1

Serving a static document

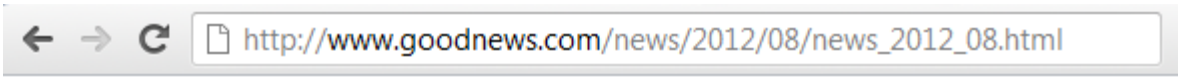
To serve a static document (HTML, image, etc.) suffices to place the document in the web folder in the adequate location. Given that this static content the information has to be classified in an ordered manner to facilitate the location of the resource to users, developers and also to search engines. This is one part of the taxonomy of the website.

Let's say we want to serve the news file of the newsletter sent on August 2012. This file would be located from the web root folder in the folder: /news/2012/08/news_2012_08.html



And in the browser the link would be:

http://www.goodnews.com/news/2012/08/news_2012_08.html



And that rule applies to every static content.

Creating a static document with HTML

Let's create our first static document with HTML, and of course we are going to do the cliché example of Hello world.

We create an HTML file called hello.html and save it at the web root folder.

The content of the file is:

```
<html>
<head>
  <meta charset="utf-8">
  <title>Hello World!</title>
</head>
<body>
       <h1>Hello world!</h1>
       <p>A very simple HTML example.</p>
</body>
</html>
```

If it was saved at root level the file can be tested at: http://127.0.0.1/hello.html

Serving pictures

Pictures can be shown directly inserting their URL in the browser or can be called from an HTML file inserting the path to the image. For example:

```
<img src="/images/picture.jpg">
```

This HTML code will make a request to the web server asking for that file in that path relative to the web root folder.

When serving pictures it is necessary to select the right format and compression.

Usual formats for the web:

- **JPG:** Pictures rich in colors, photographies. This format has a high level of compression, for example a 800 kb BMP file will weigh 30 kb when converted to JPG at 80% quality.
- **GIF:** Pictures with few colors, illustrations, icons. Can use transparencies. Simple animations.
- **PNG:** Icons, buttons, illustrations with effects as shadows and transparencies.

Never use TIFF or BMP on the web because these formats are too heavy and therefore are not efficient to transport.

Path referencing

When referencing resources on a web page there are various ways of doing it depending on the needs. It is important to precisely define the path reference of a resource, otherwise it might lead to small errors like not showing an image to big errors like not invoking a JavaScript library, in that case it might break the web site functionality. These ways of referencing the path of an image, a script, another HTML file or a style are:

- Absolute path

- Relative to the current folder
- Relative to the web root folder

Absolute path

An absolute path is a path that contains the full URL of the resource; this is the protocol, the domain, probably the port, the folders and the file name.

Absolute paths are mainly used to reference resources from other websites. For example a link to other site or maybe a script that is hosted in Google. Some websites have the strategy of storing images in another server; those images would need an absolute path to be accessed.

Examples:

```
<img src="http://192.168.1.123/images/picture.jpg">

<a href="http://www.domain.com/news/october12.html">

<script src="http://www.domain.com/js/actions.js">

<link href="http://www.domain.com/css/style.css">
```

Relative path to the current folder

This type of path is useful to refer resources that are in the same folder, usually another HTML file or to access a resource that will certainly be in a folder whose relative location is proven. This can be a folder at the same level, or two levels down or one level up. Beware when moving a page from one folder to another because if it is moved to a different level probably will not find its resources.

```
<img src="images/picture.jpg">

<a href="october12.html">

<script src="js/actions.js">

<link href="../../css/style.css">
```

It is very used by libraries and frameworks to access their own resources (scripts, css or images) because the position relative to the root folder is not known, it can be one level down, two levels down, it is uncertain. That is why this type of referencing is useful.

Relative path to the web root folder

In this case the referencing is very simple and safe because it all depends from the web root folder. There is no need to know if the resource is one level up or down. What matters is to know how deep that resource is from the root of the website.

```
<img src="/images/picture.jpg">

<a href="/october12.html">

<script src="/js/actions.js">

<link href="/css/style.css">
```

Compression

Since 4D v13 the web server has built in GZIP compression that brings an enormous leap in performance because it will send smaller files or responses over the network that traduces in more speed.

The 4D Web server will send compressed content to those web clients that tell the server that they support gzip compression. This information is sent by the browser in the HTTP header.

```
▼ Request Headers      view source
  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
  Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.3
  Accept-Encoding: gzip,deflate,sdch
  Accept-Language: en-US,en;q=0.8,es;q=0.6
  Cache-Control: no-cache
```

DYNAMIC WEB

When the request to the web server is not a file in the drive then the Web Server analyzes the request to process it and generate the content accordingly.

A dynamic request can be a list of products whose price is greater than $50 and less than $100. Because there is no file called products_gt_50_lt_100.html, it is necessary to create a dynamic request that will return the products in that range.

A request for this kind of file can look like:

http://mystore.com/products.shtml?bottomprice=50&topprice=100

http://mystore.com/Products/Search/PriceRange/50/100

It all depends on how the developer wants to design and handle the request. There are many possibilities.

In 4D there are a number of ways of achieving this but first we need to look at how to create dynamic content in 4D

4D TAGS

4D Tags are special pieces of HTML code written in the format of HTML comments. When a dynamic request is done to the 4D Web Server and it finds 4D Tags these are parsed either to execute a method or to retrieve a variable, array or field.

- **4DTEXT** inserts 4D variables or 4D expressions
- **4DHTML** inserts HTML code
- **4DSCRIPT** executes a 4D method
- **4DINCLUDE** it includes the content of one page inside another
- **4DIF, 4DELSE and 4DENDIF sets** conditionals inside HTML code
- **4DLOOP and 4DENDLOOP** handles loops inside HTML code

4DTEXT

It is used to retrieve variables, array elements, fields and it even executes 4D expressions such as Current time. Examples of 4DTEXT:

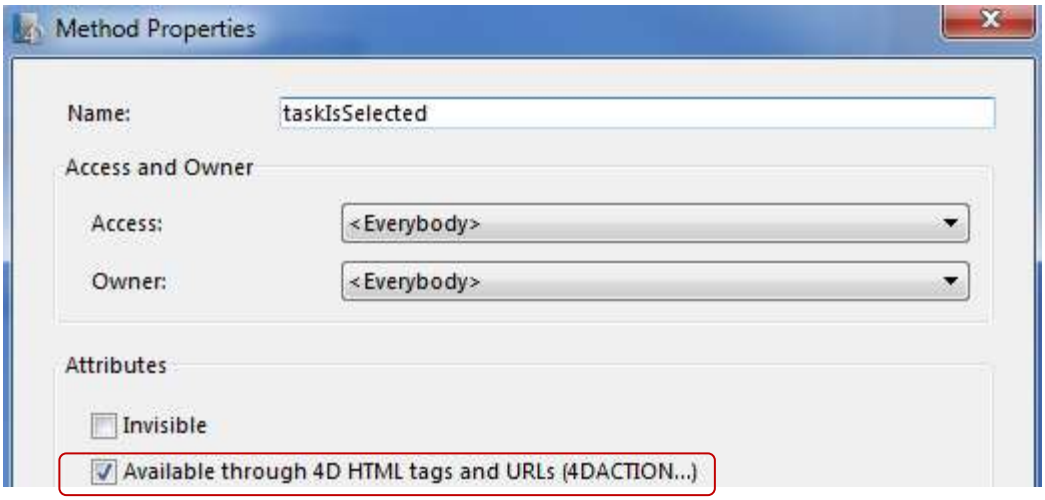
```
<h1>Welcome <!--#4DTEXT [CONTACTS]FirstName--></h1>
<p>The user name is: <!--#4DTEXT  vtUserName--></p>
<!--#4DTEXT myArray{7}-->
<!--#4DTEXT Current time-->
There are <!--#4DTEXT Records in selection([INVOICES])--> invoices pending
```

4DHTML

It works similar to 4DTEXT but it displays HTML code.

4DSCRIPT

Invoking 4DSCRIPT executes a 4D method inside the web process that called it. In order to work, the method must have enabled the **Available through 4D HTML tags and URLs** attribute.



Syntax:

```
<!--#4DSCRIPT/MethodName/Param-->
```

In the method the parameter is received as "/Param", note that it include the slash, which has to be removed to properly process the information.

4DINCLUDE

4DINCLUDE inserts the content of one page inside another. This is useful to handle in a centralized way common pieces of a website such as headers, footers and other interface elements that appear regularly.

The following code inserts a menu page inside another page. **Header** is a folder at the same level as the page that calls 4DINCLUDE.

```
<!--#4DINCLUDE header/menu.html-->
```

4DIF, 4DELSE,4DELSEIF and 4DENDIF

These 4D tags allow evaluating conditionals inside an HTML page. It works similar to conditionals in 4D where an expression is tested to check if the value is true or false to execute one action or the other. In this case, to display one information or the other.

```
<div>
<!--#4DIF (vNumRecords>0)-->
        <p><!--#4DTEXT vNumRecords--> records were found.</p>
<!--#4DELSE-->
        <p>No records were found. Please try another criteria</p>
<!--#4DENDIF-->
</div>
```

4DLOOP

This tag loops through the current selection of records in a table, an array and it can be used with a method as well.

Example of loop with a table:

```
<!--#4DLOOP [CONTACTS]-->
       <p><!--#4DTEXT [CONTACTS]FirstName--></p>
       <p><!--#4DTEXT [CONTACTS]LastName--></p>
       <p><!--#4DTEXT [CONTACTS]Email--></p>
       <p><!--#4DTEXT [CONTACTS]Telephone--></p>
<!--#4DENDLOOP-->
```

Example of loop with an array:

```
<!--#4DLOOP arr_cities-->
       <option><!--#4DTEXT arr_cities{arr_cities}--></option>
<!--#4DENDLOOP-->
```

Example of loop with a method: in this case 4DLOOP is executed **while** the result of the method is true, similar to loops done with **While** in the 4D language.

```
<!--#4DLOOP my_method-->
       <p><!--#4DTEXT vtVar--></p>
<!--#4DENDLOOP-->
```

4D Tags at work

The following examples will show how to use 4D Tags. The first thing is to create a **template** that will be the base to create the HTML files. This template will have extension **.shtml** when the server "sees" a file with that extension it processes the 4D Tags in that file.

This example will show the list of contacts in a 4D database.

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>Contacts</title>
  <meta name="description" content="List of contacts generated from 4D">
</head>
<body>
       <h1>List of contacts</h1>
</body>
</html>
```

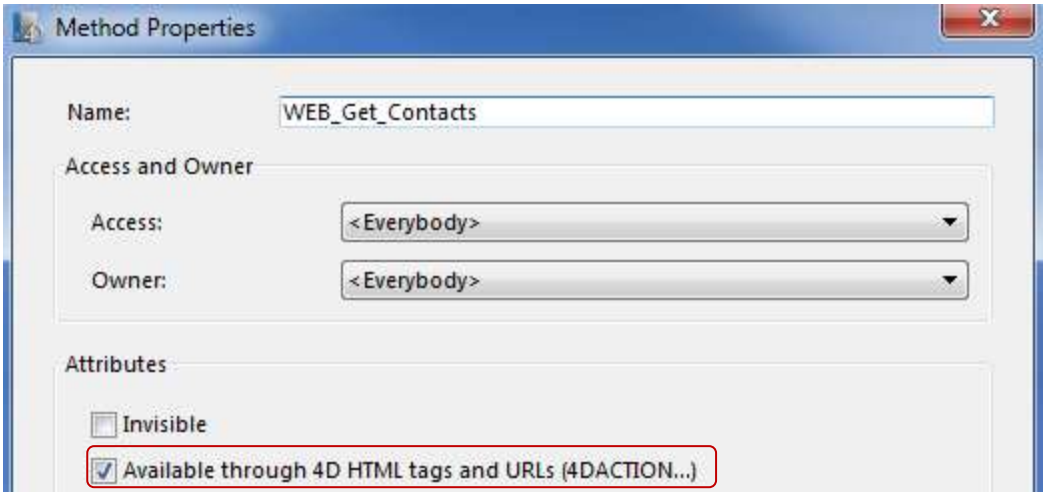
From this template we begin to add 4D tags elements. First let's add the date after the H1 tag:

```
<h6><!--#4DTEXT Current date--></h6>
```

Then we need to create a selection of records from the [Contacts] table calling a method that selects all the records in that table. The parameter $0 and $1 must be declared if the application is going to be compiled.

```
  //Method WEB_Get_Contacts

C_TEXT($1;$0)
ALL RECORDS([Contacts])
ORDER BY([Contacts];[Contacts]Firstname)

vlNumRecords:=Records in selection([Contacts])
```

This method must have enabled the Available through 4D HTML tags and URLs attribute.



Now we can add the 4DSCRIPT tag to the HTML code.

```
<!--#4DSCRIPT/WEB_Get_Contacts-->
```

After the method is called via 4DSCRIPT all the process and interprocess variables and table selections in that process are available.

```
<p>Listing <!--#4DTEXT vlNumRecords--> contacts</p>
```

And with 4DLOOP we can loop through the records in the current selection of the STYLES table.

```
<!--#4DLOOP [Contacts]-->
      <h3><!--#4DTEXT [Contacts]FirstName--> <!--#4DTEXT [Contacts]LastName--></h3>
<!--#4DENDLOOP-->
```

Putting everything together the final file looks like:

```
<!doctype html>
<html>
   <head>
     <meta charset="utf-8">
     <title>Contacts</title>
     <meta name="description" content="Contacts">
   </head>
   <body>
       <h1>List of Contacts</h1>
       <h6><!--#4DTEXT Current date--></h6>
       <p>Listing <!--#4DTEXT vlNumRecords--> Contacts</p>
       <!--#4DSCRIPT/WEB_Get_Contacts-->
       <!--#4DLOOP [Contacts]-->
         <h3><!--#4DTEXT [Contacts]FirstName--> <!--#4DTEXT [Contacts]LastName--></h3>
       <!--#4DENDLOOP-->
  </body>
</html>
```

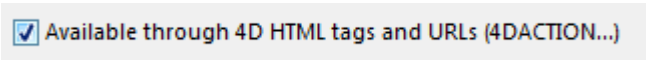
Then we save this file as contacts.shtml at the root folder and when we test it in the browser looks like:

Serving SHTML files is the easiest way to serve content from a 4D database, however it is not the only one. There are other two ways of making a request to the 4D Web Server.

- **4DACTION:** it allows calling 4D methods directly from a URL
- **On Web Connection:** Database method that handles customized URLs

4DACTION

A 4D method can be called directly from the web using a URL defined for that purpose. Obviously the method has to be available via web enabling the aforementioned attribute.



This is the syntax: http://127.0.01/4DACTION/4DMethod

As an example we will show the list of tasks in the database. The procedure is similar to the one done to show the contacts but with the difference that the current selection for the table is created in the called method and not through 4DSCRIPT.

First we create a 4D method that will be called via 4DACTION and it will create the selection of records.

```
// Method Tasks_List

C_TEXT($1;$0)
ALL RECORDS([Tasks])
vlNumRecords:=Records in selection([Tasks])
WEB SEND FILE("tasks.html")
```

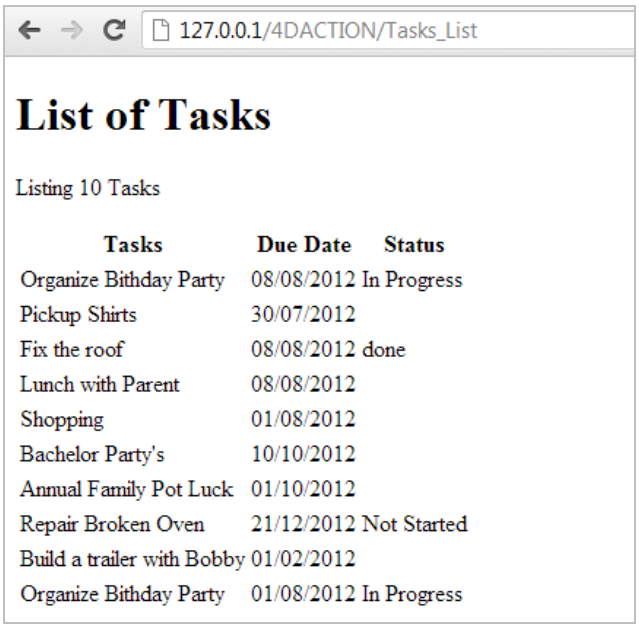
What is new in this method is the final line that calls the 4D command WEB SEND FILE. This command does two things. It processes the 4D tags in the file and it sends the response to the browser.

Note that the file passed has extension HTML and not SHTML. So it is not necessary to have that extension here to indicate that there is dynamic code because WEB SEND FILE processes any 4D TAGS that it finds. The HTML code for the tasks.html is:

```
<!doctype html>
<html>
       <head>
         <meta charset="utf-8">
         <title>Tasks</title>
```

```
        <meta name="description" content="Tasks">
      </head>
      <body>
        <h1>List of Tasks</h1>
        <p>Listing <!--#4DTEXT vlNumRecords--> Tasks</p>
             <table>
             <tr><th>Tasks</th><th>Due Date</th><th>Status</th></tr>
        <!--#4DLOOP [Tasks]-->
             <tr><td><!--#4DTEXT [Tasks]Title--> </td><td><!--#4DTEXT [Tasks]DueDate--
></td><td><!--#4DTEXT [Tasks]Status--></td></tr>
        <!--#4DENDLOOP-->
        </table>
      </body>
</html>
```

The URL to test would be: http://127.0.0.1/4DACTION/Tasks_List



Commands that send responses

In the example of 4DACTION we responded with the command WEB SEND FILE. However that command is not the only one. He complete list is:

- WEB SEND TEXT
- WEB SEND BLOB
- WEB SEND FILE
- WEB SEND RAW DATA
- WEB HTTP REDIRECT

WEB SEND TEXT

This command sends plain text as a response of the HTTP communication. That text is can be formatted in HTML, JSON or have no format at all.

WEB SEND BLOB

It is useful to send blobs such as images created dynamically in 4D, without the need of saving them as files before sending them to the client. When using WEB SEND BLOB you have to define the type of information a.k.a the MIME type. Examples:

Extension	Mime Type
.html	text/html
.shtml	text/html
.css	text/css
.pdf	application/pdf
.js	application/javascript
.json	application/json
.txt	text/plain
.text	text/plain
.gif	image/gif
.jpg	image/jpeg
.jpeg	image/jpeg
.mpeg	video/mpeg
.mpg	video/mpeg
.mov	video/quicktime
.wav	audio/wav
.xml	application/xml
.zip	application/x-zip
.gz	application/x-gzip

Sending a blob variable that contains a JPEG picture:

```
WEB SEND BLOB(vxImage;”image/jpeg”)
```

WEB SEND FILE

It sends the HTML file defined in the path parameter. This parameter is the path from the web root folder. The command processes any 4D tags in the file, if there were any.

```
C_TEXT($1;$0)
ALL RECORDS([Tasks])
vlNumRecords:=Records in selection([Tasks])
WEB SEND FILE("tasks.html")
```

WEB SEND RAW DATA

In the commands mentioned above, 4D takes care of the HTTP communication, i.e. the developer defines the body content of the HTTP communication and lets 4D to worry about forming the header. Whereas **WEB SEND RAW DATA** gives the developer the possibility of defining not only the body but also the HTTP header in one line. This is useful to do a fine control of the HTTP communication, for example it allows sending chunked responses.

WEB SEND HTTP REDIRECT

It redirects the request of the client to another URL. For example you may have noticed that after sending a registration form in the web you are redirected to a welcome page, or if your session in a website times out you are redirected to the main page. Examples.

After the registration of a contact in a webpage the client is redirected to the thanks page

```
CREATE RECORD([Contacts])
[Contacts]Firstname:=firstName
[Contacts]Lastname:=lastName
[Contacts]City:=city
[Contacts]Email:=emailAddress
SAVE RECORD([Contacts])
WEB SEND HTTP REDIRECT("/thanks.html")
```

Redirection to another website

```
WEB SEND HTTP REDIRECT("http://www.othersite.com/main.html")
```

On Web Connection

Another way of making requests to the 4D Web Server is by using customized URLs that are processed by the **On Web Connection** database method. So whenever the request to the web server is not done to a file or a call to a method with 4DACTION that request is sent to the **On Web Connection** database method.

This method receives 6 parameters from the web

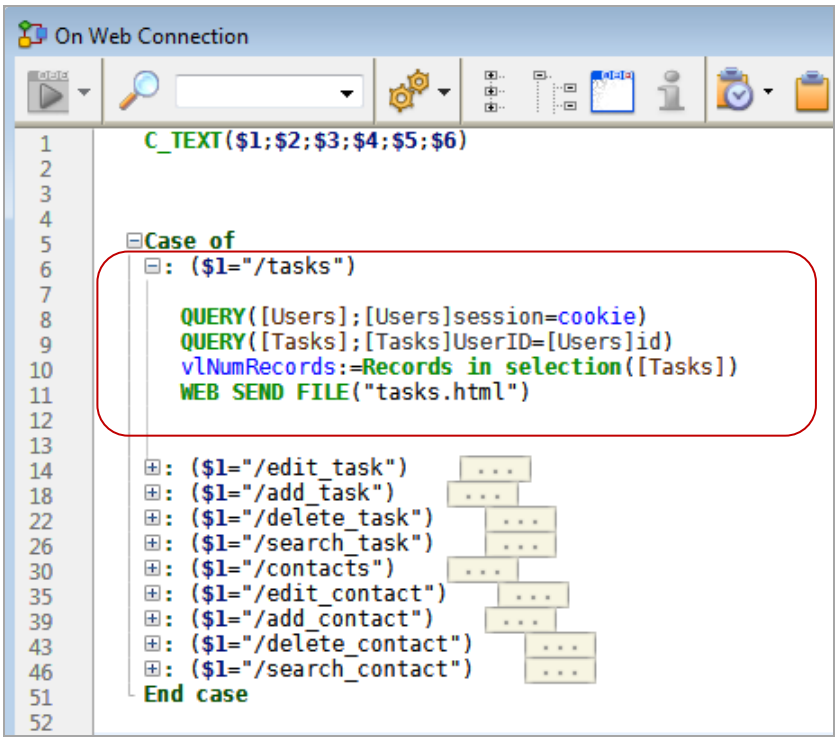
- $1: requested URL
- $2: HTTP header + HTTP body (up to 32 kb)
- $3: IP address of the web client
- $4: IP address of the server
- $5: User name
- $6: Password

We will focus in the first parameter, the URL, based on this parameter we can decide what information should be sent or what actions should be done. All this it is done in a centralized manner, which gives the developer fine control over what is going on with the web server.

The advantage of using the On Web Connection approach is that the developer customizes the URL and gets rid of the 4DACTION part of the URL. If we want a list of tasks we can pass a very simple URL like:

http://127.0.0.1/tasks

And then in the On Web Connection method we decide what to do analyzing the **$1** parameter, in this case we create a selection of records for the table TASKS and then we send a previously created HTML template that contains 4D tags.



SESSION MANAGEMENT

Web session management is very important for Web Applications especially with today's security risk and requirements for specific user experience. The problem is the implementation is never easy and often slow due to the tasks required to maintain each session. Don't worry, the new 4D v13 HTTP Server has you covered! There is no need to redo your queries, rebuild selections, or repopulate your variables every time 4D receives a web request. All web session and context data is maintained for you automatically. It's like having the database in the front-end.

HTTP Overview

The nature of HTTP is that it has no state. Every time a client sends a request, HTTP has no idea of the caller's identity. It simply forwards the request to the Web server. The protocol simplifies the server design because there is no need to dynamically allocate storage to deal with conversations in progress. If a client dies in mid-transaction, no part of the system needs to be responsible for cleaning the present state of the server.

All HTTP request initiates from an HTTP client. Each request is handled by

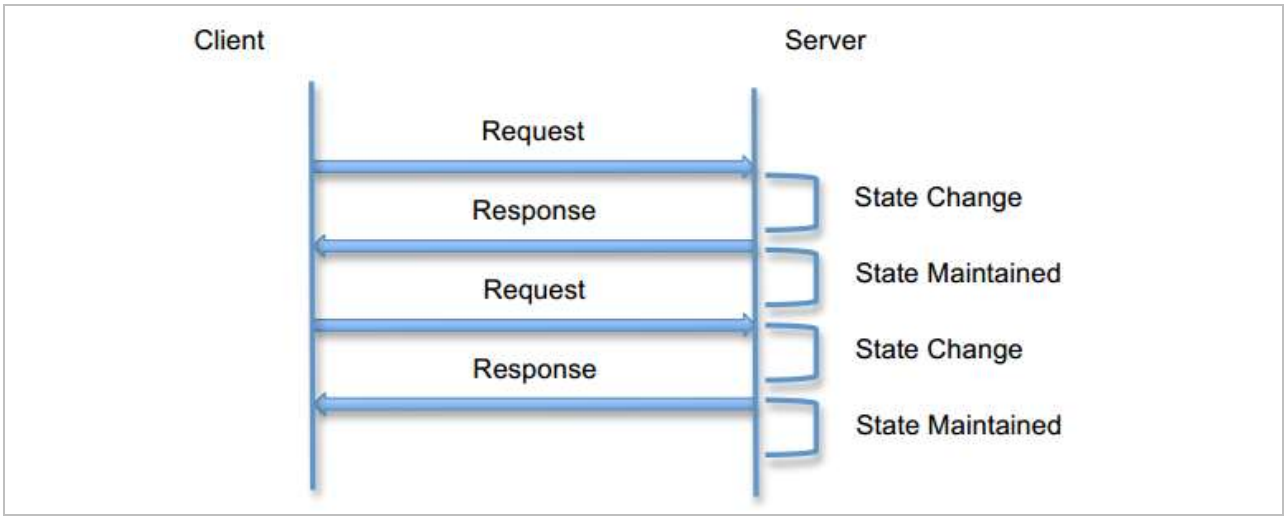
- Open a connection to the Web Server
- Download the document
- Drop the connection

A simple example of a stateless connection is a request for a static web document, which is very common to have a URL that point to a specific document on a Web Sever.

In most cases, stateless communication fulfills many requirements of Web site design. However, the combination of specific security and user interaction requires an adoption of another methodology. This methodology is called session management.

Generally, session management is a way to manage and maintain tasks in a given environment. It includes keeping track of which applications are open and which documents each application has opened. This tracking allows the same state to be restored when the user to revisit.

In the Web Serving context, session management is a way to overcome the stateless nature of HTTP protocol. Web developers use this technique to track, maintain and restore user's recent activities and state. The tracked information is stored on the server-side between HTTP requests.



The typical uses of session management are in a form of Web application: shopping cart, online banking, web access emails or any other type of web application that requires user identification.

What Session Management tries to address

The session management provides an efficient way for end-users to interact with a Web application. It transparently authorizes a user for every HTTP request without having the user repeatedly login. The use of session managements often results in the access speed increase, better user interaction and better navigational interface.

The session management is sometime responsible for reducing the number of tasks on the server-side.

Because session management is all about maintaining and reusing what was already done, the Web Server can really gain back its processing power and maintain its resource for other area of the application.

Client / Server Analogy

In client and server, the connection is persistent and process can be created for performing specific task. Each process created on the client application maintains its own memory space and it is used to store relevant information used and state within the process. This relevant information is considered temporary data. As soon as the process is terminated, the memory space is cleared and returned to the system.

In a client and server environment such as in 4D, database access is the most common operation. Creating and maintaining a data selection takes processing time. When a data selection is loaded into a process, it remains there through the life of the process.

When the client application request for a selection from the server, the selection is created and loaded into the process. This process can be thought of as a session. The state of the session is simply the result of what is done to the selection such as ordering, current record, selected record and etc.

Web Application Analogy

In Web application, the session management is done differently. But the overall goal is the still the same: creating a temporary storage of what had happened and making it accessible to the next HTTP request of the same client.

Because Web Application still follows the basic rule of HTTP protocol, the communication is still initiated from the HTTP client. The initial request to the server always includes the user (or client) identification.

The server interprets the identification and a session id is generated and sent back to the client.

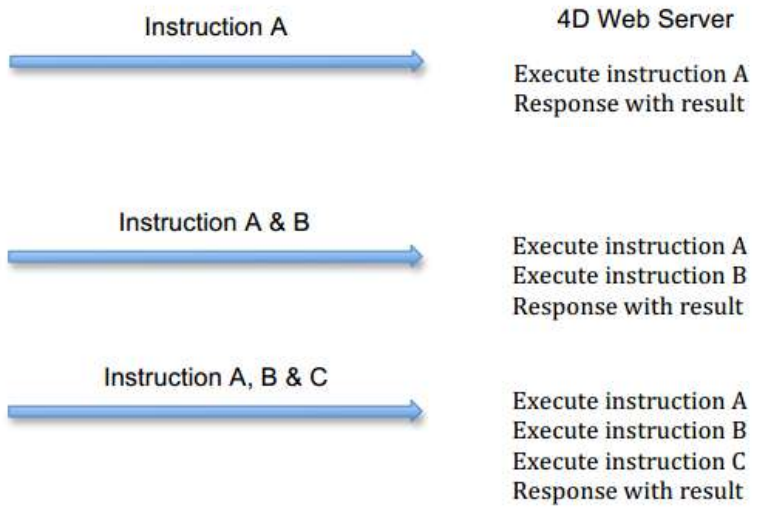
The session ID can be sent back to the client in 2 ways:

- Part of the response body
- Part of the HTTP header (Cookie).

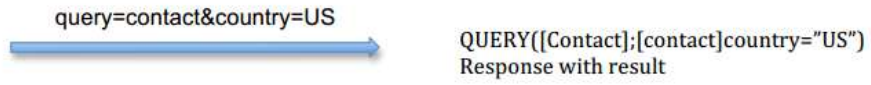
The use of session id allows the server to identify if an HTTP client has already established a communication with the server already. If so, the server will make the previously maintained state of the same HTTP client accessible for the current request. Any changes to the current state will be maintained and made available for the next request.

Session Management Implementation

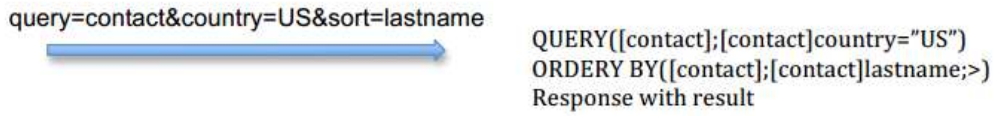
When implementing session management in 4D, there are a few options that you can choose to follow. However, it is never an easy and a straightforward design. Often time, the implementation requires a lot of variable maintenance, saving and restoring selection from scratch. To have a better understanding, let’s examine the non-session-management handling approach.



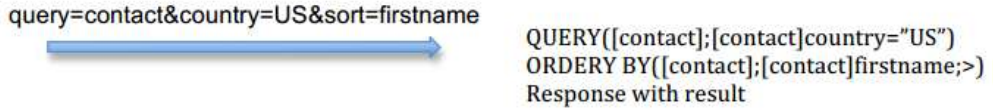
The problem with the non-session-management is that every request is considered unique. The same operation was done previously must be done all over again. Suppose a request is made to 4D Web Server to get a report of the contact names that live in the United States. In a simple interpretation, the client request and server response may look like this:



While the current result set is being displayed, the user wants to view the current selection in the sorted view on contact last name. The instruction may be resent to the Web server as shown below:



Suppose the user wants to do view the sorted selection on contact first name instead. A full operation request must be sent to the server again:



Because the selection is built and removed for every request, the Server is redoing a lot of work that it has previously done. For the operation such as query, the execution time can be high depending on the size of the data in the table. Therefore it is important to avoid task repetition as much as possible. The solution to this problem is to execute once and reuse the result again.

In 4D database, the result of the execution can be maintained in many ways. Some are maintained automatically and some requires additional implementation to maintain the state of execution. There are number of things that can be saved or maintained starting from a generated selection, a visited record, a set of populated process variables and other part of the application that accessible to the Web process.

Without going through the trouble of creating your own session management, 4D v13 offers an automatic session management for all HTTP request. This new session management is intended to address many areas that 4D developers must deal with.

New HTTP session management

4D v13 offers the easiest way in integrating a Web session management into a Web application design. The feature is intended to address many issues and restriction imposed by the standard HTTP protocol without requiring a huge amount of coding from the developer.

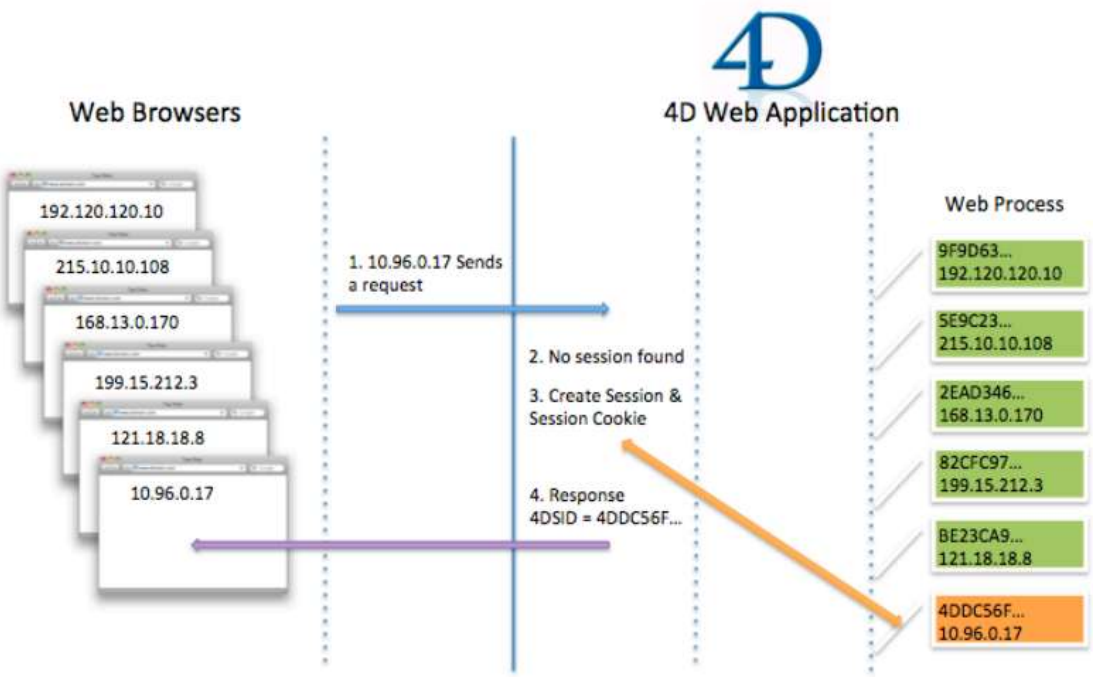
In the past, the automatic form of session management is the contextual mode Web serving. However, the contextual mode has a lot of restrictions and can be quite difficult to get around. With the new HTTP Server implementation,

4D v13 takes the concept of the old Web contextual mode and redesign it so that the session management can be used freely and efficiently without restriction.

In non-contextual mode, when an HTTP request is made to the 4D Web server, a temporary process is created on the server side to handle the request. Once a response has been made, the process is terminated. In 4D v13, all Web process is create to handle a Web session. Each session has a timeout period. It means a process that was created for an HTTP client stays alive until the timeout period has elapsed. If the new request from same HTTP client is made to the server before the session has expired, 4D will route the request to the correct process that is maintains the session. From there, all process variables, sets and selections that were created in the previous request will once again be accessible to the current request.

Session ID

The key to the new HTTP session management is session ID. By default, 4D v13 will assign a UUID as session ID to every new HTTP connection. When an HTTP connection is received, 4D immediately checks if the request header contains a cookie named 4DSID. If so, the value of 4DSID will be checked against the name of exist processes. If a process is found, then 4D routes all execution for the current request to that process. If the name of the process is not found nor the cookie named 4DSID does not exist, it is assumed that the connection is new and a new session ID will be generated for it.

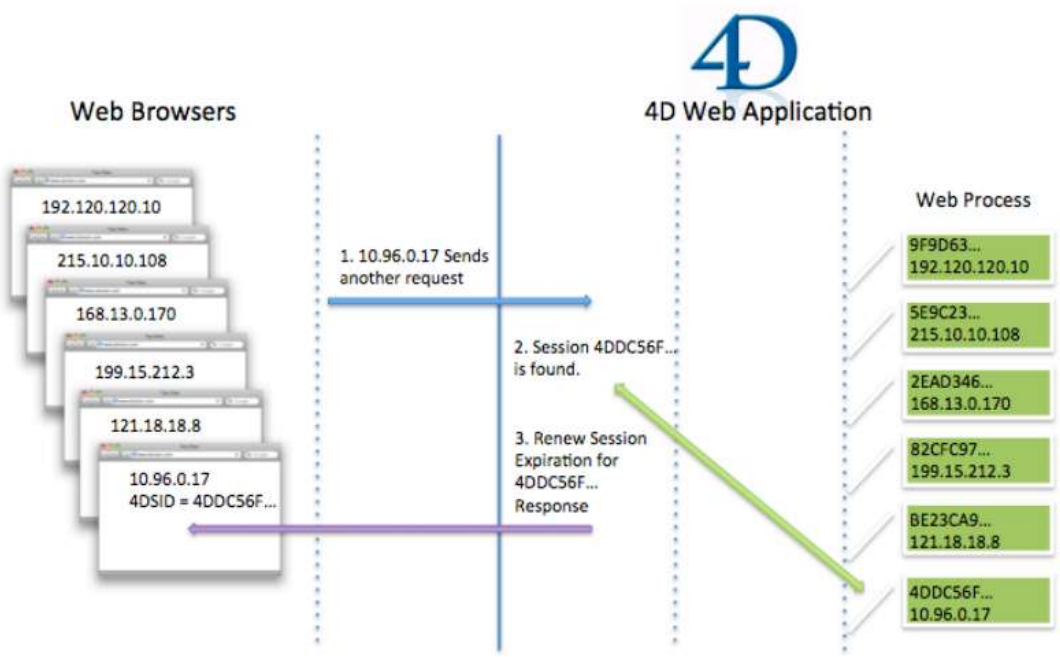


The above diagram illustrates how 4D v13 handles a new HTTP connection.

1. A new connection is made from 10.96.0.17.

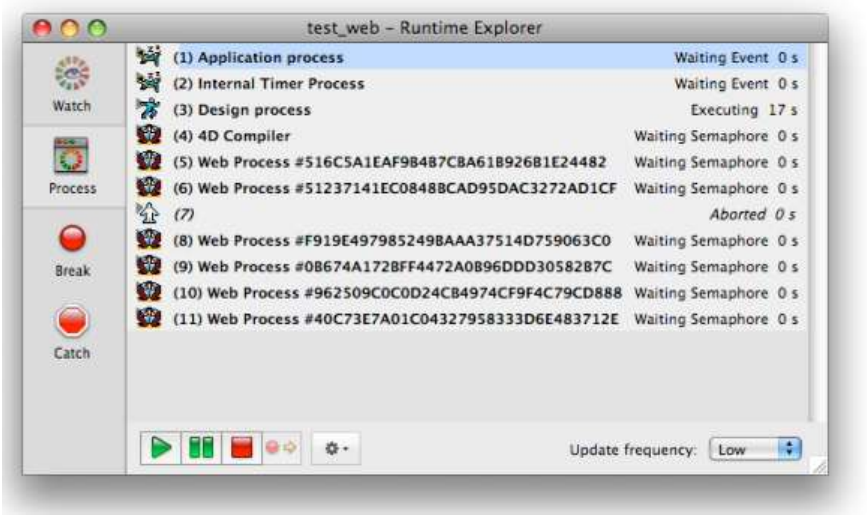
2. The request header does not include the session id. Therefore 4D will immediately assume that the connection is new.
3. 4D create a session ID (UUID) and use it to create a Web process to handle the session.
4. Before the Web Server sends the HTTP response, the session ID is set to the cookie named 4DSID and sent along as part of the response header.

The following diagram illustrates how 4D v13 handles a connection made from an HTTP request with valid session information.



1. Another request is made from the same IP address 10.96.0.17.
2. 4D found the Web process for the given session ID and routed all execution to be done within the process.
3. The session expiration is extended before a response is made.

Here is an example of the Web session processes listed in the Runtime Explorer.



Session Validation

There are 2 pieces of information in a request that 4D uses to identify a session.

1. IP Address of the Client (Web Browser)
2. Session ID (Cookie named 4DSID)

In order for 4D Web Server to route the execution scope to the right session, the above two values must be validated to one of the previously generate sessions. If the request has a valid session id but the IP address is different from the last request, 4D recognizes it as a security thread and returns a 400 error. This assumption is that each session ID is guaranteed to be unique and it can be assigned one IP address only. If an unidentified IP is making a request with a known session ID, it will be rejected.

Determining the current session ID

It is possible that a certain implementation requires its own tracking of the session ID. As stated earlier, each HTTP connection gets a unique UUID assigned to it. This ID can be retrieve through the command named **WEB Get Current Session ID**. The successfully retrieve the session ID, the command **WEB Get Current Session ID** can only be used within the scope of Web process only.

Renaming the Session Cookie

By default, the name of the session cookie is 4DSID. For a new Web implementation, keeping the default name is recommended. But it is possible that the same term may already be used. If that is the case, renaming is possible through the use of the command **WEB SET OPTION**.

```
WEB SET OPTION (Webserver session cookie name;"SESSID")
```

Once the command is executed, all subsequent Web connections will have their session ID set to the cookie named SESSID. To restore the default name (4DSID), pass an empty string in the 2nd parameter.

Set maximum number of concurrent sessions

The default maximum number of concurrent sessions is 20. This number may be changes due to the implementation and available resource such as memory and CPU on particular machine. When the maximum number is reached and a new session needs to be created, 4D will terminate the old session and allow the new session to be created.

To set this value, execute the command WEB SET OPTION with Webserver max sessions constant in the first parameter. Here is an example of a call that set the maximum concurrent sessions is 50.

```
WEB SET OPTION (Webserver max sessions;50)
```

To restore the default number of maximum session, execute the same command with 0 passed as 2nd parameter.

Set Session Timeout

The session timeout setting is intended to regain the resources that may no longer use. This allows 4D to recycle the memory more efficiently.

The session timeout consists of 2 parts:

1. Session cookie (4DSID) timeout (default at 24 hours)
2. Session process timeout (default at 2 minutes)

Session cookie timeout

When a new session cookie is created for a session, 4D set the expiration period to 24 hours ahead of the current time. This means that if a request is made from the same Web Browser within the

next 24 hours, the same session cookie will be sent as part of the header request to the HTTP server.

To set a new session cookie timeout value, execute the command **WEB SET OPTION** with the constant Webserver inactive session timeout. Here is an example of how to set web session timeout to 10 minutes.

```
WEB SET OPTION (Webserver inactive session timeout;10)
```

Session process timeout

The session process timeout applies only to a process that is created to handle a Web session. If the process is no activity within the duration of the timeout period, the process will be terminated immediately. The default timeout period is set to 2 minutes. This is obviously much different than the session cookie timeout. Please refer to the later section about maintaining critical states and information for more discussion about the difference of these 2 default settings.

To set a new session process timeout value, execute the command **WEB SET OPTION** with the constant Webserver inactive process timeout. Here is an example of how to set web session timeout to 10 minutes.

```
WEB SET OPTION (Webserver inactive process timeout;10)
```

Deactivate 4Dv13 HTTP SessionManagement

Although the new HTTP session management is the default web handling option in 4D v13, it is not mandatory. For an application that does not require session management or an upgraded application that needs to maintain its behavior, the session management can be deactivated from the language.

To deactivate this feature, call the command **WEB SET OPTION** with the constant Webserver keep session. Here is an example of how to deactivate the feature.

```
WEB SET OPTION (Webserver Keep session;0)
```

Passing 0 in the 2nd parameter simply tells 4D to terminate the Web process as soon as it finishes its execution.

Passing 1 is to activate the feature.

Please note that the above execution affects only all subsequent Web connections. For all sessions that are created prior, remains unaffected by the command execution.

Force Web process termination

When a Web process is created and maintained in the background, it is not possible to gain access of that process without making another HTTP request to the server. If the process is stuck, used up too many resources or simply needed to be shutdown, it can be done by calling the **WEB CLOSE SESSION** command.

WEB CLOSE SESSION command takes a session ID as input parameter. If a web process whose name is the same as the given session id is found, 4D terminates the process immediately. Here is an example:

```
WEB CLOSE SESSION ("60DE88178BEC4C97B598F7B74F54694F")
```

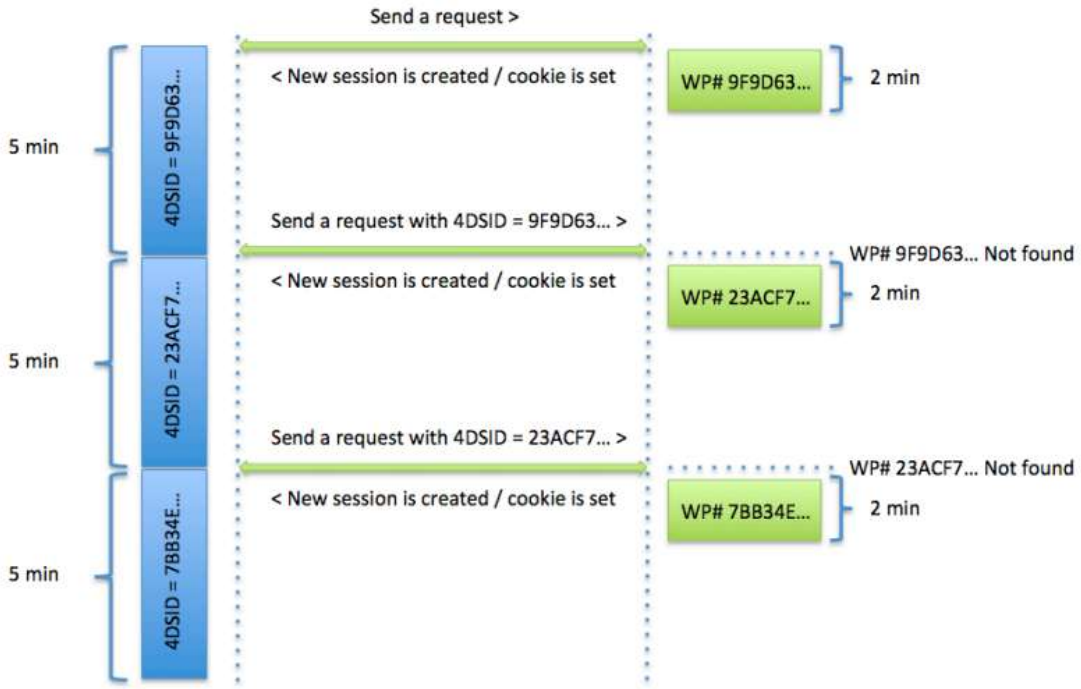
While in a Web process, the current session id can be retrieved by calling the command **WEB Get Current Session ID**. The ID can be saved in an inter-process variable (array) that other process do have access to.

Maintaining critical states or information

When the Web process is terminated, 4D frees up the memory. The information that was saved in the memory such as local variables, process variables, selections or sets is wiped out clean. In a certain user interface design, it is necessary that some of those information and data state is restorable when the user revisiting the Web site. This means that the information about the state must be saved before the Web process is terminated. When the same user is accessing the Web application for the next time, the previously saved state can be restored, allowing the user to continue from where he or she had left off. As part of the Web session management feature, there is a new database method created to accommodate such requirement. The name of the method is **On Web Session Suspend**.

The method On Web session suspend is executed in the same process as the Web process that is going to be terminated. When 4D terminating a Web process, it tells the process to execute the method **On Web Session Suspend**. Because the execution happens within the context of the Web process, all information such as variables, selections or sets is still accessible and savable during this time.

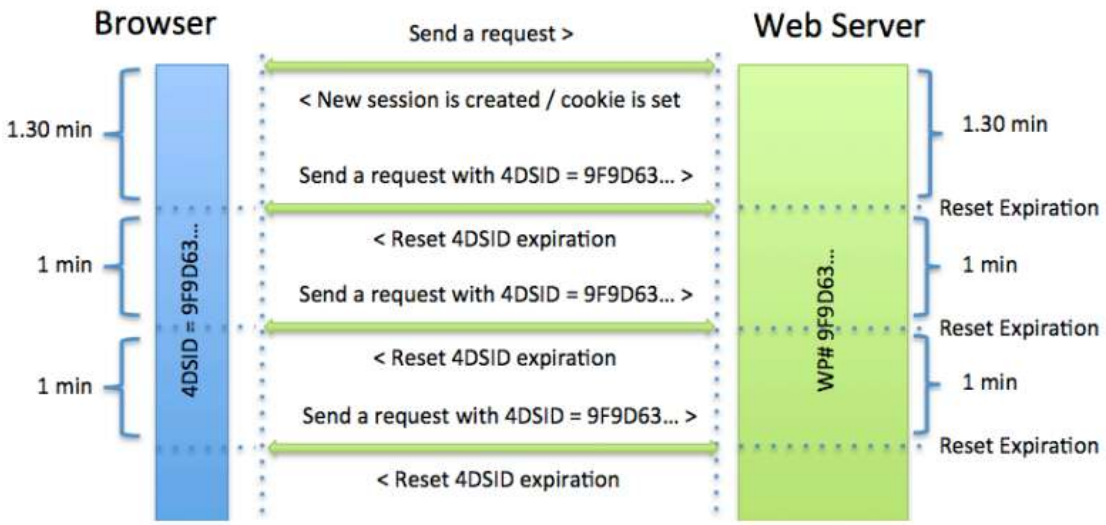
For better understanding, let's have a look at an example of a session timeout on both ends when a request is made after the session process timeout has expired.



The above diagram shows how 4D manages the web sessions when the timeout has elapsed. When an initial HTTP request is sent to the Web server, 4D creates a session process whose life span is 2 minutes if there is no activity. A response is made to the browser with a 4DSID cookie with the life span of 24 hours.

After 2 minutes of inactivity, 4D terminates the process. 5 minutes later another request is sent to the server with the last set cookie (4DSID = 9F9D63…). Because 4D has already terminated the web process, a new process is created and a new cookie is sent back to the browser to replace the old one. As long as a future request is not made within the 2 minutes timeout of the session process, the same exact behavior will be expected.

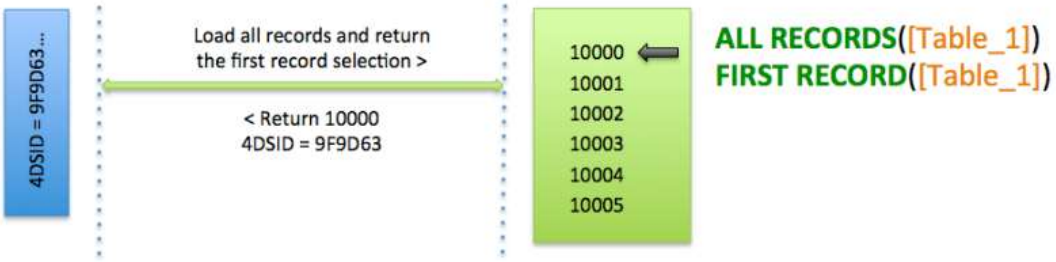
Here is another look of how session timeout is managed for a very active session.



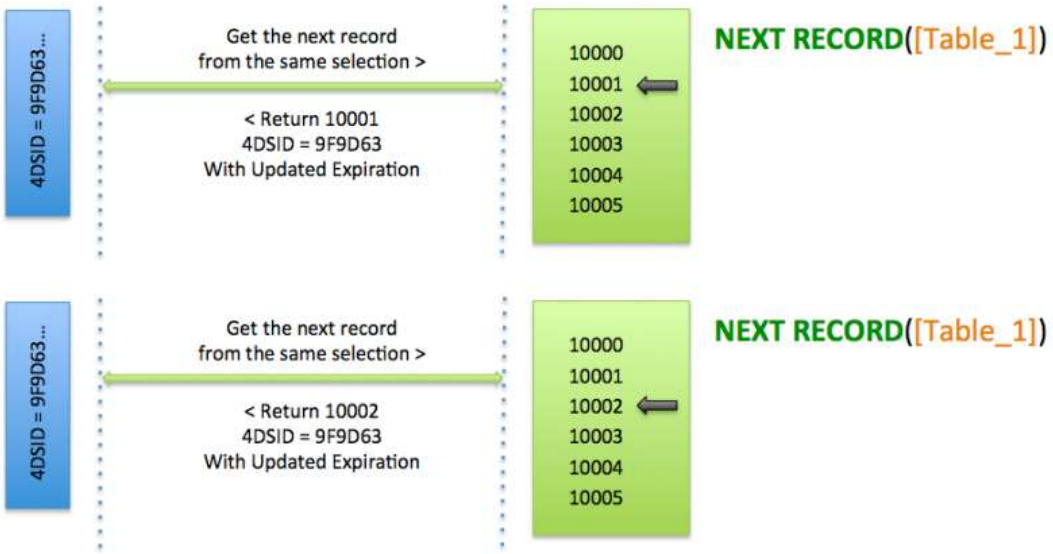
When a request is made to the Server before the timeout has expired, 4D set the next timeout of the session to Now+2 minutes. For the 4DSID, the expiration is set to Now+24 hours. However, in real world situation the length between one HTTP request to another is always unpredictable. Counting on keeping one web process running indefinitely is also not feasible. Therefore, the developer must also take on a role in saving the current state of the session before the Web process is terminated.

As described earlier, a new method **named On Web Session Suspend** is created to accommodate this task.

Suppose a request is made to 4D to load all records and return the first record as the result. The simple communication diagram may look like this.

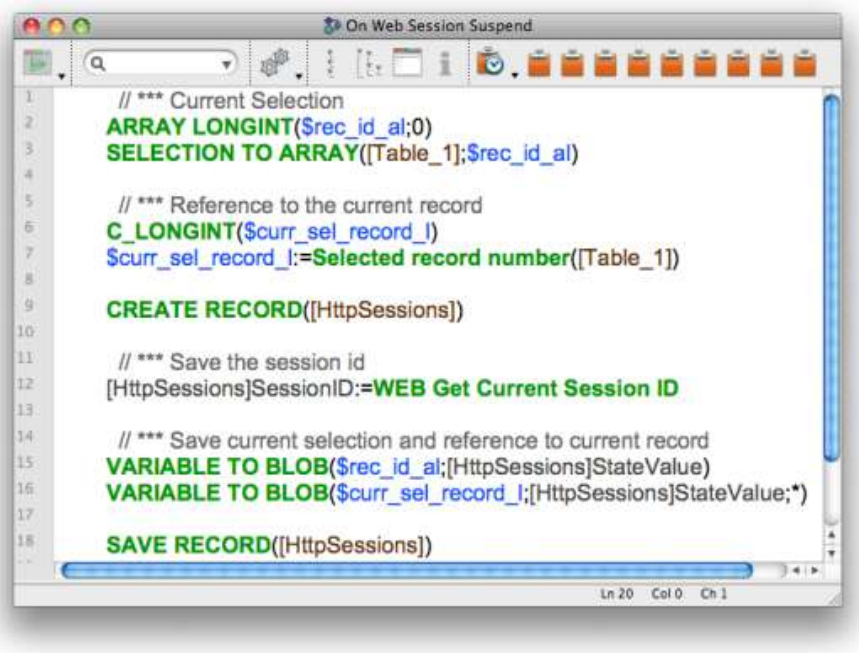


Suppose the next request is made to get the next record from the same selection before the 2 minutes timeout has elapsed. Because the Web process maintains the selection, the command NEXT RECORD can be executed to make the next record the current record.



As long as the next request is made before the timeout is expired, there is no need for 4d to repopulate the selection for the user nor requires additional instruction to locate the previous, current or the next record.

However, this will not always be the case. Suppose the Web process is going to be terminated due to the lack of activity within the 2-minute timeout. A procedure can be added to the **On Web Session Suspend** method to save the current state. The following example is one of many approaches in dealing with this situation.



Since the method **On Web Session Suspend** is executed before the process is terminated, the selection and the reference to the current record in the selection can be saved into a temporary table. This temporary table is used only to store the current state of the session that about to expire. When a request with the same cookie (4DSID) is sent to the server, the saved state can be restored. Here is an example code in the **On Web Connection** method on how the saved session is restored when the incoming request is attempting to reestablish the last session state.

Please note that it is prohibited to execute a command or method requires user input or interaction (such as ALERT, CONFIRM, DIALOG and etc.) in the ***On Web Session Suspend method****.*

```
C_TEXT($current_cookie_t)

$current_cookie_t:=UTIL_WEB_GetCookie ("4DSID")
If ($current_cookie_t#"") & ($current_cookie_t#WEB Get Current Session ID)

  C_LONGINT($offset_l;$curr_sel_record_l)
  ARRAY LONGINT($rec_id_al;0)

   // *** Restore the session if still available
  QUERY([HttpSessions];[HttpSessions]SessionID=$current_cookie_t)
  If (Records in selection([HttpSessions])>0)
    BLOB TO VARIABLE([HttpSessions]StateValue;$rec_id_al;$offset_l)
    BLOB TO VARIABLE([HttpSessions]StateValue;$curr_sel_record_l)
    DELETE RECORD([HttpSessions])  // *** No longer needed
  End if

   // *** Load the selection
  CREATE SELECTION FROM ARRAY([Table_1];$rec_id_al)

   // *** Set the current record
  GOTO SELECTED RECORD([Table_1];$curr_sel_record_l)

End if
```

Note: The method ***UTIL_WEB_GetCookie*** is available at: http://kb.4d.com/search/assetid=76339.

Given that the session 9F9D63 expired and the same HTTP client is making another HTTP request to navigate to the next record, the **On Web Connection method** will handle this request by first locating the session 9F9D63 in the temporary table. If the session is found, the selection will be restored. Once the selection is completely restored, the **NEXT RECORD** command can be executed on the selection.

Although the above implementation allows 4D to restore the session state, it is important to understand that the current session is not 9F9D63. 4D automatically issue a new process to handle the session and the 4DSID cookie is set to a new value.

CSS - CASCADING STYLE SHEETS

Cascading Style Sheets are in charge of giving the look and feel of the user interface. With CSS you can define the following properties of an HTML region:

- Font: type, size, color…
- Backgrounds
- Padding, margins
- Positions
- Transitions
- Round corners
- Shadows
- Scrolls
- Opacity
- Gradients
- Skew
- Text in columns

CSS Types

The style can be defined in three ways.

- Inline
- Internal
- External file

Inline CSS

When the CSS is defined as an attribute in the tag.

```
<p style="color:#336699;text-align:center;padding:30px;">Contacts</p>
```

Internal CSS

When the CSS is not defined in the tag, but within the HTML file in the <style> tag

```
<style>
body {
       background-color:#CCCCCC;
       font-family:Arial, Helvetica, sans-serif;}
a {
       color:#639CBF;
       text-decoration:none;}
</style>
```

External CSS file

When the CSS is defined in an external file and it is linked to the HMTL file as follows:

```
<link rel="stylesheet" href="/css/style.css" type="text/css">
```

This is the recommended way in order to have a complete separation between the content and the presentation. However there are cases when it is practical to define inline CSS or internal CSS.

There is a hierarchy in the way styles are applied to an HTML region.

- Inline CSS overrides Internal CSS
- Inline CSS overrides External CSS

- Internal CSS overrides External CSS

How does it look like?

A CSS file looks like the following code

```
body {
       background-color:#CCCCCC;
       font-family:Arial, Helvetica, sans-serif;
}
a {
       color:#639CBF;
       text-decoration:none;
}
header h1 {
       text-shadow: 1px 1px 10px #ddbbbb;
       width:200px;
       display:inline;
}
#cover {
       float:left;
       padding:4px;
}
img{
       padding-right:10px;
       border:0px;
}
```

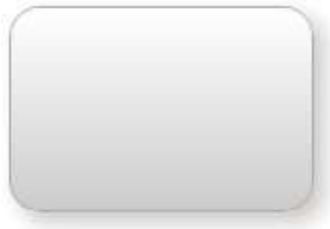
This code defines that the background color is grey, that the default font used for that document is Arial or Helvetica. It defines the color and decoration of the links, the padding and border of all the images and the float property and padding for the region identified as cover.

The advantages is that a single CSS file can be used by many HTML files, therefore it unifies the look and feel for all those files. That is why it is important to use external CSS files instead of defining the styles inline or internally.

CSS3

With the introduction of HTML5 comes CSS3 which gives new possibilities for defining the interface look and feel because thanks to CSS3, Web Applications can look like Desktop Applications because it supports, round rectangles, transitions, shadows, gradients opacity and some other nice treats.

Some CSS3 Examples



```
#rect1 {
 height:100px;
 width:150px;
 border: 1px solid #aaaaaa;
 -webkit-border-radius: 15px;
 background: -webkit-gradient(linear, left top,  left
bottom,color-stop(0%,#ffffff), color-
stop(100%,#cccccc));
 box-shadow: rgba(40, 20, 0, 0.25) 5px 5px 15px;
}
```

```
#button1 {
 padding: 6px;
 width:150px;
 text-shadow: 0 1px 0 #FFFFFF;
 text-align:center;
 -webkit-border-radius: 10px;
 background: -webkit-gradient(linear, left top,  left
bottom,color-stop(0%,#ffffff), color-
stop(100%,#666666));
 -webkit-box-shadow: 3px 3px 5px #999999;
```

	`}`

Row 1
Row 2
Row 3
Row 4
Row 5
	`.row:nth-child(even) {`
	`background: #eee;}`
	
	`.row:nth-child(odd) {`
	`background: #cce;}`

CSS3 demos	`h1{`
	`-webkit-transform: rotate(-10deg);`
	`}`

Use any font	`@font-face {`
	`font-family: 'Magneto';`
	`src: url(Magneto.TTF);}`

Antialiasing	`h3 {`
	`-webkit-font-smoothing: subpixel-antialiased;`
	`-webkit-text-stroke: .10pt black;`
	`}`

Reflection	`#reflect {`
	`-webkit-box-reflect: below 0px`
	`  -webkit-gradient(linear, left top, left bottom,`
	`from(transparent), to(rgba(255, 255, 255, 0.46)));`
	`}`

JAVASCRIPT

JavaScript is an interpreted language that runs on the browser but also runs on the server side in technologies like Wakanda. This object-oriented language is very popular because it is widespread in all modern browsers.

The language is easy to learn, versatile, loosely typed and it is crucial for Web Developers.

What does JavaScript do?

JavaScript has many responsibilities in a Web Application. The main tasks are:

- Event handling
  - Click, hover, form submission, error messages
- Data validation
- DOM manipulation
  - DOM – Document Object Model
- Communication with the server
  - AJAX requests
  - Parsing of JSON or XML files

JavaScript Libraries

To make the job of the developer easier there are a lot of libraries and frameworks that serve different purposes. Here it is a non-extensive list.

- jQuery
  - Most popular

    - Plenty of plugins
- YUI
    - Yahoo User Interface
- Sencha
    - Great for data grids
- Dojo
- Mootools
- Prototype

Learn JavaScript

There are many websites and books from which you can learn JavaScript. One of the most important is the JavaScript guide from the Mozilla Developer Network.

https://developer.mozilla.org/en-US/docs/JavaScript/Guide

A WEB DEVELOPMENT TALE - EPISODE 1 - GO WEB!

This document describes step-by-step how to build the application presented in the preClass. In order to facilitate the understanding, the code may vary from the final application, some features may be missing. This description focuses on concepts and how to use them so that you can make it by yourself.

To make this journey more pleasant, we chose to follow the collaborative work of two people, Joe and Sue.

Joe is an experimented 4D Developer mastering database development with v12. Joe is editing a solution in the healthcare domain. From sometimes now his clients are asking him to get web versions of the application. But Joe has always been reluctant going to the Web. Even if he knows the basics of static and dynamic page generation in 4D, he is not comfortable at all with html5 stuff and the so-called "Web 2.0" approach.

Recently the demand of the clients has even evolved towards supporting mobile devices like iPad or Android tablets. Joe knows from different readings that web technologies are well-supported on mobile devices. So, it seems he has no issue but learning to develop with 4D for the Web. But how to start?

Joe has a niece, Sue, which is very fond of new technologies and Web stuff. They both try to be agile in their development process and often debate on this topic.

Joe gets the idea of asking Sue to help him mastering Web development. He decides to learn-by-example and ask Sue if she is ok to help him building the web-equivalent of a simple task-list and address-book desktop application. Sue adores her uncle and accepts with enthusiasm to share her knowledge. So, they decide to start on next Saturday at Joe's home.

This is the record of their incremental development...

User-stories

Joe has prepared his business example. He introduces the user stories to Sue.

First, for my task-list:

#1 **Task Management**. As a member of Sue World Company (SWC), I want to keep track of the tasks on my to-do list so that I can check what has been done or not.

- Sue: isn't it a bit too large?

- Joe: Yes, let's refine it and split in two. In order to accomplish task management, I need to be able to do the following things.

  - #2 **Tasks CRUD**. As a member of SWC, I want to be able to create-retrieve-update-delete a task so that I can monitor it
  - A task has a title, a due and a start date, a status and is affected to an existing category.
  - #3 **Tasks filtering**. As a member of SWC, I want to be able to filter my tasks by categories so that I can know what I've got to do in these categories.

Now, for the address-book functions:

- #4 **Contacts CRUD**. As a member of SWC, I want to be able to create-update-delete a contact so that I can manage it.
- contact has Category, firstname, lastname, address, city, state, zip, country, mobilePhone, WorkPhone, HomePhone, email and photo informations.
- #5 **Contacts filtering**. As a member of SWC, I want to be able to filter my contacts by categories so that I can retrieve them more easily.

- Sue suggests adding a new story. How does the application know about the user? Let's add this one:

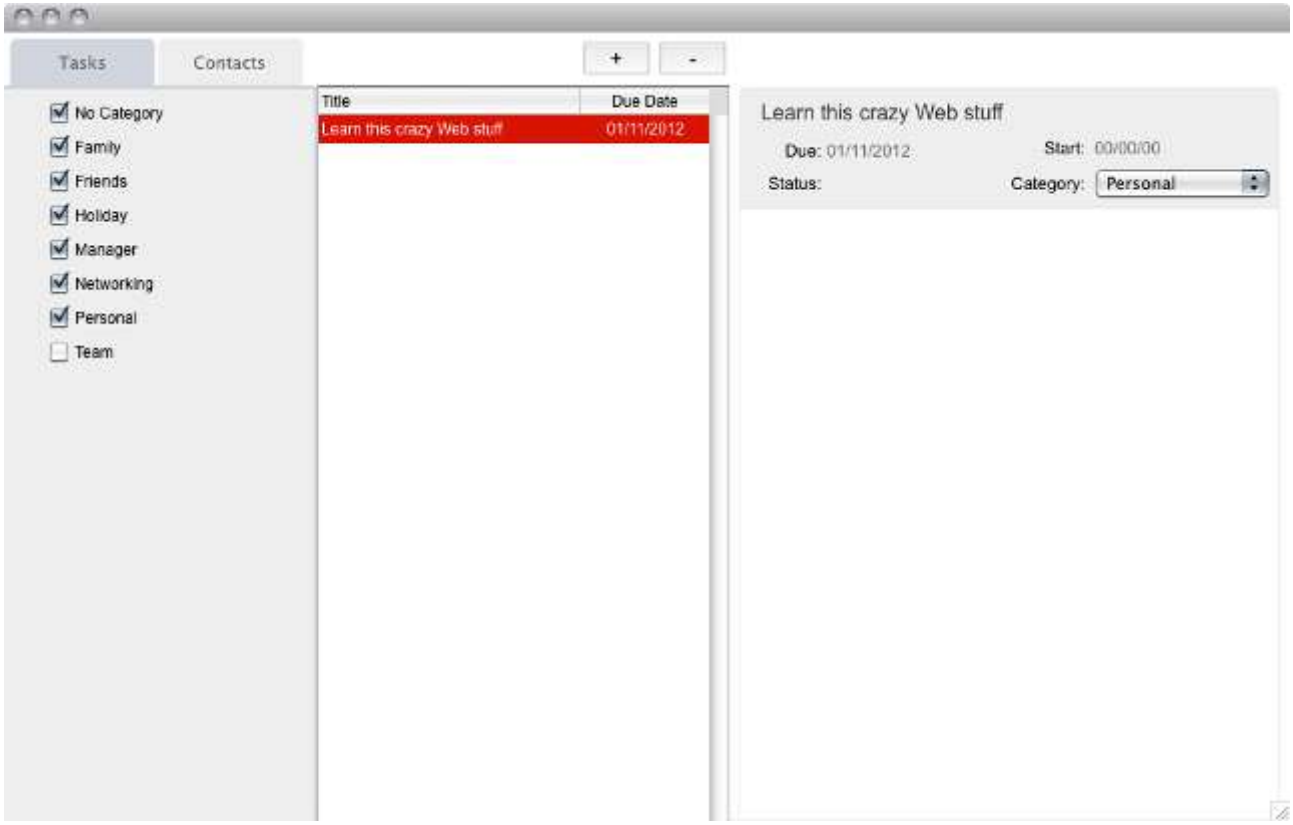
- #6 **User authentication**. As a member of SWC, I want to authenticate myself so that I can get access to my own list of tasks and contacts in order to avoid someone else looking at my private stuff.

Joe nods:

I simply forgot it! You're right...Let's add it. Ok, it seems ok now for starting working together. I will show you the 4D version.

Database example

Joe launches 4D v13 and opens the **swc** database package. The main screen displays:



- Joe: You see Sue it's easy for me in 4D to gather all the information in a single form. On web-side, seems to me that everything is page-based so I don't know how to get the same user experience...

Sue acknowledges:

- I suggest we add two new stories describing User experience (UX):

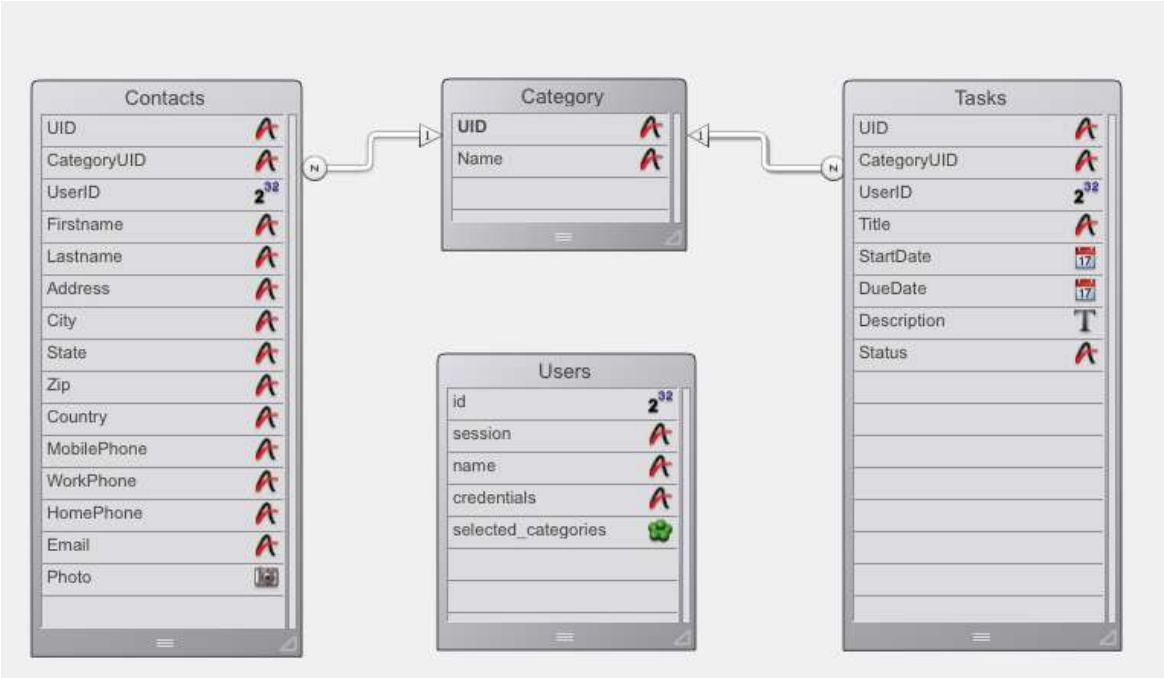
#7 ***Single-Page****. As a user of the application, I want to get all the information on a single page so that it remains easy and natural to use.*

#8 ***Filtering by categories****. As a user of the application, I want to be able to select categories just by checking the desired category in a list so that I can filter displayed items by checked categories*

Sue asks smiling:

- You know I'm as dummy in 4D as you are for the Web. How do you store and retrieve information? Kinda MySQL engine?

Joe explains that 4D has an integrated database engine and shows Sue the database structure for the example:



- Sue: what about the Web Server, is it integrated too?

- Joe: yes, everything is simply running out-of the box.

Joe:

- So, how do we start developing our WebApp now? Shall we design an html page? Which tool shall I use? How do I know when categories are filtered? How...

Sue laughs:

- stop, stop Joe! Too many questions at the same time! Let's simply start by analyzing the communication between the front and the back-end. Then, you will be able to work on the 4D back-end while I will start thinking about structuring the front-end. Proceeding this way, we won't lose time waiting each other.

- Joe: brilliant! Let's enumerate the exchanges. First, the user must authenticate, see #6: User authentication.

- Sue: I suggest we just set priorities of our stories. I think we can start with a single fixed-user and add the login feature later. Why not begin with Contacts management? I propose we just focus on a first version centered around #4: Contacts CRUD. If we can succeed in building this first step, we'll have a good basis for completing the application in a following iteration. And we will be satisfied to get something working asap.

- Joe: oh...seems like kind of “test-early, fail fast” agile mantra, doesn't it?

- Sue: indeed! Let's go?

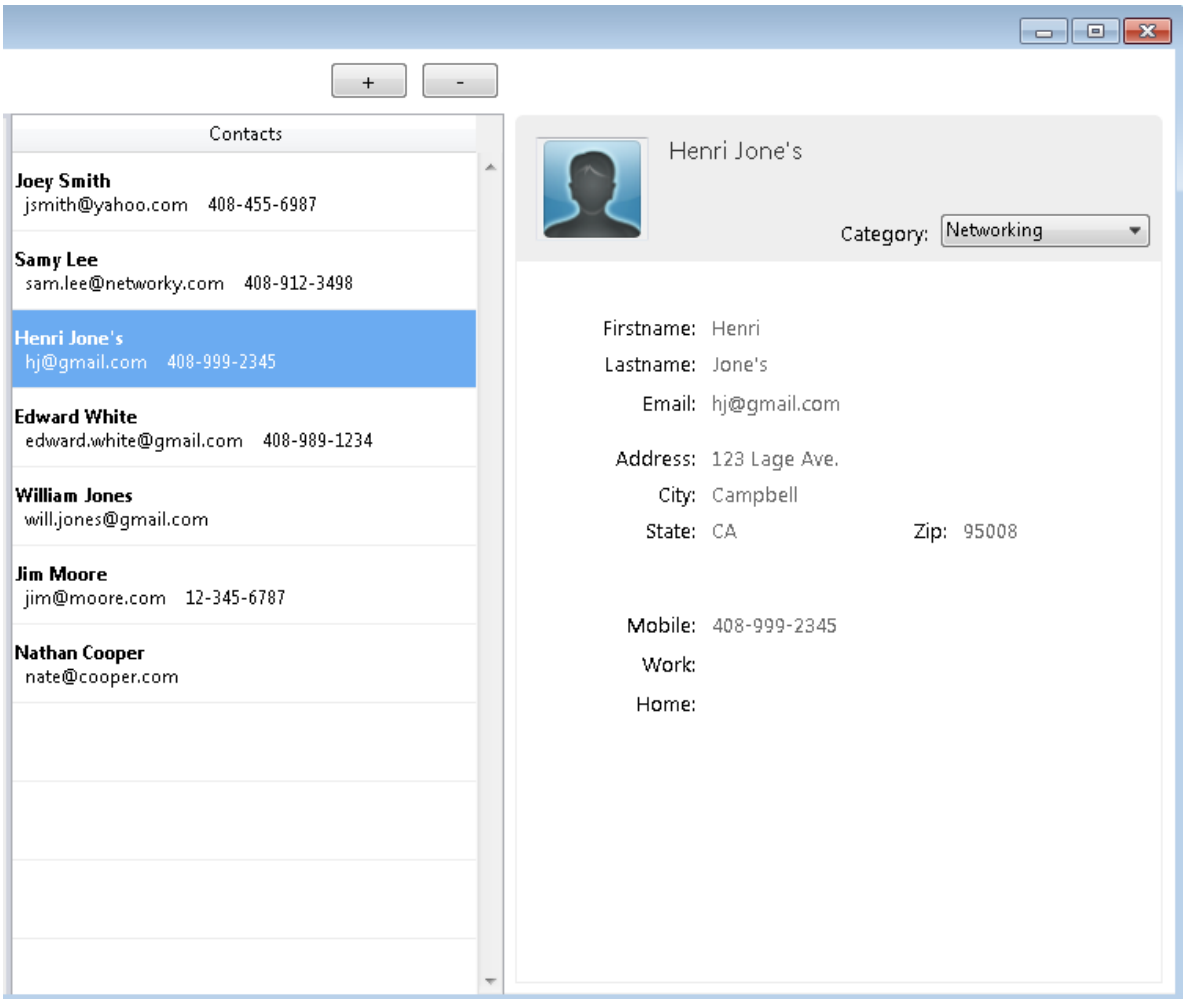
- Joe: okay! Fine!

Step1: Contacts List

Analyzing the communication

- Sue: So, the application must receive a collection of contacts in order to display them. Using a grid will be fine, what should be the columns?

- Joe: in fact, in 4D, I just use one-single column with rich-text for the name. But maybe it's too complicated for web display?



- Sue: that's ok, just send me these data.

- Joe: hmm...how?

- Sue: the client will send a GET request to the Web Server indicating which resource is requested. As we expect to get contacts, `GET /contacts` sounds logical to me! Do you see how you can handle this request?

- Joe: What shall I return an html page? This is just what I know how to do!

- Sue: fine, it's enough for the moment. Just prepare a web page containing an html list with the same information as in 4D. Then I will be able to check the request in the browser and if it's correct to display it in the application.

Building the Contacts html list

Joe usually uses a simple text editor to build his HTML page. He has found an efficient technique to quickly build dynamic pages. First, he simply writes a static html page corresponding to what he wants to build. He checks the page in the browser.

He just proceeds this way for creating "contactsList.html" static page.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN">
<html>
```

```
	<head>
		<title></title>
	</head>
	<body>
		<ul>
			<li uid="2E51E24F52454076B7B4A254791FA4F7">
				<span style="font-weight:bold">Nathan Cooper</span><br>
				nate@cooper.com  555-01-02
			</li>
			<li uid="E3BCF256B49F45A3ABFCE5F420217453">
				<span style="font-weight:bold">Samy Lee</span><br>
				sam.lee@networky.com  
			</li>
			<li uid="DCBB452DD8A148D3B944E9B8AD0AB3C4">
				<span style="font-weight:bold">Joey Smith</span><br>
				jsmith@yahoo.com  
			</li>
		</ul>
	</body>
</html>
```

The static result displayed in the browser:

- **Nathan Cooper**
  nate@cooper.com 555-01-02
- **Samy Lee**
  sam.lee@networky.com
- **Joey Smith**
  jsmith@yahoo.com

Using 4D Tags for dynamic pages

Then, Joe duplicates the file to "contactsList.shtml" dynamic page. The ".shtml" suffix will indicate to 4D that internal 4D tags should be processed before sending to the browser. The resulting file after replacing static values by 4D tags:

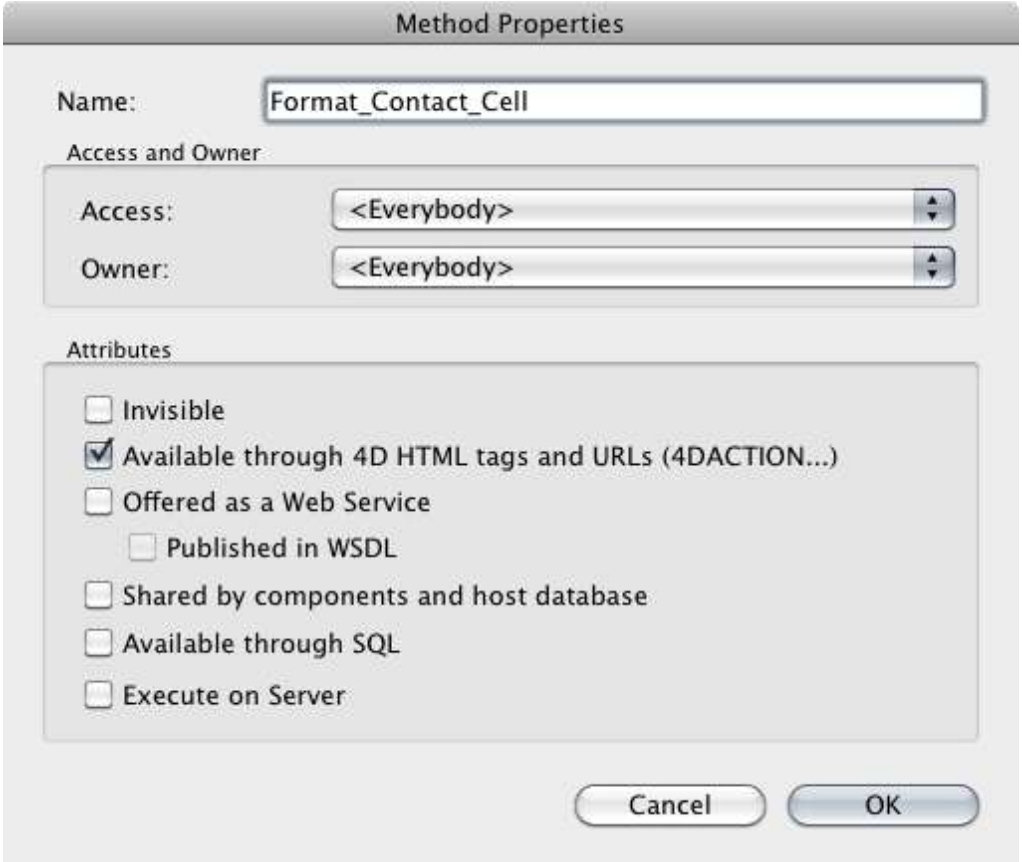
```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN">
<html>
	<head>
		<title></title>
	</head>
	<body><!--#4dscript/w_contacts-->
		<ul><!--#4dloop [Contacts]-->
			<li uid="<!--#4dtext [Contacts]UID-->"><!--
#4dscript/Format_Contact_Cell--></li>
		<!--#4dendloop--></ul>
	</body>
</html>
```

The following tags are used:

- 4DSCRIPT to call a 4D function, here the *w_contacts* function is called which simply builds a selection of all [Contacts].
- 4DLOOP and 4DENDLOOP to loop on the selection of [Contacts]
- 4DTEXT to insert the value of a [Contacts] field
- 4DSCRIPT to call an existing 4D method that builds and returns the correctly html formatted string for the current [Contacts] record.

Notes 4D:

- Remember that you need to allow the corresponding methods to be called by a web request like in the following example for *Format_Contact_Cell*:



- Please refer to the documentation for more information on 4D HTML Tags.

- In this example Joe is building upon an existing solution and re-using a 4D function to generate html. Sue could argue that this introduces a maintenance issue, if a web designer wants to modify the presentation; he can't because is generated internally in 4D. This is why it's preferable not generating html in 4D functions. Easy amelioration could be to replace 4D code by processing a chunk of html template using **PROCESS 4D TAGS** command.

Then Joe simply requests the dynamic page from the browser to check if the behavior is maintained. It works fine, so now it's up to Sue to use the provided stuff.

- Sue: I want to do several things with your list of Contacts:

- First of all, I want to load this list dynamically as a part of an existing page

- I want to display them a bit nicer
- Then when selecting a contact I want to be able to display all the information on the contacts in a form.

Introducing jQuery

- Sue: Let's address the first item. In order to do that, I will use jQuery.

- Joe: heard of that? What's for?

- Sue: jQuery is a very well-known JavaScript framework for the Web. First of all, it dramatically simplifies HTML DOM manipulation which is critical for managing dynamic interfaces. It also allows sending HTTP requests to the Server without reloading the whole page (that is called XmlHttpRequest (XHR)). Combining both features, the page can be designed as several modules which can live independently. The initial web page acts as a global container.

- Joe: that's seems interesting to me! Sounds like story #7 Single-Page!

- Sue: you've got it! And cherry on the cake, jQuery hides the differences in behavior between browsers so that you can write a single code and let it manage the compatibility under cover.

- Joe: Feel like I'm going to like this jQuery!

- Sue smiles: soon you'll see that life without it will sound impossible. Let's see how it works in this example I've prepared while you were finishing your backend request. The code is stored in swc1.html file. Read it together if you don't mind?

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN">
```

- **Sue:** above it is just a declaration that this is an html page

```
<html lang="en">
	<head>
		<meta charset="utf8">
		<title>
			SWC step1 - Contacts CRUD/List
		</title>

		<!-- Loading related stuff: CSS-->
		<link rel="stylesheet" media="all" type="text/css" href="/css/jquery-ui-
1.8.24.custom.css">
		<link rel="stylesheet" media="screen" type="text/css"
href="/css/screen.css">
```

- **Sue**: above, I just load some css that corresponds to this concern "I want to display the list a bit nicer". I will explain CSS later if we have time.

- **Joe**: for the moment, I'd rather focus on html and JavaScript

```
	</head>
	<body>
		<!-- declaring an empty item using a div-->
		<div id="contactList"></div>
```

- **Sue**: this one is VERY IMPORTANT. We don't load the contacts list immediately. Instead we just declare a placeholder using a DIV section. This div is identified by an id attribute. Remember that an id value MUST be unique throughout the whole document.

```
        <!-- Loading js scripts-->
        <script src="/js/jquery.js" type="text/javascript"></script>
```

- Sue: we always load JavaScript ONCE all other html stuff has been loaded. Here, we add jQuery library to the page.

```
        <!-- when we're sure the page is loaded, run the dynamic code-->
        <script type="text/javascript">
                     $(function(){       // on document ready
```

- Sue: the $() function does double duty as an alias for `$(document).ready()` if you pass it a function

```
                           console.log( 'ready!' );
                           // fills the contacts list
                           $.get("/data/ontacts", function(data){
$("#contactList").html(data) })

                     });// document ready
             </script>
</body>
</html>
```

- Sue: Dollar Sign is nothing but an alias for JQuery. Here, it allows to declare and run an anonymous function (without a name). Then we ask jQuery to send a request to the Web Server (XHR) and when receiving the response to replace the html content of the designed element ("#contactList") with the content of the response `data`.

- Joe: huh...

- Sue: I know, the last piece of js code may seem awkward the first time. Let's do it again slower...

$ means "jQuery". When you call the $() function and pass a selector to it, you create a new jQuery object.

$.get("/data/contacts" means "jQuery, call the server and ask him to get the Contacts resource".

`$.get("/data/contacts", function(data)` means "jQuery, call the server and ask him to get the Contacts resource. When you will have received the response in the data parameter, execute the code after."

`$("#contactList")` means "find the div whose id is contactList"

`$("#contactList").html(data)` means "find in the DOM tree representing the html page, the element whose id is 'contactList' and replace its html content with the value of the data parameter which is in fact the response of the Server"

This is the elegant and concise coding style pushed by jQuery.

- Joe mumbling: not sure, I find it elegant...

- Sue: just a matter of habit. You'll get used to it, you'll see. Compare the code with the explanation in plain English, looks shorter, no?

- Joe: hum, yes. But wait a minute: what about this “/data/contacts” resource on the Server? Is it my table?

- Sue: not really. In fact, you'll have to handle the request to this virtual resource on server-side and build a suitable response containing an html code that can be inserted into contactList DIV. I just suggest to use this "data" virtual folder to explicitly distinguish from html pages.

- Joe: oh I see, I just need to return the list I built previously?

- Sue: congratulations uncle. You're grade 1 in jQuery!

- Joe: wait a minute. Let's make the whole stuff run.

Note:

DIV is just a mechanism for structuring a document by declaring a block of information.

Creating On Web Connection handler

Joe knows that an http request for a known resource (that means in the Web folder declared in the database) sends directly the resource back to the client. On the opposite, a request for an unknown resource going to the Web Server triggers the `On Web Connection` database method (OWC). A bunch of parameters is passed to this method: $1 contains the url of the request. Using a simple case of on $1 value allows building what is called a **virtual routing** of incoming requests.

Joe handles this case of in the OWC of the database and adds the following code:

```
w_sendFile("contactsList.shtml")
```

Where *w_sendFile* is just a utility routine calling **WEB SEND FILE** command to send the file which name is passed as a parameter and which is stored in the current folder for dynamic web serving.

Instead of running any html code to check the server-side behavior, Joe simply opens a new window in his favorite browser and type http://127.0.0.1/data/contacts in the URL. As he has previously set a breakpoint on his code, he can check that the new code is being called:

```
131    : (w_action="Contacts@")
132      w.sendFile ("contactsList.shtml")
```

After disabling the breakpoint and continuing the execution, he gets the contacts list displayed in the browser.

Joey Smith
jsmith@yahoo.com 408-455-6987

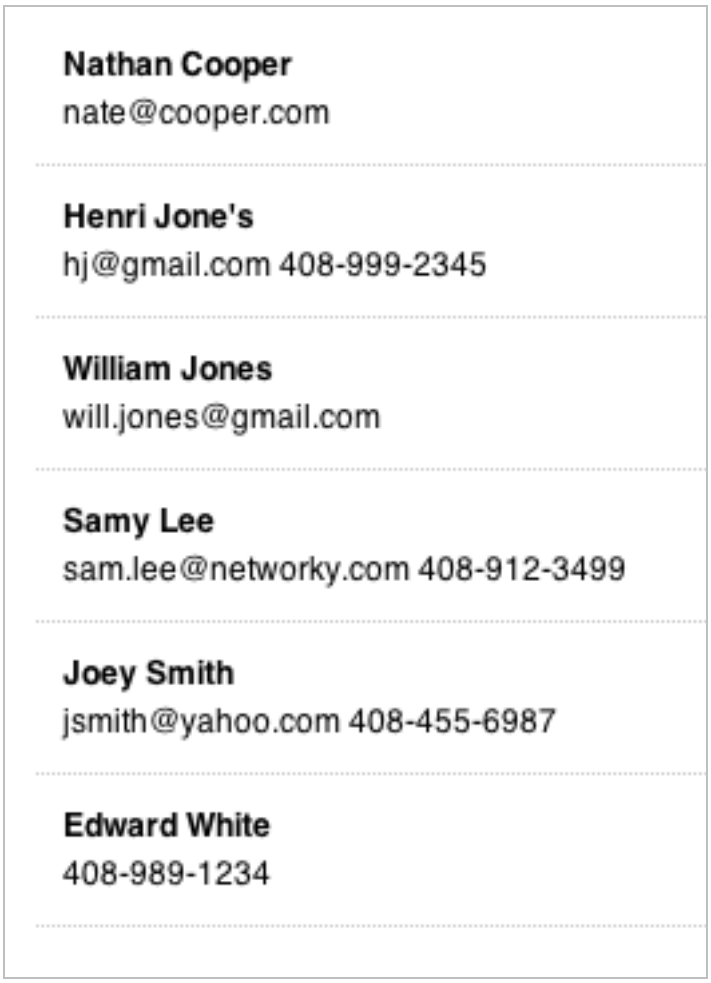
Samy Lee
sam.lee@networky.com 408-912-3499

Henri Jone's
hjones@gmail.com 408-999-2345

Edward White
e.white@ca.gouv 408-989-1234

William Jones
w.jones@hastings.net 68-945-2632

He is now ready to test Sue's new swc1.html page. This is what he gets in the browser:



Joe applies as much as possible a development strategy known as **baby-steps**: trying to validate each step of code as soon as possible in order to tumble as early as possible on a failure. This makes it easier to correct as the code has just being written.

Client-side debugging

Joe is happy with this first result but he has a question to Sue:

- Joe: I know how to check the behavior using the debugger on server-side. Sue, does an equivalent exist for client-side, the front-end as you call it?

- Sue: yes. It depends on the browser you use and its version. For example in Firefox you can install Firebug while in WebKit-based browsers as Safari and Chrome, you've got native Developer tools.

- Joe: Can you show me how it works?

- Sue: Sure! I'm using Chrome, so I just display Developer tools, using the Display menu. . There I can see the different requests, status code, time elapsed,...:

Developer Tools - http://127.0.0.1/ck/swc1.html

Elements Resources Network Sources Timeline Profiles Audits Console

Name / Path	Method	Status / Text	Type	Initiator	Size / Content	Time / Latency	Timeline (796ms, 1.1)
swc1.html /ck	GET	200 OK	text/html	Other	1.08KB 899B	3ms 2ms	
jquery-ui-1.8.22.custom.css /css/cupertino	GET	200 OK	text/css	swc1.html:10 Parser	7.45KB 32.54KB	89ms 78ms	
jquery-1.7.2.min.js /js	GET	200 OK	applicat...	swc1.html:19 Parser	38.34KB 92.62KB	113ms 107ms	
screen.css /css	GET	200 OK	text/css	swc1.html:11 Parser	1.05KB 1.66KB	101ms 86ms	
nudge-icon-arrow-up.png pioclpoplcdbaefihamjohnefbikjil	GET	200 OK	image/...	chrome-extension Script	(from c...	203ms 203ms	
nudge-icon-arrow-down.png pioclpoplcdbaefihamjohnefbikjil	GET	200 OK	image/...	chrome-extension Script	(from c...	199ms 199ms	
nudge-icon-arrow-lr.png pioclpoplcdbaefihamjohnefbikjil	GET	200 OK	image/...	chrome-extension Script	(from c...	197ms 197ms	
nudge-icon-return.png pioclpoplcdbaefihamjohnefbikjil	GET	200 OK	image/...	chrome-extension Script	(from c...	195ms 195ms	
Contacts	GET	200 OK	text/html	jquery-1.7.2.min.j Script	1.27KB 948B	134ms 30ms	
mzload.js d37u147w1ofw0w.cloudfront.n	GET	200 OK	applicat...	chrome-extension Script	7.68KB 21.25KB	1.75s 1.75s	

If I click on an item I can check its content:

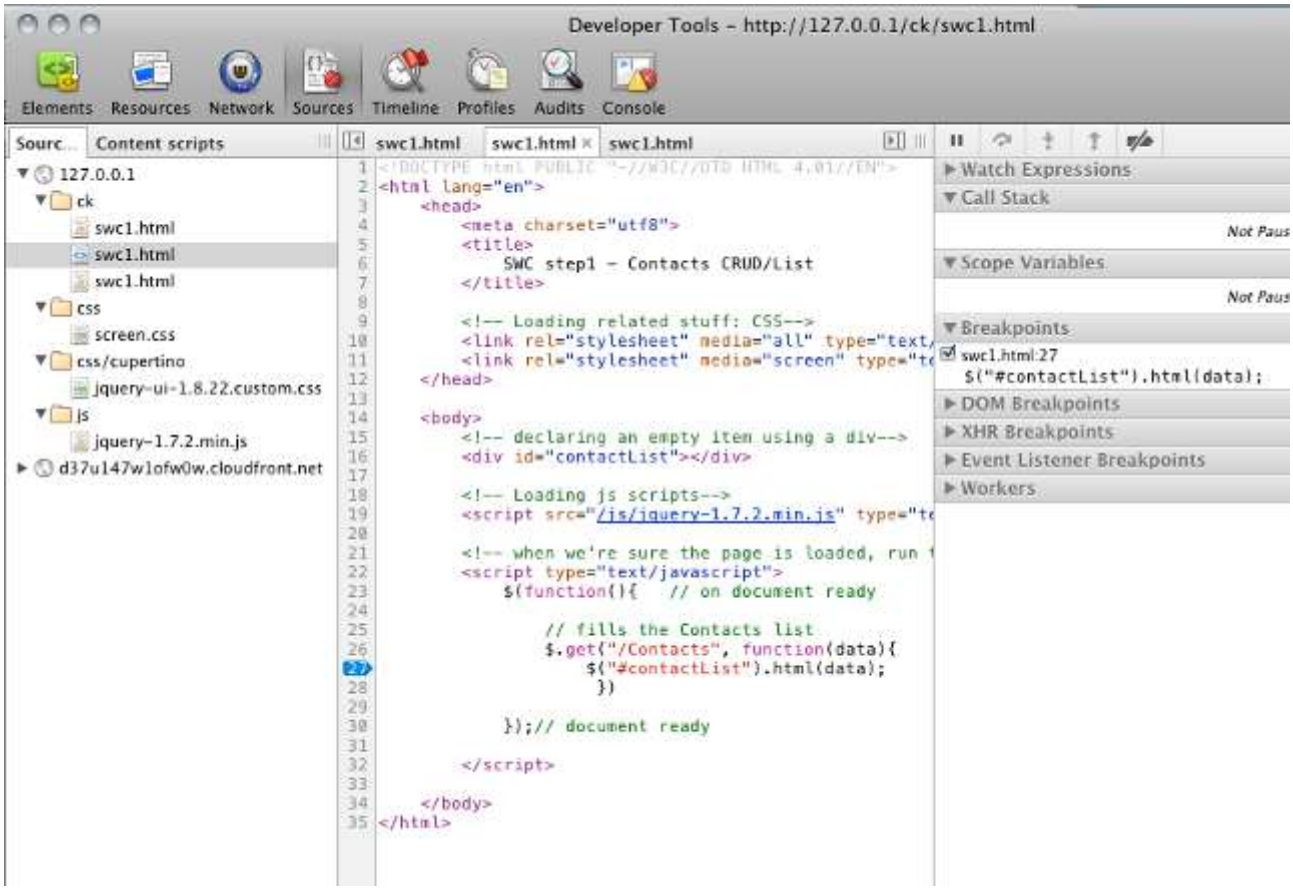
Name / Path

- swc1.html /ck
- jquery-ui-1.8.22.custom.css /css/cupertino
- jquery-1.7.2.min.js /js
- screen.css /css
- nudge-icon-arrow-up.png pioclpoplcdbaefihamjohnefbikjilc/images/nudge-
- nudge-icon-arrow-down.png pioclpoplcdbaefihamjohnefbikjilc/images/nudge-
- nudge-icon-arrow-lr.png pioclpoplcdbaefihamjohnefbikjilc/images/nudge-
- nudge-icon-return.png pioclpoplcdbaefihamjohnefbikjilc/images/nudge-
- Contacts
- mzload.js d37u147w1ofw0w.cloudfront.net

Headers | Preview | Response | Cookies | Timing

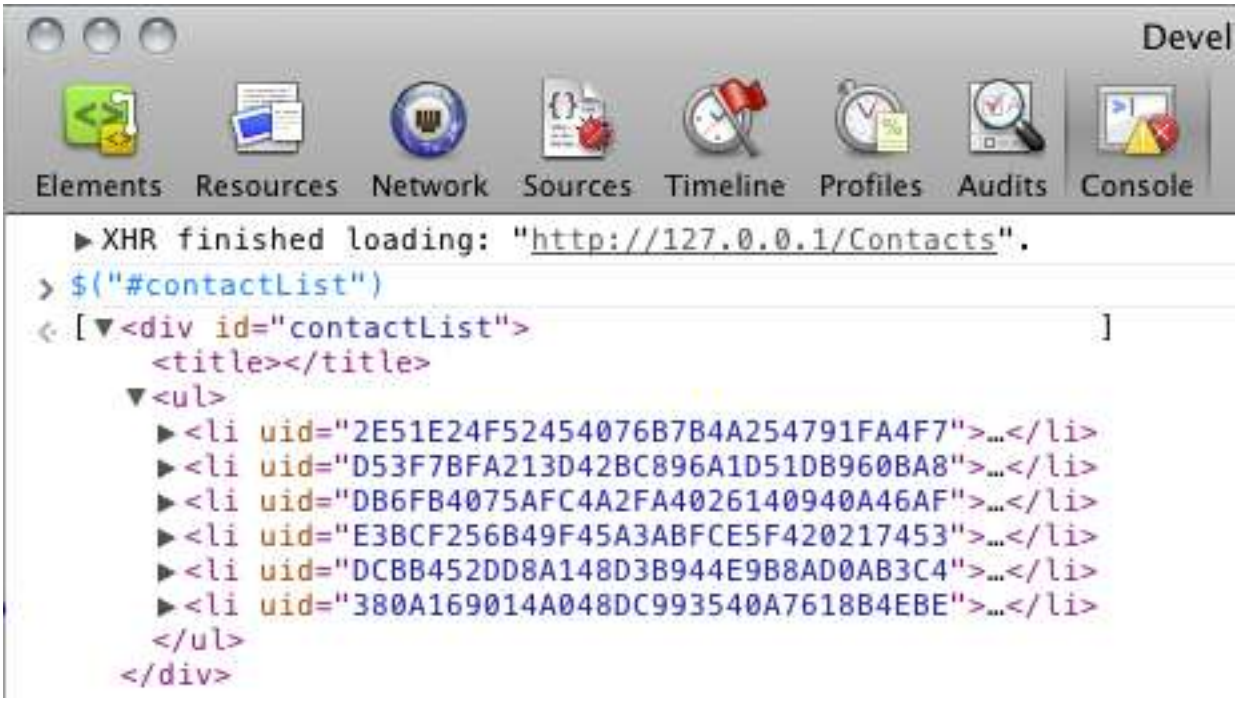
```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN">
2  <html lang="en">
3      <head>
4          <meta charset="utf8">
5          <title>
6              SWC step1 - Contacts CRUD/List
7          </title>
8
9          <!-- Loading related stuff: CSS-->
10         <link rel="stylesheet" media="all" type="text/css" h
11         <link rel="stylesheet" media="screen" type="text/css
12     </head>
13
14     <body>
15         <!-- declaring an empty item using a div-->
16         <div id="contactList"></div>
17
18         <!-- Loading js scripts-->
19         <script src="/js/jquery-1.7.2.min.js" type="text/jav
20
21         <!-- when we're sure the page is loaded, run the dyn
22         <script type="text/javascript">
23             $(function(){   // on document ready
24
25                 // fills the Contacts list
26                 $.get("/Contacts", function(data){
27                     $("#contactList").html(data);
28                     })
29
30             });// document ready
31
32         </script>
33
```

Then, in the Sources tab, I can display the code of the current page and add a breakpoint by clicking in the margin.



Reload the page; you'll see that code execution will stop on the selected portion.

You can also directly type JavaScript in the console for example in order to evaluate jQuery selectors:



- Joe: oh, I didn't know you had so much information at your disposal! I do like these possibilities, especially checking the communication between browser and server.

- Sue: yeah, that's pretty cool, but other tools are much valuable too.

Step 2: Displaying Contact detail

Choosing an approach

- Sue: okay, remember I just wanted them when selecting a contact to be able to display all the information of the contacts in a form. We need to make a choice right now!

- Joe: which one?

- Sue:

- **solution 1**, instead of simply the list of Contacts you send all the information for each contact as a whole and then when selecting a contact, we fill the Contact detail with data we already have in memory. Advantage: less requests to the Server, efficient with slow network (you said mobile?)

- **solution 2**, each time selecting a contact, the code sends a new request to the server to get Contact information.

I suggest adopting solution 2 as it is simpler to build on our existing code. Once you master this approach, you can change if you want.

- Joe: I can see both advantages and inconveniences. So I agree with you for the simplest solution for the moment.

Detecting contact selection

- Sue: we need to detect when the user clicks on a contact. For that, I'll add the following code in the script section of the html page:

```
	// click on a contact
	$("#contactList li").live("click", function(e) {
		unhiliteContacts();
		hiliteContacts(this);

		var uid = $(this).attr("uid");
		getContact (uid);
	})
```

`$("#contactList li")` is a jQuery selector that returns all list elements in "contactList" DIV.

I attach an observer handler to these elements in order to detect a click event and in this case to run the anonymous function immediately following.

- Joe: What's the meaning of this "live" call?

- Sue: Remember, when the code is run the elements don't exist. They are created on reception of Server response. So, it's too early to attach directly an event handler to them. The "live" call asks jQuery to observe the click event on this collection of elements permanently. It requires more resources of course.

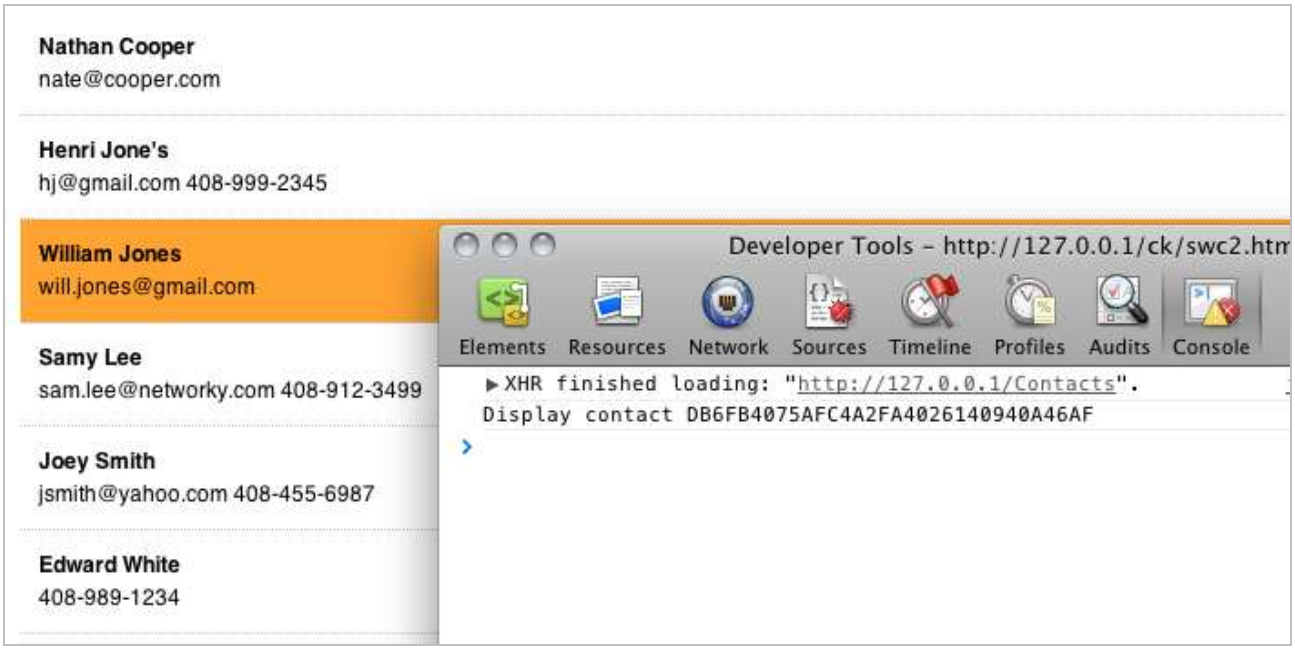
Joe: ok. Where are the other functions declared?

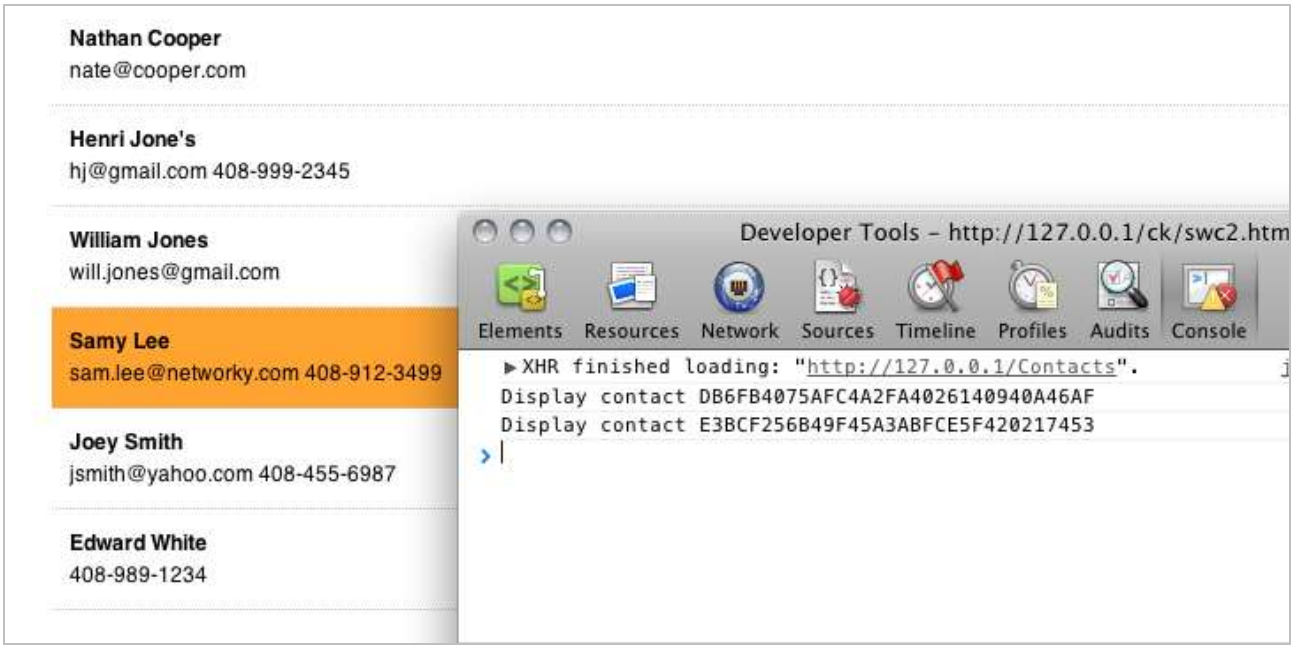
Sue: in order to make it easier to read, understand and debug, I prefer declaring all functions with meaningful names in a separate JavaScript file. I suggest to create a new "scripts" folder and to put a contacts2.js file in it. Here is its content, you will see that I have also externalized the display of the list in a `displayContacts)` function. To load this new script in my page, I've simply added the instruction: `<script src="/scripts/contacts2.js"</script>`

This is the content of swc.js:

```
function displayContacts() {
	// fills the Contacts list
	$.get("/data/contacts", function(data) {
		$("#contactList").html(data);
	});
}

function unhiliteContacts(){
	// dis-hilite every contact in the list
	$("#contactList li").each(function(){
		$(this).removeClass("row_selected");
	})
}

function hiliteContacts(item){
	$(item).addClass("row_selected");
}

function getContact(uid){
	console.log("Display contact "+uid);
}
```

- Sue: let's first try it and then I'll explain further. We will check that the id of the selected Contact is displayed in the Console.





- Joe: it works indeed. I understand that each time a new contact is selected a log entry is added in the console. I understand the jQuery selectors but what is this `.addClass()` and `.removeClass()` about?

- Sue: very simple, in fact. It's about using CSS. You know CSS declarations customize the display using stylesheets. You can declare a stylesheet for an html element or for a given class. We use this approach for highlighting the row. In css/screen.css file, I have declared a style for the row_selected class that simply changes the background color of the element: `.row_selected{background-color: orange}`. When I want to highlight a contact, I just attach the class to the selected element and to dim it I remove the class attachment on all elements of the list.

Requesting Contact Detail information

- Joe: If I'm right, we need to add some code in `getContact()` in order to send a request to 4D to get Contact detail?

- Sue: uncle, you're perfectly right: I suggest sending you a request `GET /data/contact(uid)`.

- Joe: you mean the contact id is sent in the url between parenthesis? Strange?

- Sue, malicious: you know, I've studied a bit this new 4D product Wakanda. And that's exactly how it works, so I thought it would interesting to follow the convention?

- Joe: you always have the perfect answer! Shall I send you an html form this time?

- Sue: in fact, a good practice is to separate the presentation from the data so that maintenance is made easier. I suggest you simply send back the data in a structured format and I will take the presentation in charge. Being on the front-end, it's my responsibility.

Structured format = JSON

- Joe: seems logical indeed. What about the structured format? May I use XML which is well-supported on 4D Side?

- Sue: in fact, when building JavaScript front-ends, we rather use JSON (JavaScript Object Notation) which is an equivalent especially designed for easy transformation to JavaScript objects.

- Joe: hum...heard of that format, but I'm afraid 4D won't support it.

- Sue: I saw how you generated your dynamic html. I think you can use the same approach for building the JSON string.

- Joe: oh, I see. Build a static JSON strings, validate the correct behavior and make it dynamic using 4D tags?

- Sue: yes, seems possible, no?

- Joe: oh yes, will be quickly done. Just need to get more information on JSON. Found that url: http://www.json.org/

- Sue: And you've got this useful one to check the validity of a JSON string: http://jsonlint.com/. I will be preparing Contact display meantime.

- Joe: let's synchronize when we'll be ready.

- Sue: ok, I bet I will have finished before you!

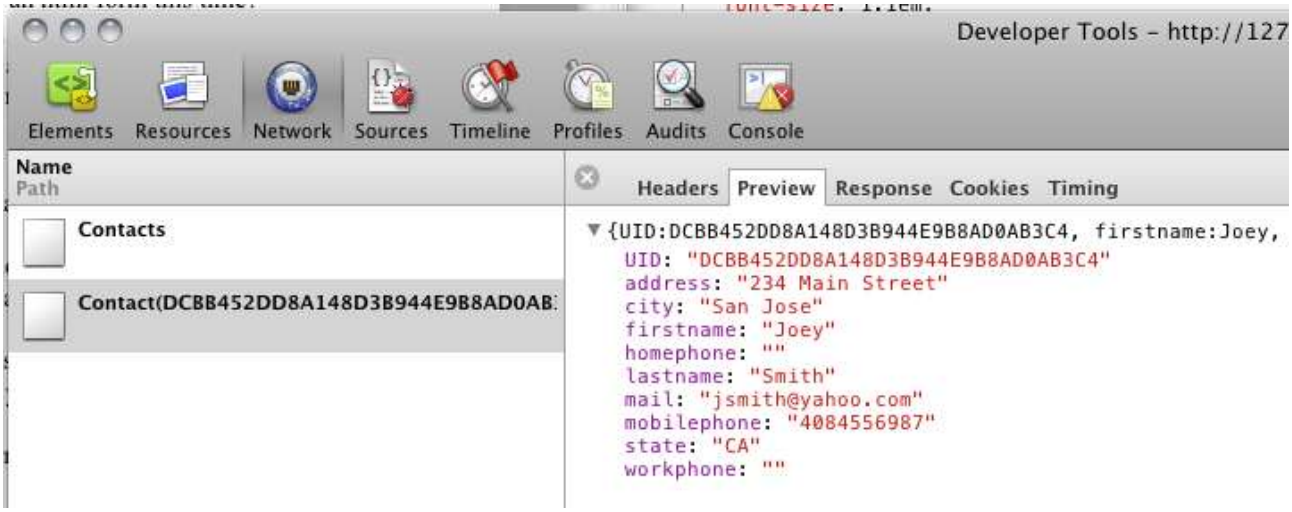
Displaying the detail

- Joe: I'm waiting!

- Sue: I was just checking my facebook page! Let's see what you've done.

- Joe: type GET http://127.0.0.1/data/contact(DCBB452DD8A148D3B944E9B8AD0AB3C4) in the browser

- Sue: ok I get a valid JSON, I can see it in Developer tools:



- Joe: this is the code that is run when detecting a GET "data/contact@" in On Web Connection:

```
$uid:=w_getIDfromURL

QUERY([Contacts];[Contacts]UID=$uid)

w.sendFile ("contactDetail.shtml")
```

and the template I use to build the dynamic JSON string, was simple indeed:

```
{
"UID": "<!--#4dtext [Contacts]UID-->",
"firstname": "<!--#4dtext [Contacts]Firstname-->",
"lastname": "<!--#4dtext [Contacts]Lastname-->",
"mail": "<!--#4dtext [Contacts]Email-->",
"address": "<!--#4dtext [Contacts]Address-->",
"city": "<!--#4dtext [Contacts]City-->",
"state": "<!--#4dtext [Contacts]State-->",
"mobilephone": "<!--#4dtext [Contacts]MobilePhone-->",
"workphone": "<!--#4dtext [Contacts]WorkPhone-->",
"homephone": "<!--#4dtext [Contacts]HomePhone-->"
}
```

- Sue: I've added the following html declaration in swc2.html to add a ContactForm:

```
<div id="contactInfo">
                    <form id="contactForm" class="round shadow" action="/data/contact"
method="post" name="contactForm">
                           <input type="hidden" id="uid" name="uid">
                           <img id="photo" src="/media/face.jpg" name="photo">
                           <div>
                                  <label for="firstname">First name</label> <input
type="text" id="firstname" name="firstname" class="required" title="Enter your first
name">
                           </div>
                           <div>
```

```
                                <label for="lastname">Last name</label> <input
type="text" id="lastname" name="lastname" class="required" title="Enter your familly
name">
                          </div>
                          <div>
                                <label for="address">Address</label> <input
type="text" id="address" name="address" minlength="10" title="Address is too short">
                          </div>
                          <div>
                                <label for="city">City</label> <input type="text"
id="city" name="city">
                          </div>
                          <div>
                                <label for="state">State</label> <input type="text"
id="state" name="state">
                          </div>
                          <div>
                                <label for="mail">e-mail</label> <input type="text"
id="mail" name="mail" class="required email" title="e-mail is required">
                          </div>
                          <div>
                                <label for="mobilephone">Mobile</label> <input
type="text" id="mobilephone" name="mobilephone">
                          </div>
                          <div>
                                <label for="workphone">Work phone</label> <input
type="text" id="workphone" name="workphone">
                          </div>
                          <div>
                                <label for="homephone">Home phone</label> <input
type="text" id="homephone" name="homephone">
                          </div>
                          <div>
                                <input id="submitContact" class="submit"
type="submit" value="ok" style="display:none"><br>
                          </div>
                    </form>
             </div>
```

- Joe: I suppose class declarations are used for enhancing the display?

- Sue: exactly. Let me show you the code for calling the Server and managing the response. It's all contained in contacts2.js. First of all, I've modified `getContact()`:

```
function getContact (uid){

       // Ajax call to 4D to get the details of the selected contact
       $.getJSON("/data/contact("+uid+")", function(data){
              // on success: fill the form
              console.log("Display contact "+uid);
              displayContact(data);
       })
}
```

All the work here is done by jQuery `.getJSON()` function. It automatically transforms a JSON string into a JavaScript object. So the data passed to `displayContact()` is ready-to-use.

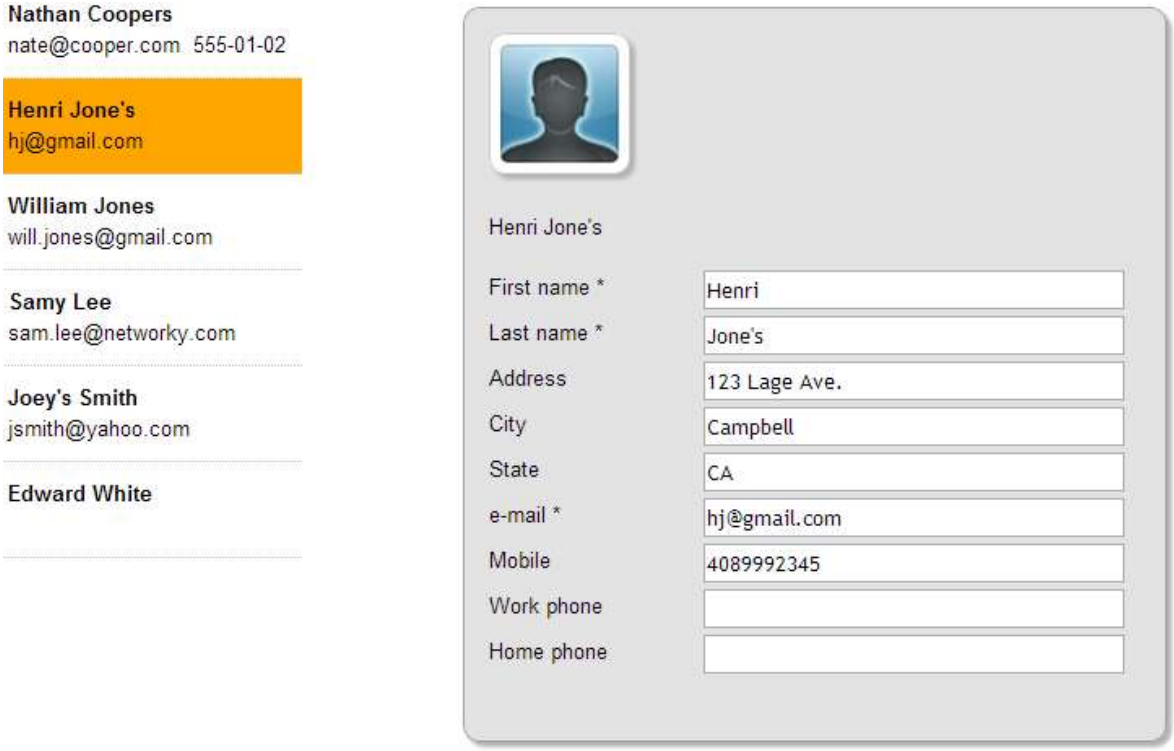
Then, I've added `displayContact()` function to fill all the fields of contactForm.

```
function displayContact(contact){
       $("#uid").val(clean(contact.UID))
       $("#firstname").val(clean(contact.firstname))
```

```
        $("#lastname").val(clean(contact.lastname))
        $("#mail").val(contact.mail)
        $("#address").val(contact.address)
        $("#city").val(contact.city)
        $("#state").val(contact.state)
        $("#mobilephone").val(clean(contact.mobilephone))
        $("#workphone").val(clean(contact.workphone))
        $("#homephone").val(clean(contact.homephone))
        $("#photo").attr("src", "/data/contact("+contact.UID+")/photo");
}
```

When you select an item by jQuery selector, the `.val()` function allows you to set/get its current value. Let's check?

- Joe: YES! I can't wait! Yeap it works!



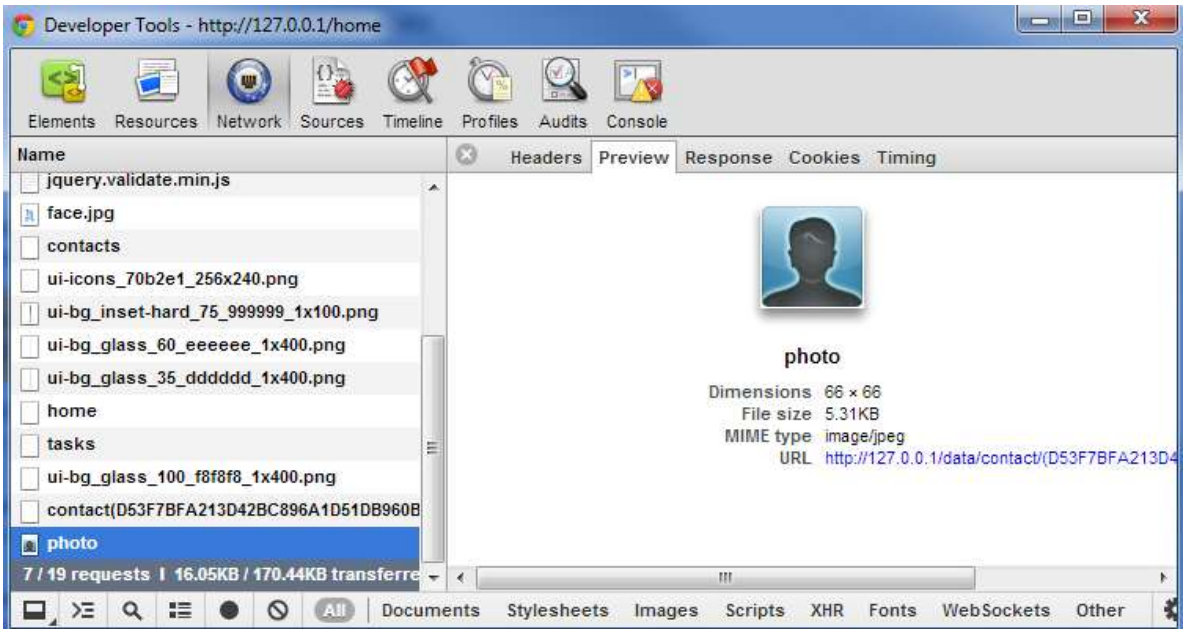
Display photo

Note: In fact, there is still a subtle code to be completed before getting the above display. Look at the declaration of the photo element. Sue doesn't set directly a value but the src attribute. This is simply how html works. When the browser will parse this code, it will send a new request to the server to get the photo.

So, Joe has to complete the code in OWC to send the photo when he receives:
`GET data/contact({uid})/photo`:

```
If (Position("/photo";w_url)#0)

       If (Picture size([Contacts]Photo)#0)

            PICTURE TO BLOB([Contacts]Photo;$b;"jpeg")

            WEB SEND BLOB($b;"image/jpeg")
```

```
        Else

             WEB SEND HTTP REDIRECT("/media/face.jpg")
            // if no photo exists, redirect to a standard photo

End if
...
```



Step 3 - Update contact

Submit Contact

- Joe: our next step will be to update the current contact. As informations are displayed in a form, should not be too difficult?

- Sue: yes, in fact the code is already there, in the form declaration:

```
<form id="contactForm" class="round shadow" action="/data/contact" method="post"
name="contactForm">
```

When the user will click on the ok button of the form, the content of the form will be submitted to the Server through a `POST /data/contact` request. You will receive the data in the body of the request.

- Joe: wait a minute, I don't see any ok button on the form?

- Sue: you're right. There is a trick. I just wanted to provide you an example of setting the visibility of an element, so I simply hide the button when the form is loaded. Then I show it when a keyboard is pressed.

- Joe: oh, you have your secrets! Where is this code? Ah I can see where you install the event handler to show the button. It's in swc2.html:

```
// show submit
$("#contactForm").keypress(function(){
        $("#submitContact").show();
})
```

And yes, you simply set the style of the button to `display:none` in the html declaration:

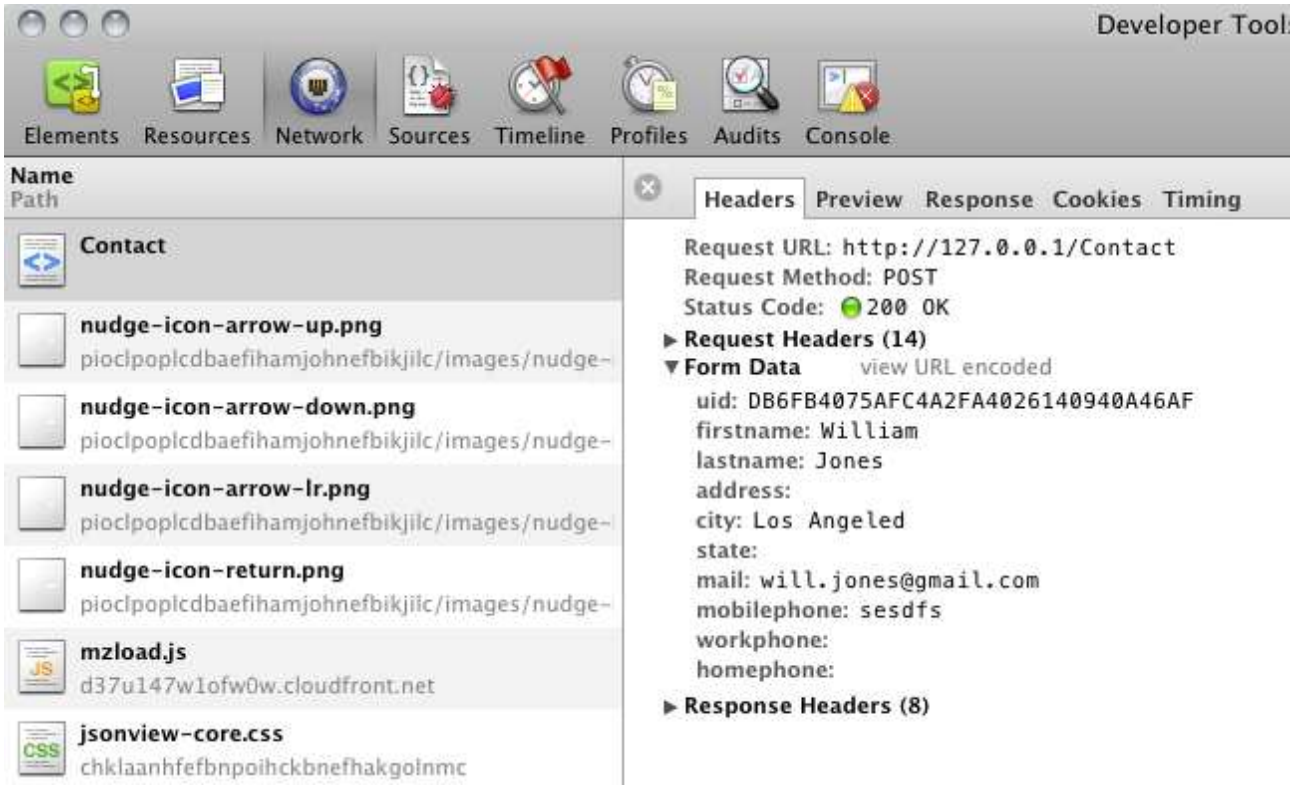
```
<input id="submitContact" class="submit" type="submit" value="ok"
style="display:none"><br>
```

- Sue: you've found the eastern egg! Let's see how you manage the update now.

Update on Server-side

- Joe: I'm curious to see how the data are transported back to the Server. I think I can see that using Developer tools?

- Sue: sure. Let's go!



- Joe: in fact, there is a command in 4D for getting form values into arrays. Using this, it should be pretty easy to make the Update. Let me 5 minutes!

- Sue: ok, I will bring two coffees meanwhile.

6 minutes later...

- Joe: Houston, we have a problem...

- Sue smiling: good: opportunity to learn something.

- Joe: I get the data on server-side. I find the Contact record using its ID. I update it with the new values. So far, so good. But then I send back the new data to the browser and I see them displayed in the window instead of my page!

- Sue: not surprised: you see when you submit a form, you actually request a new page to be sent from the Server.

- Joe: ouch, that's a problem. It would be very ugly! It's a pity as I was very satisfied with the way I knew it was an update demand on server-side...

- Sue: please tell me...

- Joe: in fact, the URL for getting a Contact or updating a Contact is quite the same. I asked myself if I couldn't discriminate the GET request from the POST request we use when updating. I tried looking in the http headers 4D provides to me , the information is in X-METHOD header. I saw that the information was also present at the first line of the request: POST /data/contact HTTP/1.1. I went back to the documentation of 4D and saw that this information is returned in the second parameter of OWC! Then, writing the small routine to detect GET or POST was a piece of cake! Well, I could have written it better but anyway it does the job for the moment.

```
WEB GET HTTP HEADER($_headerNames;$_headerValues)

  // is it a Get or a Post

w_post:=False

$k:=Find in array($_headerNames;"X-METHOD")

If ($k#-1)

  If ($_headerValues{$k}="POST")

    w_post:=True

  End if

End if

w_get:=Not(w_post)
```

- Sue: I like the language. Find it very easy to read. Sounds like plain English!

- Joe: yeap, that has always be one strength of 4D. Easy to learn and understand.

But anyway, I'm sure you've got a function in jQuery for solving my problem? And by the way, if I wanted to manage it properly, I should run some controls before sending the data to the Server.

- Sue: this time I will proceed differently and rely on a jQuery plugin to do the job. You see as jQuery is widely adopted, there are a lot of complementary plugins provided by the huge community of developers.

- Joe: just the same as 4D!

I propose to use the validate plugin. The documentation is here http://docs.jquery.com/Plugins/Validation and it can be downloaded at this url http://bassistance.de/jquery-plugins/jquery-plugin-validation/.

Let's see how it works.

Validate jQuery plugin

- Sue: first, ask the html page to load the plugin. Just after the

```
<script src="/js/jquery.js" type="text/javascript"></script>
```

add this new declaration:

```
<script src="/js/jquery.validate.min.js"></script>
```

Then, we are going to replace the automatic validation action of the desired form by our specific code. In the "on document ready" script section, add this code:

```
	$("#contactForm").validate({
		submitHandler: contactFormValidate
	});
```

Now, add the function `contactFormValidate()` in contact.js:

```
function contactFormValidate(form){
	$("#submitContact").hide();

	$.post("/data/contact("+$("#uid").val()+")", $("#contactForm").serialize(),
	function(data){
		// update current element in the list
		$("#contactList li.row_selected").html(data);
	}
	);
}
```

You see, you just declare the `.validate()` function of the desired form. In this function, you hide the submit button and post the serialization of the form to the Server.

Then, try again validating the form, you shouldn't have the page refresh anymore.

Joe: it works fine. Great!

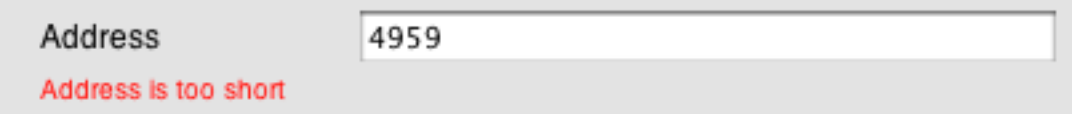
Sue: this plugin allows you to do a lot of nice other things: check that a field is an email, a credit card, an address a zip code...Even customize error messages.

Note:

In the complete listing of the example, you can find examples of custom error messaging and checking `minlength` for example on the address field.

```
<label for="address">Address</label>
<input type="text" id="address" name="address" minlength="10" title="Address is too
short"/>
```

The result when running the code:



Required

- Sue: a little bonus for the end of this part.

- Joe: wow, nice! What is it?

- Sue: I will show you how to indicate which field are mandatory in the Contact Form. The first step is to declare the "required" class on each mandatory input field. Then we want a way to add a "*" after the labels of each one.

- Joe: that sounds like a job for jQuery?

- Sue: for sure. Let's recall the declaration of an input element:

```
<label for="firstname">First name</label>
<input type="text" id="firstname" name="firstname" class="required" title="Enter your
first name">
```

You see the challenge is that the required class is on the input element and we want to modify the corresponding label element.

- Joe: I see...

- Sue: here is the sequence. Ask jQuery to find the collection of required input fields. Find the previous sibling of each one (i.e. the label element). Add the required class to the label. Then, the CSS for a required label will add a "*" after it. Here is the jQuery instruction:

```
$("input.required").prev().addClass("required");
```

and the CSS:

```
label.required:after{content:" *"}
```

Switch to Task management

- Joe: I'm very satisfied with all the job we did so far but time keeps running and we haven't started the tasks at all. I know the story ***Contacts CRUD*** is not completed as we haven't implemented the Delete feature but I suggest stopping here for lunch and meeting again this afternoon for the task management. And to be honest...I think I need a little nap before starting again! So many informations...

- Sue: I understand and that sounds good to me: I'm starving! Meet again at 2 p.m.?

- Joe, fine.

A WEB DEVELOPMENT TALE - EPISODE 2 - WHERE ARE WE?

Sue and Joe are meeting themselves after a separate lunch break in order to go on SWC development.

- Joe: I was considering our backlog:

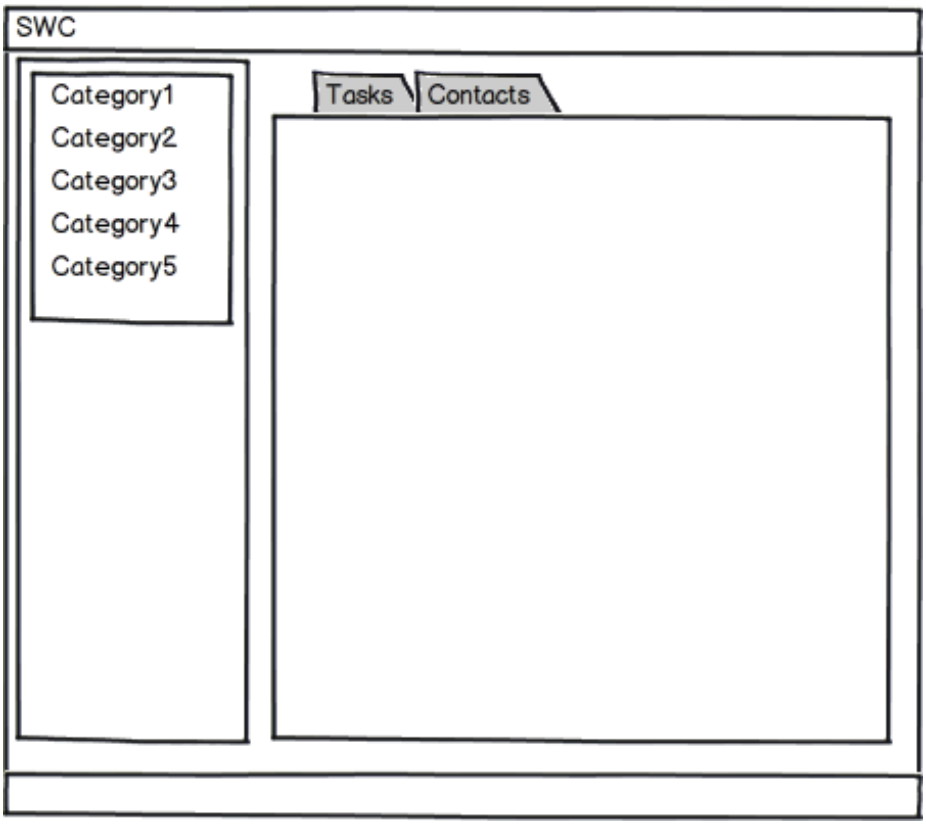
1	Task Management (Epic)	**Done**
2	Tasks CRUD	
3	Tasks filtering	
4	Contacts CRUD	**x**
5	Contacts filtering	
6	User authentication	
7	Single-Page	
8	Filtering by categories	

Table 1: Backlog of User stories

- Joe: And I suggest we start the afternoon with the story #7: As a user of the application, I want to get all the information on a single page so that it remains easy and natural to use.

On the desktop version it meant that we had three main containers: Categories, Task tab and Contacts Tab.

- Sue: Yes, I tried to draw a quick sketch of the screen organization



Step 4 – Task #7 Single-Page

- Sue: I propose adding the category list and the tab bar in this step?

- Joe: I presume we will be using those DIV blocks?

- Sue: quite a bunch, yes.

Organizing the page using DIV blocks

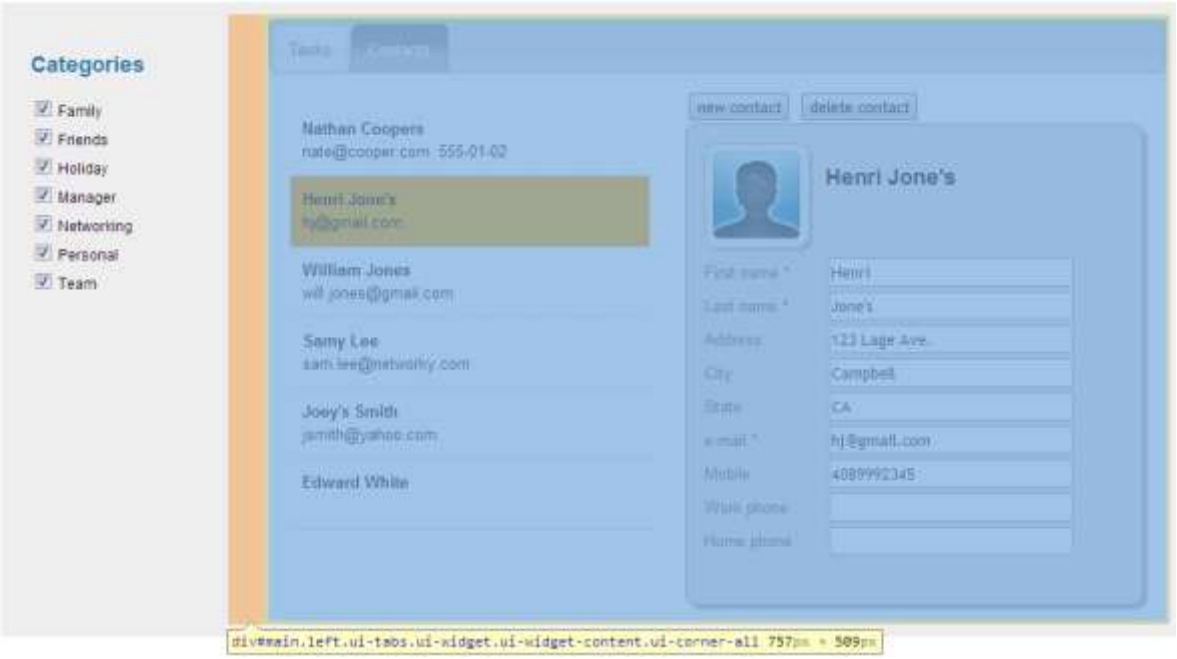
- Sue is editing the html page while talking, she modified the html structure as pictured below:

```
<body>
        <div id="page">

                <form id="categories">
                        <h2>Categories</h2>
                </form>

                <div id="main" class="left">

                        <div id="taskPanel"></div>

                        <div id="contactPanel">…

                </div><!-- main -->

        </div><!-- page-->

        <!-- Loading js libraries and scripts-->
```

Then, she loads the page in the browser and shows her uncle that he can retrieve the structure in the browser:

```
▶ <head>…</head>
▼ <body>
  ▼ <div id="page">
    ▼ <form id="categories">
        <h2>Categories</h2>
      ▶ <ul id="categoryList">…</ul>
      </form>
    ▼ <div id="main" class="left ui-tabs ui-widget ui-widget-content ui-corner-all">
      ▶ <ul class="ui-tabs-nav ui-helper-reset ui-helper-clearfix ui-widget-header ui-corner-all">…</ul>
        <div id="taskPanel" class="ui-tabs-panel ui-widget-content ui-corner-bottom ui-tabs-hide"></div>
      ▶ <div id="contactPanel" class="ui-tabs-panel ui-widget-content ui-corner-bottom">…</div>
        <!-- contactPanel -->
      </div>
      <!-- main -->
    </div>
    <!-- page-->
    <!-- Loading js libraries and scripts-->
```

Sue shows that moving the mouse over a div highlight it in the browser. For example below, the main div is hovered on the final version of this step.



Adding the tabs

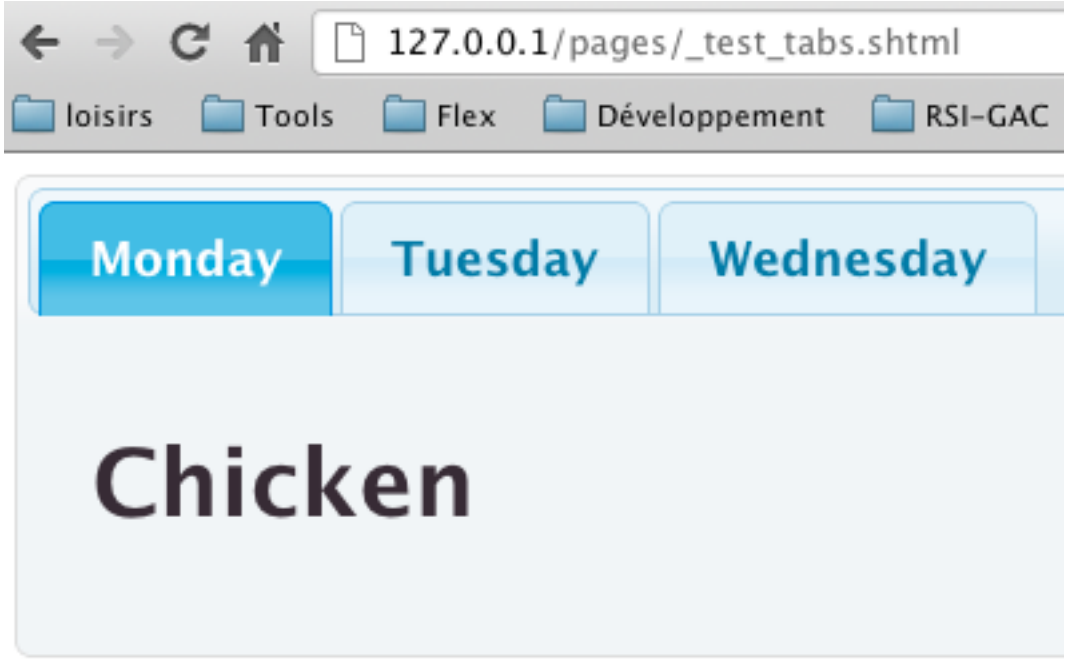
- Joe: I can see how you organize the blocks but now we need to declare two tabs? Any jQuery widget for that?

- Sue: In fact, we'll use another member of the band: jQueryUI. This framework of the jQuery family is specialized in widgets, user interface.

- Joe: Hearing the name, I could have guessed it!

- Sue: I will explain how it works on a small example and then I will let you adapt it our application.

Sue types a new url in the browser to show Joe the example of tabs she had prepared.



Then she right-click in the window to display the source code. Her comments are inserted in the source code as she talk to her uncle:

```
<html lang="en">
<head>
	<meta charset="UTF-8">
	<title>test Tabs</title>
	<link rel="stylesheet" media="all" type="text/css" href="/css/jquery-ui-
1.8.24.custom.css">
```

- Sue: not surprisingly, here I load the framework css

```
	</head>
<body>
	<div id="aBlockWithSomeTabs">
```

- Sue: it starts here. Just declare a list with items in a div block with an id.

```
		<ul>
			<li><a href="#menu-1">Monday</a></li>
			<li><a href="#menu-2">Tuesday</a></li>
			<li><a href="#menu-3">Wednesday</a></li>
		</ul>
```

- Sue: each item is an anchor referencing a div in the same page, just below...

```
		<div id="menu-1">
			<h1>Chicken</h1>
		</div>

		<div id="menu-2">
			<h1>Spaghettis</h1>
		</div>

		<div id="menu-3">
			<h1>Rosbif</h1>
```

```
            </div>
```

- Sue: you see? Each div will correspond to a page of the tab.

- Joe: don't tell me! You load now the jQueryUI framework itself, classical!

```
<script src="/js/jquery.js" ></script>
<script src="/js/jquery-ui-1.8.22.custom.min.js"></script>
```

- Sue: yes and remember the milk: on document ready, ask jQueryUI to convert the block containing the list declaration to a tab.

```
<script>
$(function(){ // on document ready
      $("#aBlockWithSomeTabs").tabs()
});
</script>
</body>
</html>
```

- Joe: can I comment this line to see what happens?

- Sue: of course, try it.

Joe comments the conversion request and reloads the page in the browser. This is what he gets:

- Monday
- Tuesday
- Wednesday

Chicken

Spaghettis

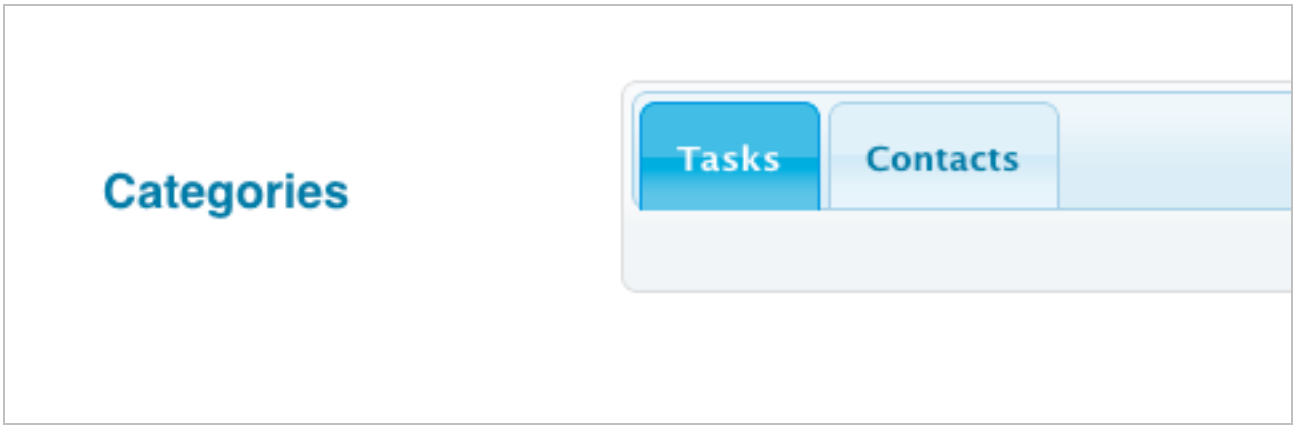
Rosbif

- Joe: my pal this jQueryUI. Good!

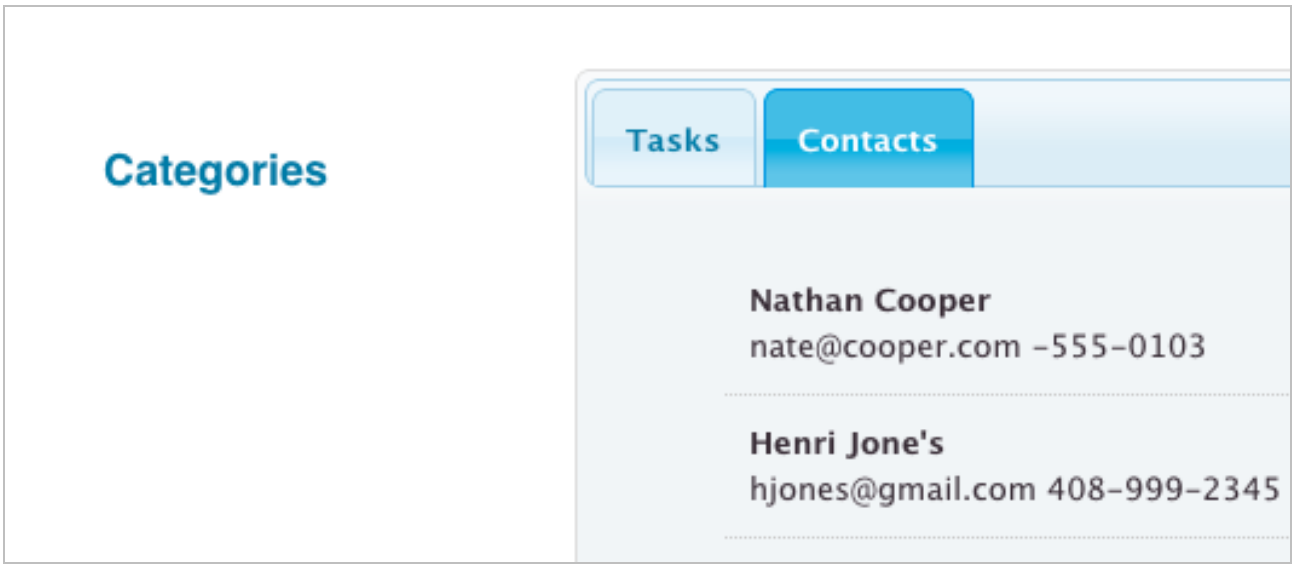
So, now it's up to me to apply this magical framework to swc app?

Sue: yes, I will look at the other user stories and see what I can prepare in advance while you'll be having fun!

A few minutes later, Joe calls Sue rather excited: Sue, look at my tabs! Look at my tabs!



And if I click on Contacts, it displays my Contacts! Hey...look at this!



- Sue: it starts looking like a real WebApp! Congratulations!
- Joe: yes, I worked rather hard for that: 2mns!

HTML page:

```
        <div id="main" class="left">
               <ul>
                      <li><a href="#taskPanel">Tasks</a></li>
                      <li><a href="#contactPanel">Contacts</a></li>
               </ul>

        <div id="taskPanel"></div>

        <div id="contactPanel">
…
```

Script:

```
$(function() { // on document ready
        $('#main').tabs(); // create tabs
```

```
        ...
```

What's next?

- Sue: I was thinking of completing the categories list, what do you think?

Listing the categories

- Joe: yes, as categories are stored in a 4D table, I propose to simply build a 4D template, check its behavior and include it in the main page. This is simple html, I think I can manage it.

Sue: so, this time it's your turn explaining

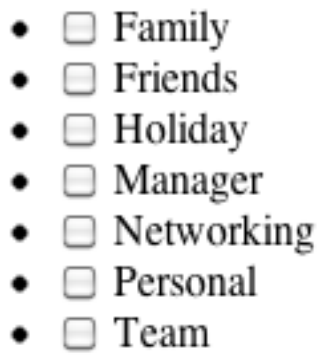
Joe quickly writes a dynamic html fragment that he stored in categories.shtml file. While writing, he explains his code to Sue as described below:

```
<ul id="categoryList">
      <!--#4DSCRIPT/w_categoriesAll-->
```

- Joe: 4DScript calls the *w_categoriesAll* 4D method. In this method I simply initialize the selection of categories by taking all records and ordering them by name. Then I loop through each category to build my list. Each category is displayed as a checkbox.

```
        <!--#4dloop [Category]-->
             <li>
                   <input type="checkbox" name="category" value="<!--#4dtext
[Category]UID-->" >
                   <!--#4dtext [Category]name-->
             </li>
      <!--#4dendloop-->
</ul>
```

- Joe: to validate it, I can simply call it from the browser.



- Joe: then I can include this template in the main page using the 4DINCLUDE tag:

```
      <form id="categories">
            <h2>Categories</h2>
            <!--#4DINCLUDE categories.shtml-->
      </form>
```

- Sue: and If I understood the 4D template mechanism, you need to change the ".html" suffix to ".shtml" in order for 4D to process the page?

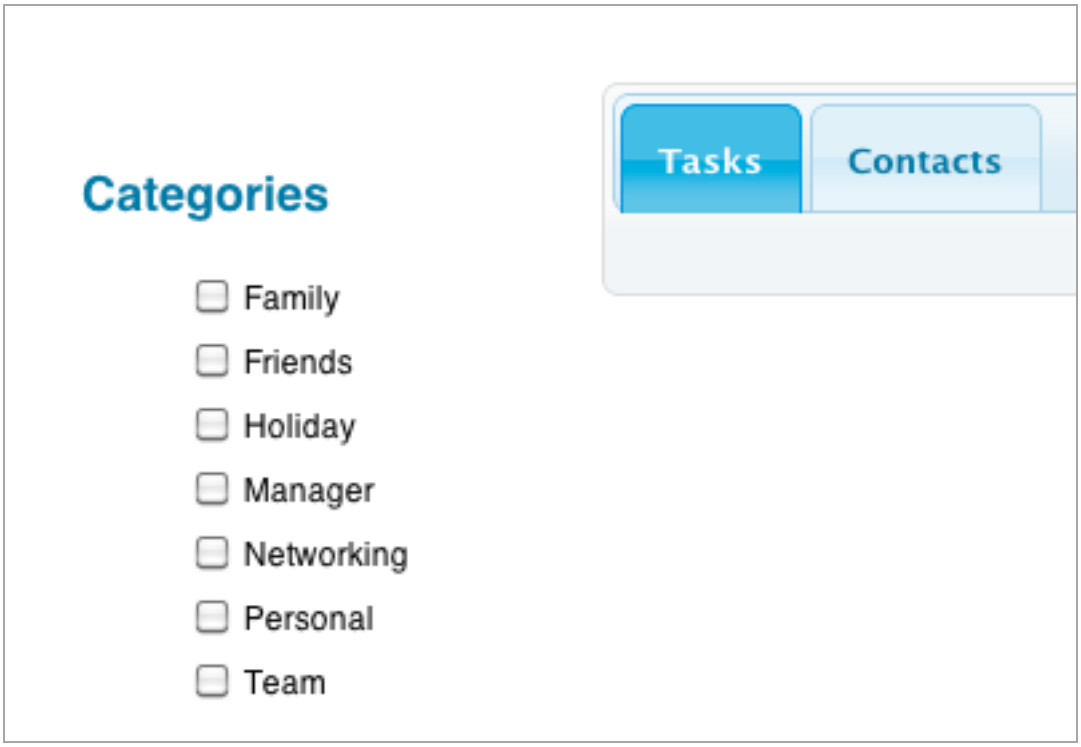
- Joe: oops, I was forgetting it: happily you're focused!

- Sue: that's a big advantage of pair-programming: if one forgets, chances the other will remember.

- Joe: you're right, we're gaining time!

- Let's try this new dynamic page:

- Joe asks the browser to load the modified page:



- Joe: nice, but almost too nice! Where are the dots of list items gone?

- Sue: look at the screen.css file. I inserted a line that removes them:

```
li{list-style: none outside none;line-height: 2em}
```

- Joe: I must remember to think of the css for everything related to UI style. It's not natural too me, but it seems interesting indeed to separate it from the html.

Joe and Sue then agrees that they are done with this story as the Single-Page is in place. They decide to move to the next story: Tasks CRUD. But before adding new features, Joe wants to refactor his code to improve the separation of items.

Just as form categories, he puts the contact form in a dedicated file contactForm.html and loads the file using a 4DINCLUDE tag:

```
        <div id="contactInfo">
              <!--#4DINCLUDE contactForm.html-->
        </div><!-- contactInfo →
```

All global scripts are put in a swc.js file so that there is no more JavaScript directly in the html page:

```
        <!-- Loading js libraries and scripts-->
        <script src="/js/jquery.js" ></script>
        <script src="/js/jquery-ui-1.8.22.custom.min.js"></script>
```

```
	<script src="/js/jquery.validate.min.js"></script>

	<script src="/scripts/contacts4.js" ></script>
	<script src="/scripts/swc4.js" ></script>
```

Then he checks that everything is still working as before.

Step 5 – Task #2 - CRUD Tasks

Setting up the HTML

- Sue: the organization of this part is similar to Contacts: a list of tasks in a grid on the left and when a task is selected the detail is displayed on the right. I suggest building first the html elements and then adding the JavaScript logic.

- Joe: I agree and I feel rather comfortable to implement this first part.

They discuss and decide to use an included dynamic page to make it easier to understand and maintain. Sue modifies the main page:

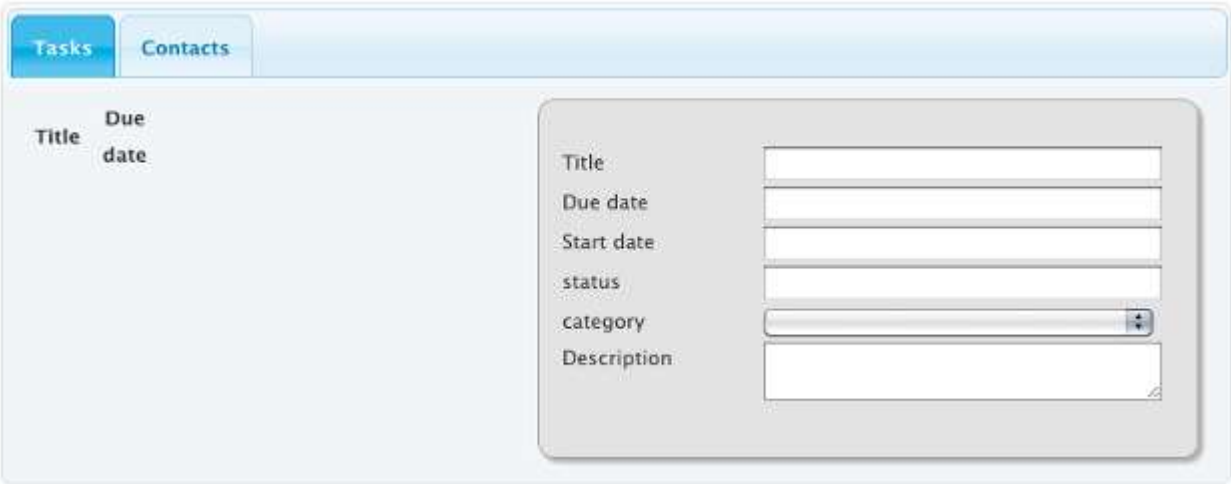
```
	<div id="taskPanel">
		<!--#4DINCLUDE taskPanel.shtml-->
	</div> <!-- id="taskPanel" -->
```

And then Joe writes the code of taskPanel.shtml including Sue suggestions on the fly:

```
<div id="taskListBlock" class="left">
	<table cellpadding="0" cellspacing="0" border="0" class="display" id="taskList">
		<thead>
			<tr>
				<th>Title</th>
				<th>Due date</th>
			</tr>
		</thead>
		<tbody>

		</tbody>
	</table>
</div> <!-- id="taskListBlock" -->

<div id="taskInfo">
	<form class="round shadow" action="">
		<p>
		<div style="display:none">
			<label for="uid">UID</label>
			<input type="text"  id="uid" name="uid">
		</div>
		<label for="title">Title</label>
		<input type="text" id="title" name="title" />

		<label for="dueDate">Due date</label>
		<input type="text" id="dueDate" name="dueDate" />

		<label for="startDate">Start date</label>
		<input type="text" id="startDate" name="startDate" />

		<label for="status">status</label>
		<input type="text" id="status" name="status" />

		<label for="category">category</label>
		<select id="category" name="category">
			<option></option>
			<!--#4dloop [Category]-->
			<option value="<!--#4dtext [Category]UID-->"><!--#4dtext
[Category]name--></option>
			<!--#4dendloop-->
```

```
            </select>

            <label for="description">Description</label>
            <textarea id="description" name="description" ></textarea>
            </p>
      </form>
</div> <!-- id="taskInfo"-->
```

Then they test the result in the browser and get the following display:



Tasks grid

- Joe: great but it seems a bit empty! We need to add the data management now!

- Sue: yes. Instead of displaying the task list in a simple (and rather dull) html table, I propose to use...

- Joe: another plugin! Let me guess? A listbox plugin?

- Sue: listbox? Kinda popup list?

- Joe: no, I mean something like a grid.

- Sue: that's it, we'll use the dataTable plugin (http://datatables.net/) that allows easy customizing of dataGrids. Basically, it will send a request to 4D to get a JSON stream containing the data to display (here tasks) and then format it nicely, adding supplementary useful features.

- Joe and Sue starts discussing the JSON structure that 4D will send for filling the table. Reading **dataTables** documentation, they learn the specific information (total records, total display records...) and structure expected by the plugin. They agree on this static example that Joe will then convert to a dynamic template using 4D tags:

```
{
  "sEcho": 1,
  "iTotalRecords": "11",
  "iTotalDisplayRecords": "11",
  "aaData": [
    [
      "Repair Broken Oven",
       "21/12/12",
       "029FC54CD1364C3F840459174F518237"
    ],
    [
```

```
      "Learn this crazy Web stuff",
       "01/11/12",
       "915AEA3F2A9645189549325C7985C35C"
    ],
...
    [
      "Build a trailer with Bobby",
       "01/02/12",
       "120666646AA8492C8E4E1FCAC5C5C74E"
    ]
  ]
}
```

For each task, three values are sent: title, dueDate, UID. Joe designed the dynamic template stored in taskList.shtml:

```
{
  "sEcho": 1,
  "iTotalRecords": "<!--#4dtext nbRecords-->",
  "iTotalDisplayRecords": "<!--#4dtext nbRecords-->",
  "aaData": [
  <!--#4dloop [Tasks]-->
    [
      "<!--#4dtext [Tasks]title-->",
       "<!--#4dtext [Tasks]dueDate-->",
       "<!--#4dtext [Tasks]UID-->"
    ]<!--#4dtext comma-->
  <!--#4dendloop-->
  ]
}
```

Joe then swaps to 4D and add a new entry in OWC to handle the `GET /data/tasks` that will be sent by the dataTable plugin:

```
      ALL RECORDS([Tasks])

      ORDER BY([Tasks];[Tasks]DueDate;<)

      nbRecords:=Records in selection([Tasks])

      w.sendFile ("taskList.shtml")
```

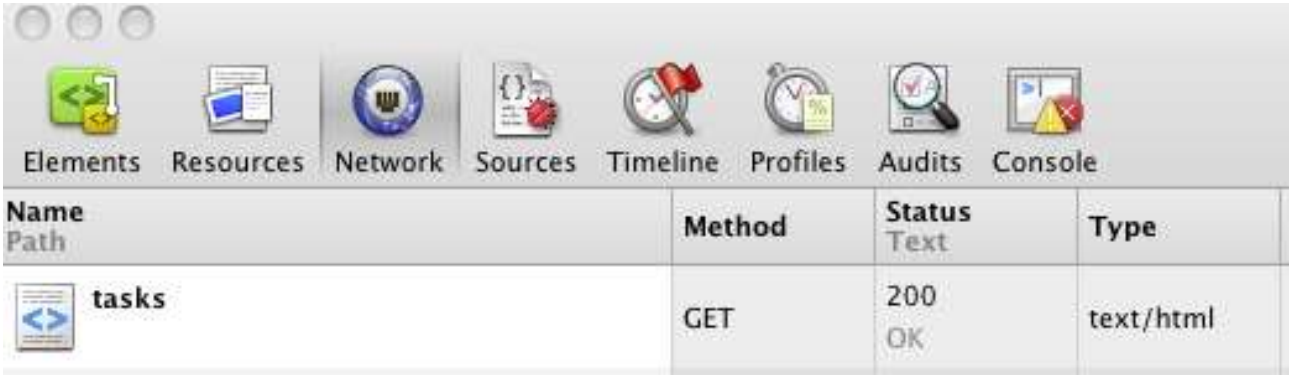
Sue types `GET /data/tasks` in the browser the validate Joe's job. She gets the JSON string displayed but she seems a bit puzzled and Joe sees her checking something in the Developer Tools.

- Joe: any problem? It looks good to me!

- Sue: result is ok, don't worry. It's just kind of detail. I was worried about the suffix of your dynamic template: ".shtml" as we are actually generating JSON not html.

- Joe: how, I see. In fact, if I use ".json", 4D won't parse the file when sending it.

- Sue: in our case, it works as both the browser and the plugin are tolerant but in theory it's not completely good. As you can see that the Content-Type header sent by 4D is text/html though the real stream is JSON:



- Joe: hum...I could fix it loading the template into a blob and using another 4D command **WEB SEND BLOB** to sends back the response. But I prefer leaving this detail for now and focusing on how to display a task detail?

- Sue: I'm with you! I just wanted to make it clear.
  Before going to the detail, I need to add some JavaScript to the swc.js script, in the "on document ready" section to actually set the different settings of the table and sends the request to 4D:

```
// initialize the tasks Table
buildTaskTable();
```

- Sue creates a new task.js file and load it in the" loading scripts" section:

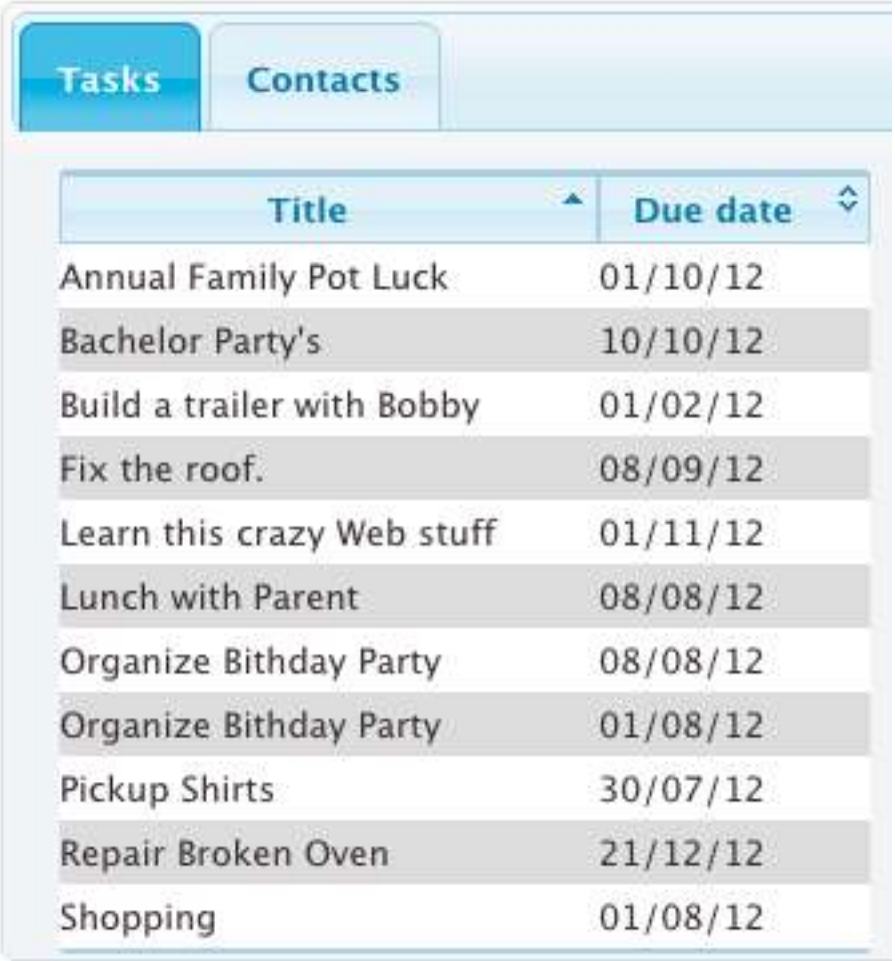
```
<script src="/scripts/task5.js"></script>
```

- She adds the `buildTaskTable()` function to task.js script:

```
function buildTaskTable() {
          $('#taskList').dataTable({
          "bJQueryUI": true,
          "bPaginate": false,
          "bLengthChange": false,
          "bFilter": false,
          "bInfo": false,
          "bSearch": false,
          "bAutoWidth": true,
          "sAjaxSource": "/Tasks",   /* url of the query to 4D */
          "aoColumns": [
                { "sWidth": "200px" },
                 { "sWidth": "100px", "sClass": "right",  "sType": "date"},
                  { "bVisible":    false }
                 ]
                });
}
```

- Sue: you can see that I've declared default width for the columns and added some information for better handling: second column justified on the right, of type date (that's for built-in sorting feature).

- Joe: let's see the rendering...



Task detail

- Joe: What are our next … tasks?
- Sue: detecting a click on a task, sending a request to 4D, managing the response on 4D side and displaying the result in the task Form.
- Joe: I take this job. I will try to adapt to task management what I learnt from Contacts.
- Sue: okay, I'm just co-piloting.

Here are Joe's actions.

- detecting a click on a task

Add the observer in the "on document ready" section of the swc.js file.

```
        // click on a task
        $("#taskList tbody tr").live("click", onClickOnTask)
```

Joe edits the task.js file

Here is the final content of this script file:

```
function buildTaskTable(){
```

```
        $('#taskList').dataTable({
              "bJQueryUI": true,
              "bPaginate": false,
              "bLengthChange": false,
              "bFilter": false,
              "bInfo": false,
              "bSearch": false,
              "bAutoWidth": true,
              "sAjaxSource": "/data/tasks",/* url of the query to 4D */
              "aoColumns": [{
                     "sWidth": "200px"
              }, {
                     "sWidth": "100px",
                     "sClass": "right",
                     "sType": "date"
              }      , {
                            "bVisible": false
                     }]
       });
}

// click on a task
function onClickOnTask(e) {
var taskTable= $('#taskList').dataTable();
// find the selected line
       var p = taskTable.fnGetPosition(this);
       var uid = taskTable.fnGetData(p)[2] // find the UID of the record in the third
hidden column
       // (Notice that arrays start at zero)
       // ajax request to get task info from 4D
       $.getJSON("/data/task(" + uid + ")", function(data) {
              displayTask(data);
       })

       // un-hilite all the lines and then hilite the current line
       unhiliteTasks();
       hiliteTasks(e.target.parentNode)

       return false;
}

function displayTask(task) {
       $("#uid").val(task.UID);
       $("#title").val(clean(task.title))
       $("#dueDate").val(task.dueDate)
       $("#startDate").val(task.startDate)
       $("#status").val(task.status)
       $("#category").val(task.category) // Notice that it is a popup
       $("#description").val(clean(task.description))
}

function hiliteTasks(item) {
       $(item).addClass("row_selected");
}

function unhiliteTasks() {
        var taskTable= $('#taskList').dataTable();
       // dis-hilite every task in the list
       $(taskTable.fnSettings().aoData).each(function() {
              $(this.nTr).removeClass("row_selected");
       })
}
```

sending a request to 4D

This is done in the `onClickOnTask()` function: `$.getJSON("/data/task(" + uid + ")", function(data)`

managing the response on 4D side

Joe has first added a new entry in the OWC routing case of:

```
C_BOOLEAN($ok)
$ok:=False

Case of
: (w_GET)
QUERY([Tasks];[Tasks]UID=w_uid)
w.sendFile ("taskDetail.shtml")
End case
```

Then Joe has created the taskDetail.shtml template:

```
{
  "title": "<!--#4dtext [Tasks]title-->",
  "dueDate" : "<!--#4dtext [Tasks]dueDate-->",
  "startDate" : "<!--#4dtext [Tasks]startDate-->",
  "status" : "<!--#4dtext [Tasks]status-->",
  "category" : "<!--#4dtext [Tasks]CategoryUID-->",
  "description" : "<!--#4dtext StrToJSON([Tasks]description)-->",
  "UID" : "<!--#4dtext [Tasks]UID-->"
}
```

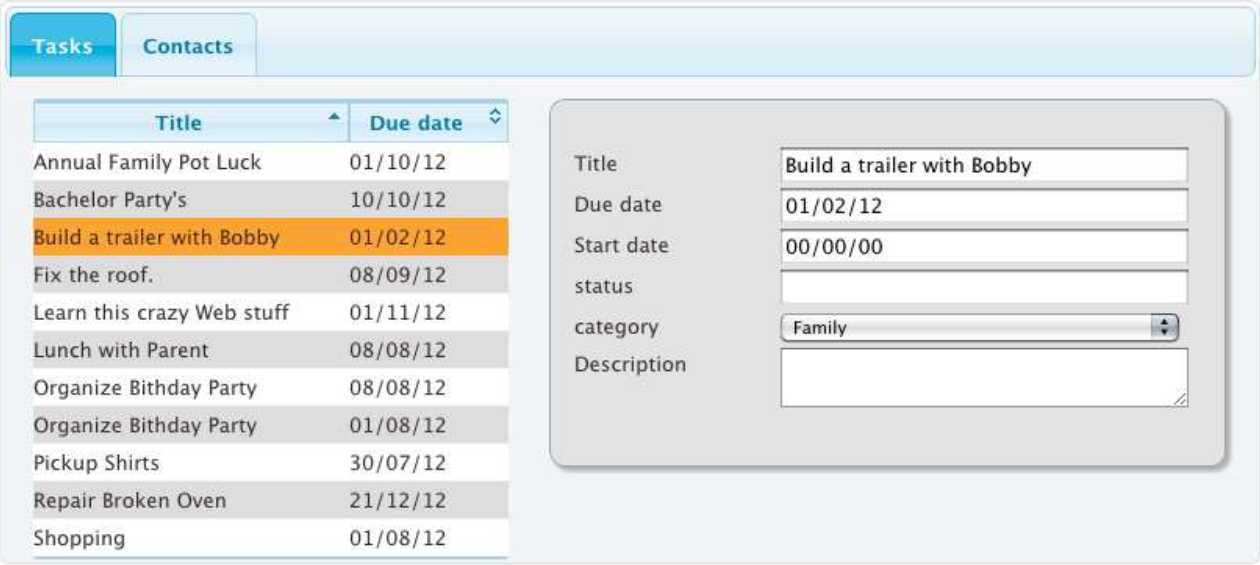
displaying the result in the task Form

This is done in the `displayTask()` function created in task.js.

Handling highlight / dim

The code is very similar to the one used for Contacts except the jQuery selector for getting each item of the table: `$(taskTable.fnSettings().aoData)`. In this case, they are using the specific API of the plugin, known from the documentation.

Final result



- Sue: I'm afraid of getting bored just watching simple adaptation of existing code. Why not change the way we update a task in order to learn something new?

- Joe: Ready for adventure, chief! What is your idea?

- Sue: for the contacts, we were sending a bulk update, all the fields in a single request. I think of updating the task on the fly, each time a field is updated. Something more dynamic!

- Joe: will mean much more requests on the network and more load on the web Server but it's interesting to go through a different approach and compare. Let's dive into this!

Instant updating

Let's list the actions that Joe will perform:

- In the "on Document Ready" section of the swc.js file, he needs to detect the change of a field of the form:

```
// edit a task field
$("#taskInfo input, textarea, select").change(onTaskFieldChange);
```

This is done using a CSS3 style selector for jQuery. It selects all elements of input or textarea or select types in the element taskInfo.

- Then, Joe must add `onTaskFieldChange()` function to task.js:

```
	function onTaskFieldChange(o) {
	var uid =$("#uid").val();
	$.post("/data/task("+uid +")?", o.target.name + "=" + o.target.value);

	switch (o.target.id) {
	case "title":
		$("#taskList .row_selected").children().eq(0).html(o.target.value);
		break

	case "dueDate":
		$("#taskList .row_selected").children().eq(1).html(o.target.value);
	}
	}
```

Joe sends a `POST /data/task(uid)` to the Server.

Notes:

The Body of the request contains the name and the value of the updated field in url-encoded format. This is easy to read in 4D using **WEB GET VARIABLES**(w_names;w_values), see *w_preprocess* method.

Joe immediately updates the list. It could be better to do it when receiving the ok response of the server as he did for Contact update.

- On 4D side, Joe edits OWC to complete the handling of `POST /data/task(uid)`

```
	: (w_post)
// Update

	// here we should run some validation code
$field:=w_names{1}
Case of
: ($field="title")
[Tasks]Title:=w_values{1}

: ($field="StartDate")
```

```
[Tasks]StartDate:=Date(w_values{1})

: ($field="dueDate")
[Tasks]DueDate:=Date(w_values{1})

: ($field="Description")

[Tasks]Description:=w_values{1}

: ($field="Status")
[Tasks]Status:=w_values{1}

: ($field="Category")
       [Tasks]CategoryUID:=w_values{1}

Else
WEB SEND TEXT("error")
End case

SAVE RECORD([Tasks])

// we must answer something (otherwise it is interpreted as a 404 error)
WEB SEND TEXT("ok")
```

Enhancing the Task form

And that's all! Joe is so good at sequencing the different actions between the front-end and the back-end now that Sue is just playing on her mobile phone!

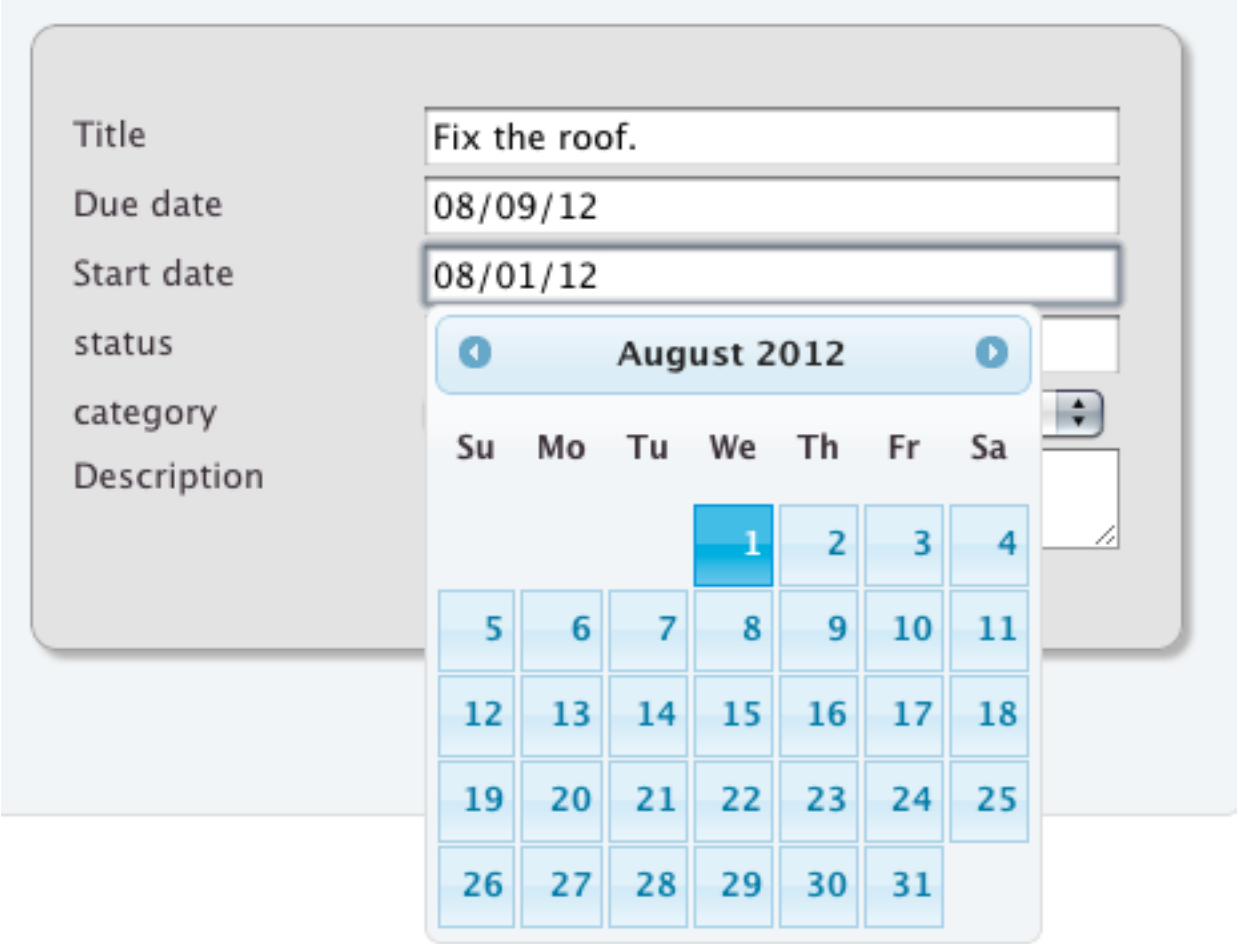
Joe: finished! It just works!

Sue: amazing, you gonna write a book on Web Development soon! But before, just let me learn you how to improve the update of date field by attaching a date picker to them:

in the "on Document Ready" section of the swc.js file, just add:

```
       // specify the handlers for the date input fields
       $('#dueDate').datepicker();
       $('#startDate').datepicker();
```

See the result?



- Joe: I must admit I've still got some lessons to take before writing the first chapter of my book! Well done Sue.

Improving sending JSON

- Sue: now that everything is in place, maybe we could address the issue with json strings sent as html?

I've just got to write an equivalent of w_sendFile for json templates.

This is the result of Joe's coding:

```
  //w_sendJsonFile(filename)


C_TEXT($1;$file)

$file:=$1


C_TEXT($virtualFolder)

$virtualFolder:="tuto"


$path:=Get 4D folder(HTML Root Folder)+$virtualFolder+Folder separator+"data"+Folder
separator+$file


$ref:=Open document($path;"*";Read Mode)
```

```
If (ok=1)
       C_TEXT($template;$json)
RECEIVE PACKET($ref;$template;MAXTEXTLENBEFOREV11)
CLOSE DOCUMENT($ref)

PROCESS 4D TAGS($template;$json)

C_TEXT($header)
$header:="Content-Type: application/json"
WEB SET HTTP HEADER($header)
WEB SEND RAW DATA($json)

Else
WEB SEND TEXT("404 : not found")
End if
```

Note: the reader may wonder why we don't simply use **DOCUMENT TO BLOB** command to load the json template? It's not possible to indicate to this command to load the file in read-only mode. So this could lead to potential problems in case of several process trying simultaneously to load the same template.

A WEB DEVELOPMENT TALE - EPISODE 3 - FINALE

Step 6 – Task #6 User authentication

Joe: ok, we have our single-page layout. I think there is only one supplementary form required: for login purpose.

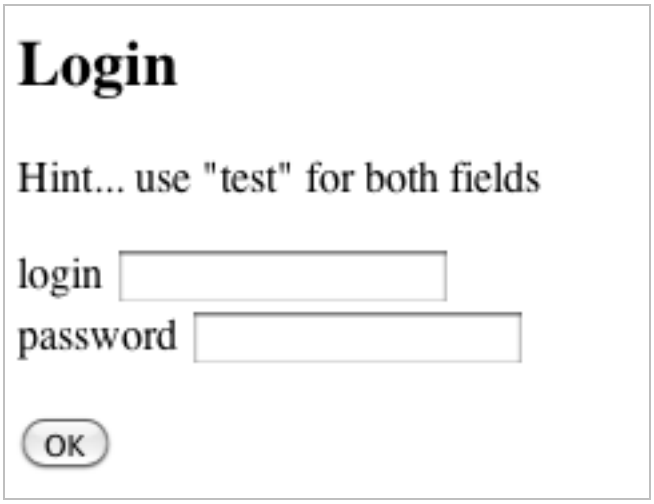
Sue: Yes for this user story:

#6 ***User authentication****. As a member of SWC, I want to authenticate myself so that I can get access to my own list of tasks and contacts in order to avoid someone else looking at my private stuff.*

Login Form

- Joe: What do you think of this form?

Joe displays the result of `GET /login.shtml` in the browser:



- Sue: seems containing the minimal fields, but If you don't mind, I'd suggest adding some CSS to beautify it.

Sue quickly adds some CSS declarations to the form

```
<style media="screen">
              label{float:left; width:150px; }
              input{width:150px;}
              input[type="submit"]{margin-left:250px; width:50px;}
              .hint {color:gray}
</style>
```

Then, she reloads it in the browser:

Login Sequence

-Joe: Look better! We just need to make it work, now.

I sum up, feel free to complete me:

- When the user first asks for the page, he is not connected, the server sends login form.
- The user fills his credentials and clicks on submit button sending a `POST /login` request to the Server.
- The Server processes the request:
  - if credentials are valid, sends application page
  - If not, sends login again
- Sue: maybe you could add a message to inform the User that credentials were wrong?

- Joe: good idea. I note that. Apart from this improvement, I think I must keep track of the connected state on server-side so that on next request, I don't send the login form again.
- Sue: you mean using some kind of session variables?
- Joe: yes, I think of managing a `w_connected` variable that will be false until the user has sent the right credentials. 4Dv13 makes it easy as it automatically maintains a session on server-side.
- I think I've got all what I need to start.

User authentication on server-side

Joe starts editing 4D application.

He opens the OWC database method and declares a boolean variable:

```
C_BOOLEAN(w_connected)  // False by default.
```

Then, in the Case of routing, he checks this value:

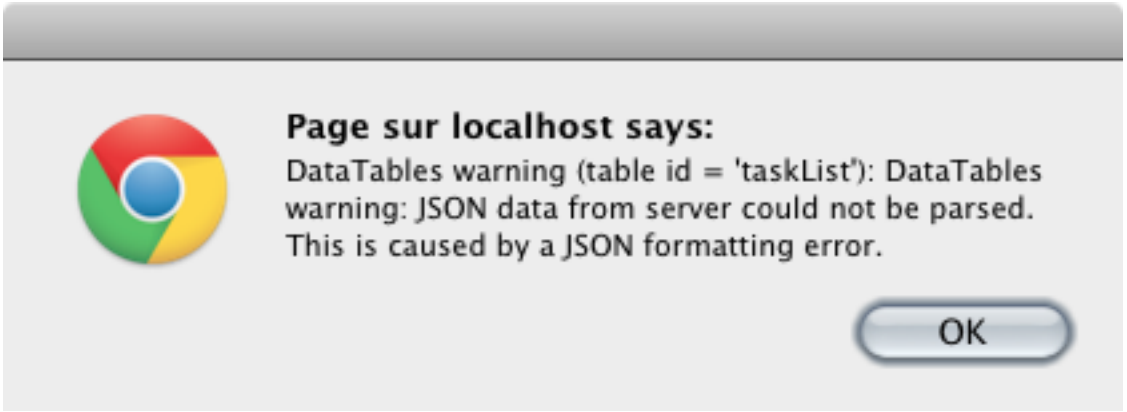
```
: (w_connected=False)

      WEB SEND HTTP REDIRECT("/login")
```

And then he handles the `GET /login` request in the same case of:

```
: (w_page="login")

      w_sendFile ("login.shtml")
```

-Joe: Let's try it!

And Joe feels rather disappointed: when he reloads the main form, he doesn't get the login form but a dataTables error!



-Joe: What's happening?

-Sue: Let's check the Network tab in developer tools, apparently the login redirection is not executed.

In fact, they can see in the Networks requests that that login redirection is asked AFTER the swc page has been sent to the client.

-Joe: Got it: I need to redirect before sending the real page to the browser. Problem is that if page exists, 4D sends it first and runs OWC after! I need to run it earlier: in the On Web Authentication (OWA) method!

Joe cuts the code from OWC to OWA:

```
C_BOOLEAN(w_connected)

If (not(w_connected)) & ($1#"/login")
       WEB SEND HTTP REDIRECT("/login")
End if
$0:=true
```

He tests again and now it's working, he gets the login form displayed in the browser.

-Joe: now, I need to handle the login submit request which is a `POST /login` action.

In OWC he just replaces:

```
       w.sendFile ("login.shtml")
```

by a call to a routine where he will handle login request and login submit request.

So he creates *w_login* method and puts the following code in it:

```
C_TEXT($0;$1)


C_TEXT(w_message)


If (w_get)

       w_sendFile ("login.shtml")

Else   //submit du login

If (w_getValue ("login")="test") & (w_getValue ("password")="test")

w_connected:=True

WEB SEND HTTP REDIRECT("/tuto/swc6.shtml")

Else

w_message:=Char(1)+"error"

w_sendFile ("login.shtml")

End if


End if
```

Joe is using some utility routines that we won't detail here:

In the OWC a call to *w_preprocess* method allows decoding the request and filling some process variables so that they are immediately available for use later. That's the case for w_get for example that indicates if it's a GET or a POST request. Just the same for *w_getValue* which is just taking advantages of arrays filled in *w_preprocess*.

He then checks that if the credentials are bad, an error message is displayed in the login form.

Session management in 4Dv13

This paragraph is a quick reminder of 4Dv13 session management.

In previous 4D version, each http request sent by a client to the Server was completely independent and executed in its own process.

Starting with v13, taking advantage of cookies to transmit a session identifier, 4D is able to automatically execute all requests coming from the same client inside the same persistent process. This new behavior has a kind of magical consequence: all process objects are preserved between two calls: variables, selections, loaded records...

Joe is using this mechanism for `w_connected` variable. First declared at false, it only becomes true if the user has been authenticated and then preserved its value between calls due to process conservation in memory.

As stated above, the whole mechanism relies on a cookie set by the Server on the browser. You can see the cookie request in the header first response, see Set-Cookie entry:

```
▼Response Headers    view source
  Accept-Ranges: none
  Connection: close
  Content-Encoding: gzip
  Content-Length: 593
  Content-Type: text/html; charset=UTF-8
  Date: Mon, 08 Oct 2012 06:57:14 GMT
  Expires: Mon, 08 Oct 2012 06:57:14 GMT
  Server: 4D_v13/13.1
  Set-Cookie: 4DSID=F724AA2F58694F018985F982B9289403; Path=/; Max-Age=28800; expires=Mon, 08 Oct 2012 14:57:14 GMT; HttpOnly; Version=1
  Vary: Content-Encoding
```

Joe wants to make the cookie value visible for debug purpose.

On the 4D side, he uses the following code to get the cookie value:

```
WEB GET HTTP HEADER($_headerNames;$_headerValues)
$k:=Find in array($_headerNames;"COOKIE")
If ($k#-1)

cookie:=$_headerValues{$k}

cookie:=Substring(cookie;Position("=";cookie)+1)

Else
```

```
cookie:="x"

End if
```

Then he is able to know the cookie value on 4D side and as he wants to easily know it on client-side too, he adds the following declaration in login.shtml:

```
        <p>
                cookie=<!--4dtext cookie-->
        </p>
```

Example of cookie display in the login form:

cookie=F724AA2F58694F018985F982B9289403

Login

Logout

Joe adds the following declaration in the main page just after the <body>:

```
<body>
                <p id="user">
                cookie=<!--4dtext cookie-->
                Connected as <!--4dhtml [users]name-->
                          -  
                        <a href="/logout">logout</a>
```

He gets the following display:

cookie=ECD31CA723274F9E89C99C326DBC7589 Connected as Test - logout

Then, he completes the routine case of in OWC with:

```
: (w_page="logout")

w_logout
```

Where the content of *w_logout* routine is:

```
If (w_connected)  // you need to be logged in before you log out

w_connected:=False

UNLOAD RECORD([Users])

WEB CLOSE SESSION(cookie)

cookie:=""

WEB SEND HTTP REDIRECT("/login")

Else

WEB SEND TEXT("error: session not active")
```

```
End if
```

Improving security

Now that the basic mechanism is in place, Joe replaces the basic check in *w_login* by something more robust:

```
$hash:=w_getLoginHash (w_getValue ("login");w_getValue ("password"))

READ WRITE([Users])

QUERY([Users];[Users]credentials=$hash)


If (Records in selection([Users])=1)  // correct!

w_connected:=True

[Users]session:=cookie

SAVE RECORD([Users])

WEB SEND HTTP REDIRECT("/tuto/swc6.shtml")

Else

w_message:=Char(1)+"error"

w_sendFile ("login.shtml")

End if
```

The new code is based on calculating a hash code by combining login-password and some random data, see `w_getLoginHash`.

What's interesting in this approach is that the only string saved in the database (`[Users]credentials`) is the hash code which is quite impossible to hack.

More on session management

Web process doesn't stay in memory forever. We have seen above that a clean logout allows 4D Server to gently get rid of the session by calling **WEB CLOSE SESSION**(cookie). But what happens if the user never clicks on logout? Will he be considered as logged forever? On the Web, as http protocol is a disconnected one, everything is under a timeout system. Same for 4D Web sessions. There is both a timeout for the cookie on client-side and for the process on server-side. Besides, 4D can decide on his own to get rid of a web process at anytime, simply because he needs memory for example.

An application that relies on automatic session management should address this concern. 4D provides a database method for that: **On Web Session suspend** (OWSS). Its is triggered when the web process is about to be released.

Joe simply adds the following code in OWSS:

```
If (Not(Undefined(cookie)))

If ([Users]session=cookie)
  // we save the session data
  // but the same cookie is still associated to the user

w_saveSession

End if
```

```
End if
```

The *w_saveSession* routine is basically an empty frame for managing more information in a complex application. It's the place to store the information of the session that the developer wants to preserve in a Blob for example. In the example, Joe simply **SAVE RECORD**([Users]) keeping ready for later use.

Last piece of the puzzle consists in restoring the session when required.

Here is an example of code that can be called from OWC:

```
If (cookie#WEB Get Current Session ID)

QUERY([Users];[Users]session=cookie)

cookie:=WEB Get Current Session ID

If (Records in selection([Users])=1)
// restore session

[Users]session:=cookie

w_connected:=True


// restore session variables and objects

SAVE RECORD([Users])

Else

w_connected:=False
// new connection: init session variables
and objects
End if
```

When to use sessions?

A web developer should avoid managing state on server-side as much as possible. Web is designed to be stateless as this allows for its scalability.

Not relying on states makes it easier to develop, understand and test. Each stateless request can be tested on its own without dependency. On the opposite stateful requests make it harder to have different teams developing in parallel on back and front-ends.

However, it can reveal expensive to recreate a context at each request. Suppose, you have to run a sequential query, implying several joins and ordering the resulting selection in a way that doesn't permit using index. It's very efficient in this case to be able to preserve the ordered selection between two requests. Using 4D automatic management session, this is very easy.

Filtering items per user

- Sue: Now that you know the current user, you can filter tasks and contacts?

- Joe: yes, it's easy now: once authenticated, the current user record is loaded in memory and kept throughout requests due to automatic session management. All I have to do is filter tasks and contacts using the current user.

Joe completes *w_contacts* in charge of Contacts selection:

```
QUERY([Contacts];[Contacts]UserID=[Users]id)
```

```
ORDER BY([Contacts];[Contacts]Lastname)
```

And *w_tasks* in charge of Tasks selection:

```
QUERY([Tasks];[Tasks]UserID=[Users]id)

ORDER BY([Tasks];[Tasks]DueDate;<)
```

Step 7 – Task #8 Filtering by categories.

- Sue: maybe we should consider filtering by categories, now?

- Joe: yes, let's recall the user story:

#8 ***Filtering by categories****. As a member of SWC, I want to be able to select categories just by checking the desired category in a list so that I can filter displayed items by checked categories*

It's a bit trickier than previous stories. I suppose we must first decide where the filtering is executed: on the front-end or on the back-end.

- Sue: yes. For example, when I use booking.com application to search an hotel on my iPhone, all hotels are first loaded, then filtering is run locally.

- Joe: there sure are pros and cons to put in the balance but as a beginner I feel more comfortable executing queries on server-side than manipulating collection of objects on client-side!

- Sue: so the sequence is the following:

- the user clicks on a category, the click event is detected and handled by JavaScript code
- the event handler asks Contacts list and Tasks list to rebuild themselves.

Sue adds the following code to swc.js:

```
	// change of the selected categories
	$("#categories input").click(function(e) {
			displayContacts();
	})
```

and she modified `displayContacts()` in order to send the selected categories to the Server:

```
function displayContacts() {
	// fills the Contacts list
	$.get("/data/contacts", $("#categories").serialize(),function(data) {
		$("#contactList").html(data);
	});
}
```

- Joe: let's manage the selected categories on the back-end:

In *w_preprocess*, an array of variables url-encoded is built:
WEB GET VARIABLES(w_names;w_values)

When the resource requested is Contacts, Joe simply copies `w_values` into the array `_selected_categories`.

Joe completes *w_contacts* in charge of Contacts selection to filter the user contacts depending on the categories selected:

```
QUERY([Contacts];[Contacts]UserID=[Users]id)
```

```
QUERY SELECTION WITH ARRAY([Contacts]CategoryUID;_selected_categories)

ORDER BY([Contacts];[Contacts]Lastname)
```

Nothing else to change, it simply works.

- Joe: Sue, let's proceed the same way for tasks?

Sue modified the event handler for click on a category:

```
// change of the selected categories
       $("#categories input").click(function(e) {
                    displayContacts();
                    reloadTaskTable();
       })
```

Then, sighing: I've got a problem to write `reloadTaskTable()`. I don't find an easy way to resend the XHR to the Server. Let me a moment to study the documentation and examples, please...

Joe: Sue having problems? That's a scoop! I'll go to the kitchen and fetch some real cookies to encourage you.

After a while...

- Sue: hurrah, I think I've got a solution. Here is the code for `reloadTaskTable()`:

```
       var url="/data/tasks?"+$("#categories").serialize();
       var table= $('#taskList').dataTable();
       table.fnReloadAjax(url);
```

You need to add fnReloadAjax script which is a dataTables plugin: http://datatables.net/plug-ins#api_fnReloadAjax.

In the html page load this script AFTER dataTables:

```
<script src="/js/fnReloadAjax.js"></script>
```

Then when you click on a category, the dataTables will send a request like this one:

```
GET/data/tasks?category=2D82B7C749A04444A084A1AD82941629&category=600B20
3CEC85496EB946F1297522DB0B&_=1349699531923
```

- Joe: what is this last item which name is "_"?

- Sue: seems it is automatically appended by dataTables.

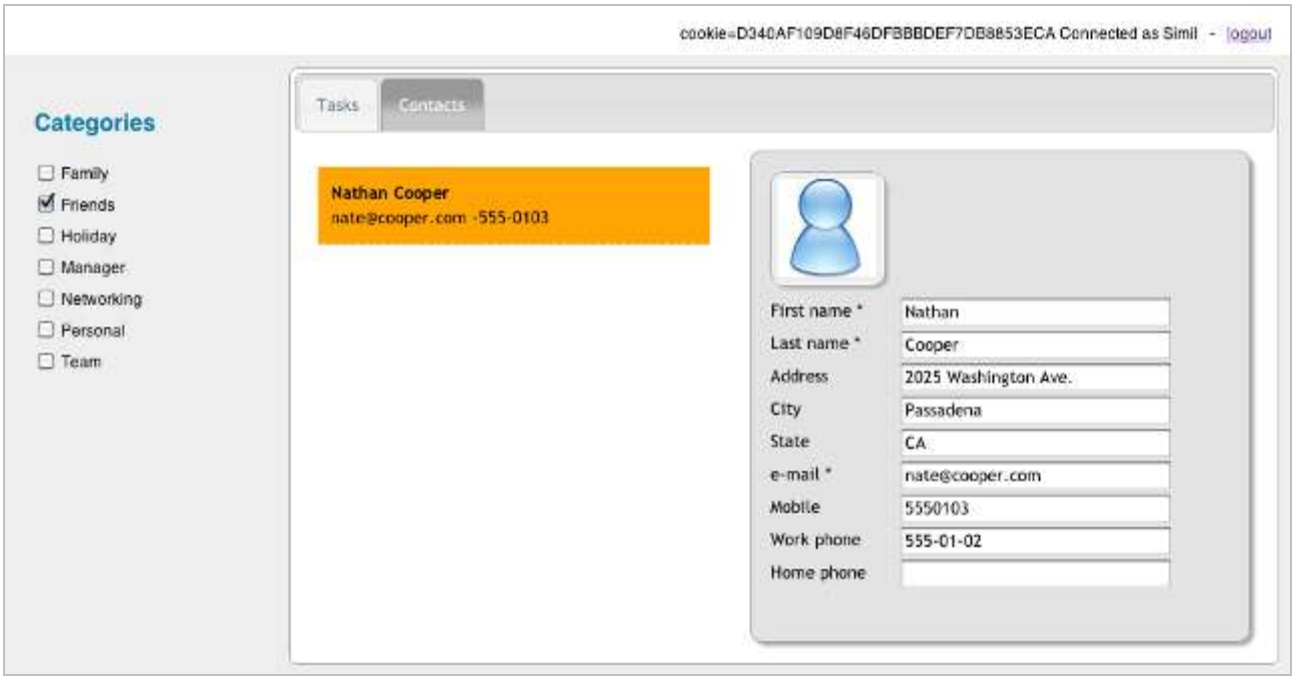
- Joe: I will filter it and after that I just need to do the same as for Contacts, I will get these values in w_values array and then use them to filter my user selection of tasks. Quick update in *w_tasks* in charge of Tasks selection:

```
QUERY([Tasks];[Tasks]UserID=[Users]id)

QUERY selection WITH ARRAY([Tasks]CategoryUID;_selected_categories)

ORDER BY([Tasks];[Tasks]DueDate;<)
```

End of the day

- Joe: didn't notice it: it's dark outside! Happily, we have nearly covered all the features. Quite impressive.

-Sue: Are you pleased with the result? I feel like we did a good job.



- Joe: great one you mean! I'm very satisfied and feel much more confident coding for the front-end. Kudos!

I've invited my brother and your mum to eat tonight with us to celebrate this training. Thanks a lot Sue you were of really great help.

MOBILE DEVELOPMENT

This document talks about some of the challenges of building mobile applications. We will look at an example of building a mobile web application as a front end to the Pre-Class sample database. We will talk about the differences between Native and Web based mobile applications and some of the advantages and disadvantages of each. The information that follows will hopefully help demystify mobile development and help you make an educated decision on which architecture to choose to develop your first rate, mobile application.

Mobile Architectures

There are two distinctively different ways to develop a mobile application. The first is using Native code and frameworks to build and compile a native application binary targeted to run on a specific device. The second architectural approach is to use web based technologies to emulate the look and feel of a native application running on a mobile device. These are very distinctively different approaches each with their own set of advantages and disadvantages. There is one other clever implementation that many developers adopt and this involves the combination of Native and Web based logic merged into one application. Lets review these architectures to see which one may be best for you.

Native Applications

A native application is an application that has been compiled to run on a specific device. The application is a binary that will only run within the targeted operating system. For example, for all of Apple's devices, these applications are developed and compiled using the XCode IDE. The language is Objective-C and the compiled binary will only run within an iOS operating system. For Android, we use Eclipse IDE and program in Java.

Advantages of Native Applications

- Richer user experience with the user of Native APIs, GPS, Accelerometer, Gyroscope, Camera, etc…
- Complex Local Data Storage
- Offline functionality (no internet connection)
- Tight integration with Push Notifications
- Background tasking
- Standardized User Experience
- Speed, Power and Control

Disadvantages of Native Applications

- Must Compile for a specific device
- More difficult to deploy
- More difficult to upgrade
- Usually more expensive in terms of development effort
- Difficult to find programmers with the required skillset

Web-Based Applications

A Web-Based application is web site that you access from any mobile browser that has been designed with a mobile interface look and feel. You basically develop two sets of HTML pages, one for the desktop and one for the mobile screen size. The web server detects which set of pages to serve out to the browser. There are a couple of leading javascript frameworks out there that help you design mobile looking interfaces, the leading ones are JQuery-Mobile and Sencha Touch. These are javascript and css files that support basic mobile features such as multi-touch and swiping.

Advantages of Web-based Applications

- Cross-device compatible
- No application to download (No App Store)
- Very easy to release updates

- Easily shared with others
- Leverage existing web site logic and content
- Huge knowledge base

Disadvantages of Web-based Applications

- Slow performance since your application runs many layers away from the Native APIs.
- Extremely limited access to the hardware
- No multi-threading
- Requires an internet connection
- Limited user interface control
- Limited local data storage
- Lack of security

Hybrid Applications

A Hybrid application is one where the bulk of the app is built using web technologies, but native code is used to access the devices native APIs. These apps are compiled for a specific device but all the web content can be cross-device compatible. This allows the developer to leverage existing web content and experience and limits the need to write the entire application in native code.

Advantages of Hybrid Applications

- Only a portion of native code has to be rewritten for multiple devices
- Reuse existing web-based content
- Allows access to many of the Native APIs
- Can take advantage of local storage
- Supports Push Notifications
- Limited Offline functionality

Disadvantages of Hybrid Applications

- Slower performance than a native application.
- Must compile for a specific device
- Deploy through an app store
- Requires an internet connection
- Limited user interface control
- Limited support for communicating between the web content and the native core

Choosing an Architecture

It is interesting to note that what drives the architectural decision, in most cases, is based on which technology the developer has learned in the past. For example, if you ask a web developer to develop a mobile application, they will obviously use web technologies like HTML 5, Javascript and css to develop a web-based application. If you ask somebody who has only developed in XCode or JAVA to write a mobile application, they will write a native application. With this unfortunate truth it is easy to see that it may be best to figure out what your application does, and what the requirements are BEFORE you decide on a specific architecture.

Let the Requirements Dictate the Architecture

The debate over Native vs Mobile will probably never be resolved in the near future. Many companies dump hundreds of thousands of dollars into one or the other and stand behind their decision. This is fine and it should be this way because in all honesty, there is no specific architecture that will work well for all types of applications. When you sit down and review the requirements of your applications, you should make the decision about the architecture based on the requirements. A simple example would be that a graphically intensive application that requires intensive processing would be best suited for a Native architecture. A contact management

application that integrates with an existing web database would lend itself best as a mobile web application.

Some Questions to Help Guide you

Questions	Native	Web-based	Hybrid
Support multiple devices and platforms		**Yes**	
Support Local complex data storage and retrieval	**Yes**		**Limited Support**
Use device hardware, GPS, Accelerometer, cameras, etc…	**Yes**	**Limited Support**	**Limited Support**
Need for Speed	**Yes**		
Offline Access	**Yes**		**Limited**
Easy & Fast Deployment		**Yes**	
Extensive Knowledge Base for developers	**Limited**	**Extensive**	**Limited**

Mobile Interface Design

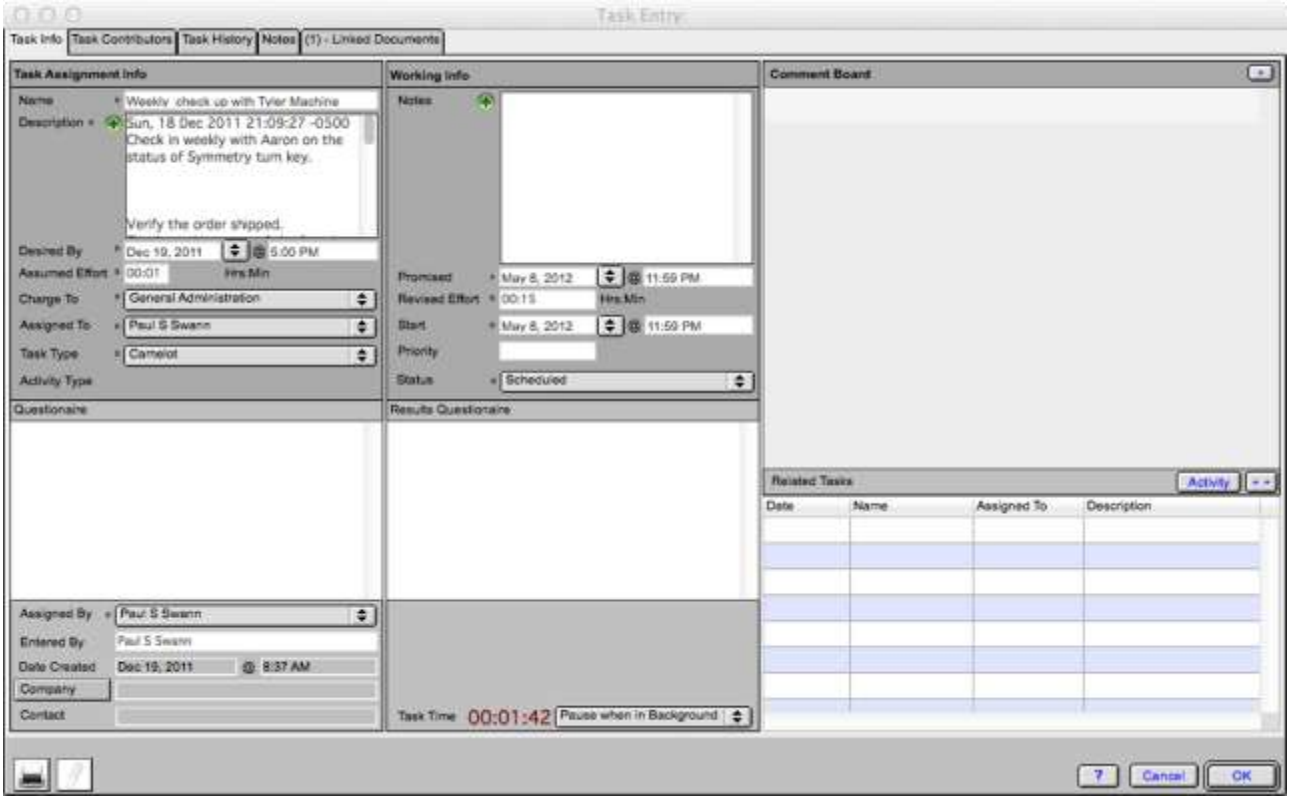
Just as there are Human Interface Guidelines for designing desktop applications, there are also interface guidelines for designing mobile interfaces. Apple has led the market on the look and feel for what is expected for mobile interfaces. The transition animation between screens, the way you swipe through lists and across views, even the way buttons are expected to respond and how field entry is controlled with the pop-out keyboard. These interface guidelines may not be published, but users expect applications on their device to have a consistent look and feel.

When designing mobile interfaces, consider that you are now targeting a small screen and our mouse is now a thumb or finger. Any interface element that will be touchable by the user must be large. This takes up screen real estate if you have many touchable items on one screen. This is where scrollable views can work to your advantage. Also, consider the use of horizontally scrollable views or pages in your interface. This allows you to group content on separate views.

For years, we have been asking for modeless systems and applications. Now that we have this in our desktop applications, we are now forced to start thinking about modal applications again. Your mobile application is completely modal. You can only view one screen at a time and cannot move between windows within your application. This forces us to sometime be tricky in our design so that keep the user experience intuitive and the user does not get lost in our application.

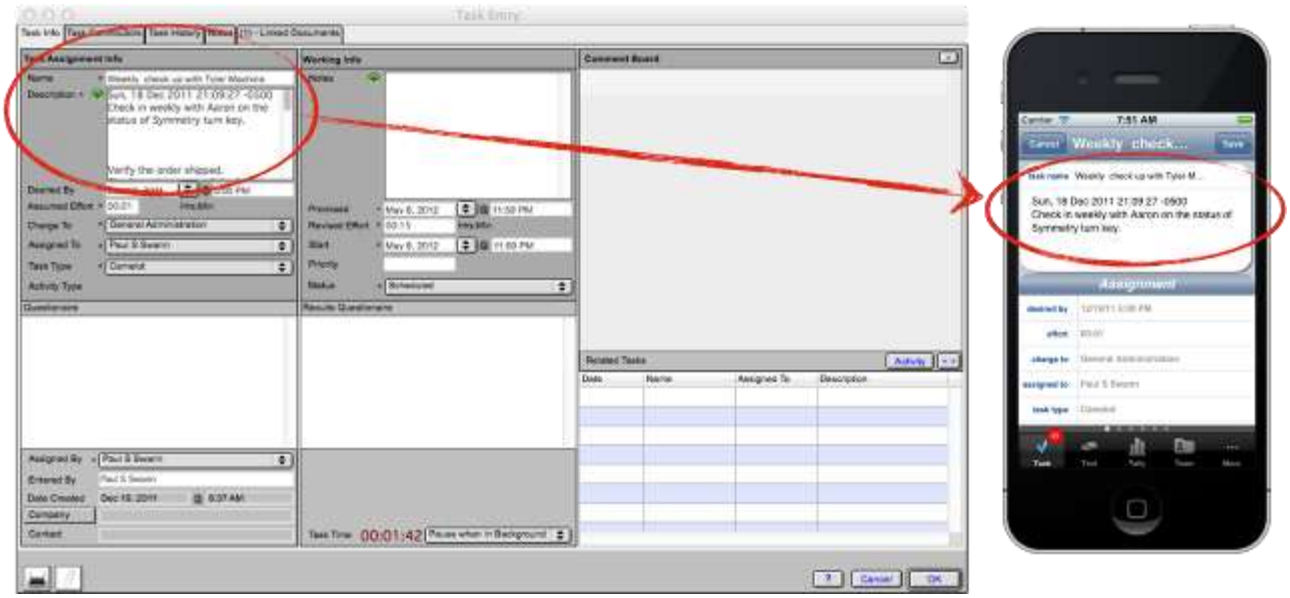
Desktop to Mobile example

In this example, we have a standard 4D desktop interface. This screen is a task entry screen where the user defines a task and assigns the task to an employee. Notice that this screen is broken out into several sections, one for Task Assignment Info where the user specifies the details about what the task is and who will be assigned to the task. Another section defines the Working Info for the task. This area manages notes and progress as the task is being performed over time. The last tab in this interface is entitled Linked Documents and is responsible for attaching documents to this task.

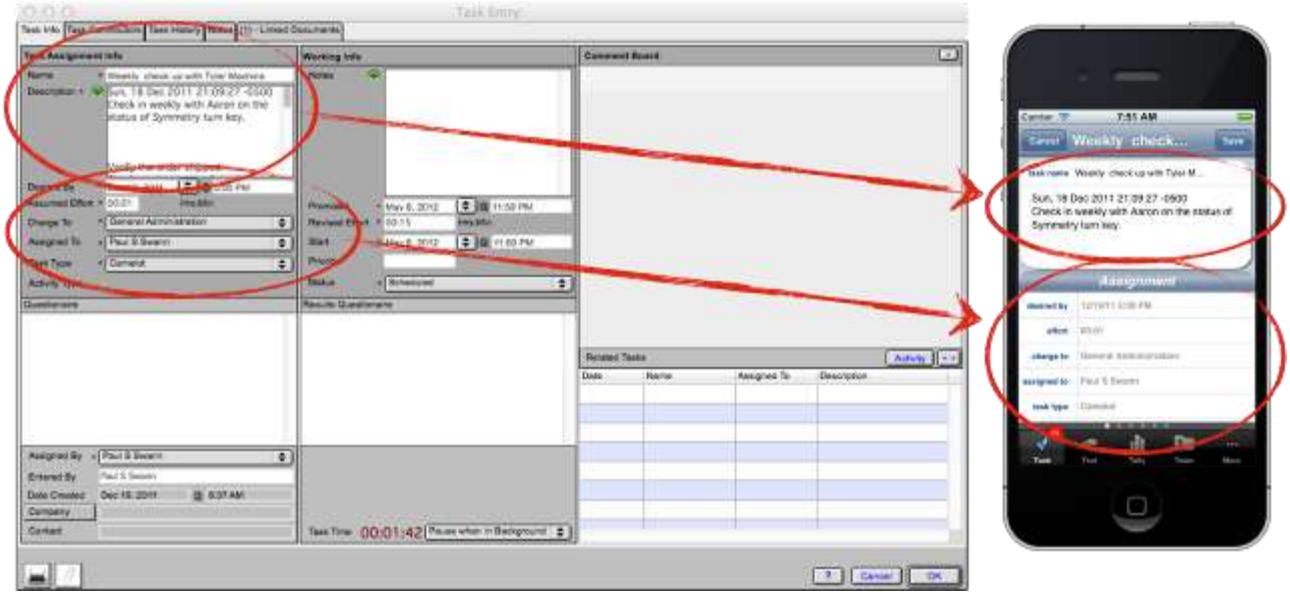


Our challenge was to create a mobile interface using portions of the task entry screen. Here is what we came up with:

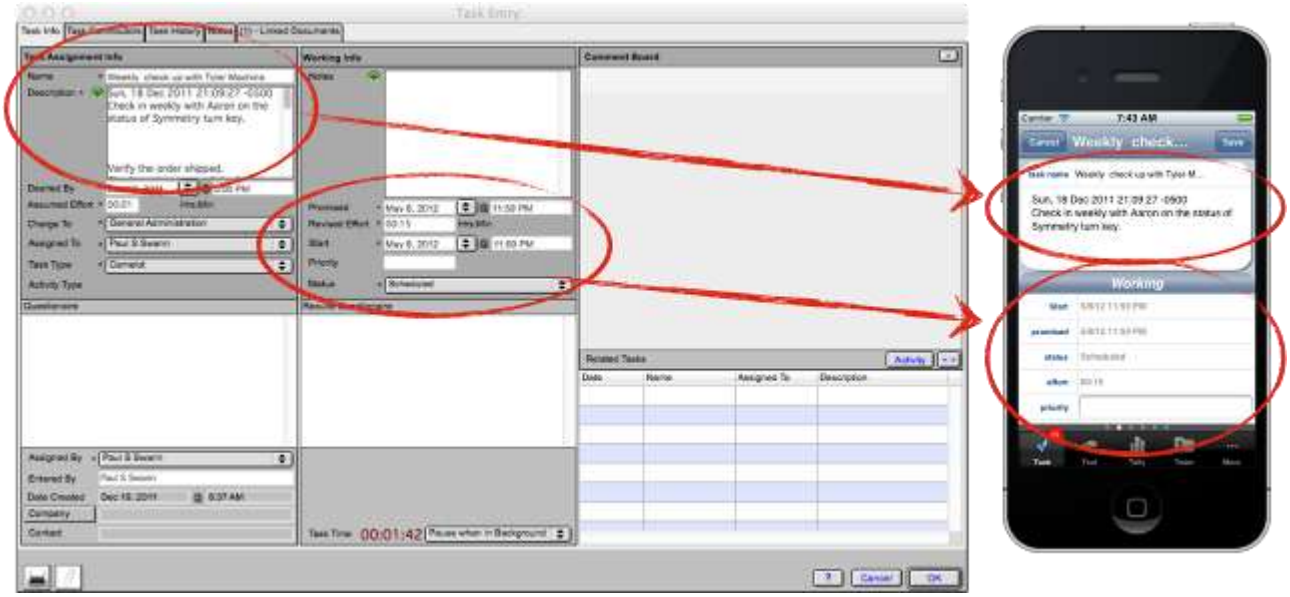
First, we decided that the upper area of the task entry screen should contain the task name and description.



Next, we decided that the lower half of the screen would scroll both vertically and horizontally. What we ended up with was the ability to scroll up and down through portions of the content, then swipe from side to side to transition between views. The first view listed the fields for the Task Assignment Info area.

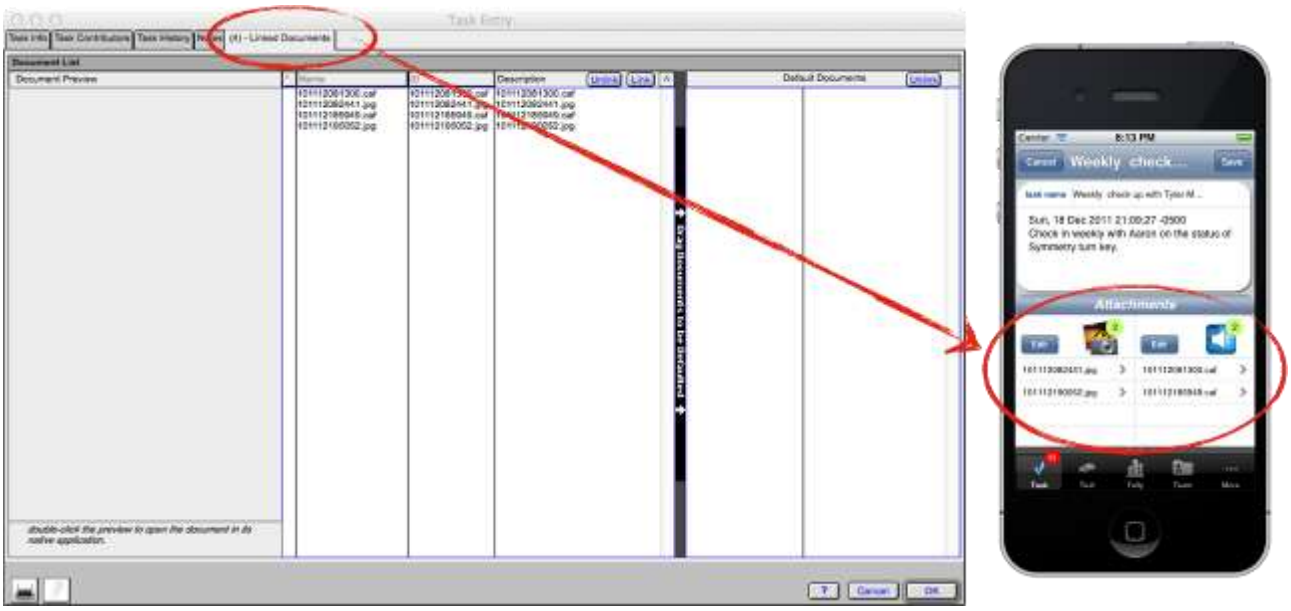


By swiping from right to left, the bottom section transitions over to expose the Working Info fields for the task. The user can then, scroll up and down through the Working Info fields as shown below:



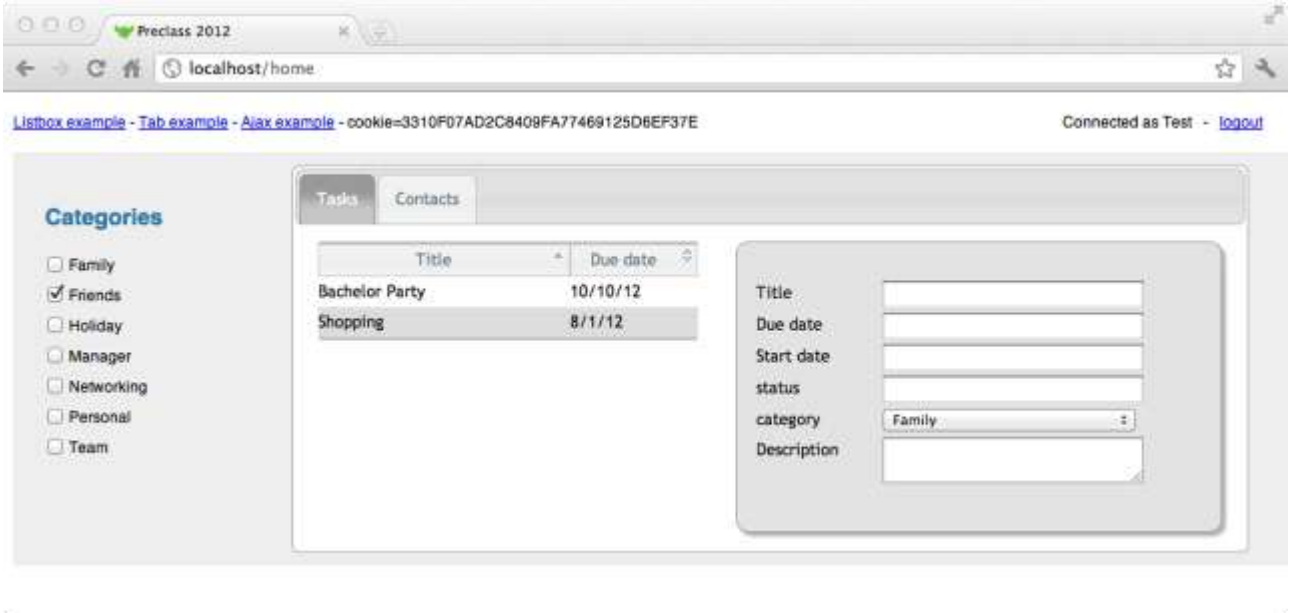
By swiping again, the lower section transitions over to expose the Attachments view. This view is the mobile interface for the Linked Documents tab in the 4D interface. This screen allows the user to attach photos either from the photo library or from the camera. It also allows the users to attach recorded video and finally, audio dictations.

What is impressive about this design is that we have not only matched the functionality of the 4D interface, but we have improved on it with the addition of recording video, taking snapshots and capturing audio dictations and linking them to the task record.

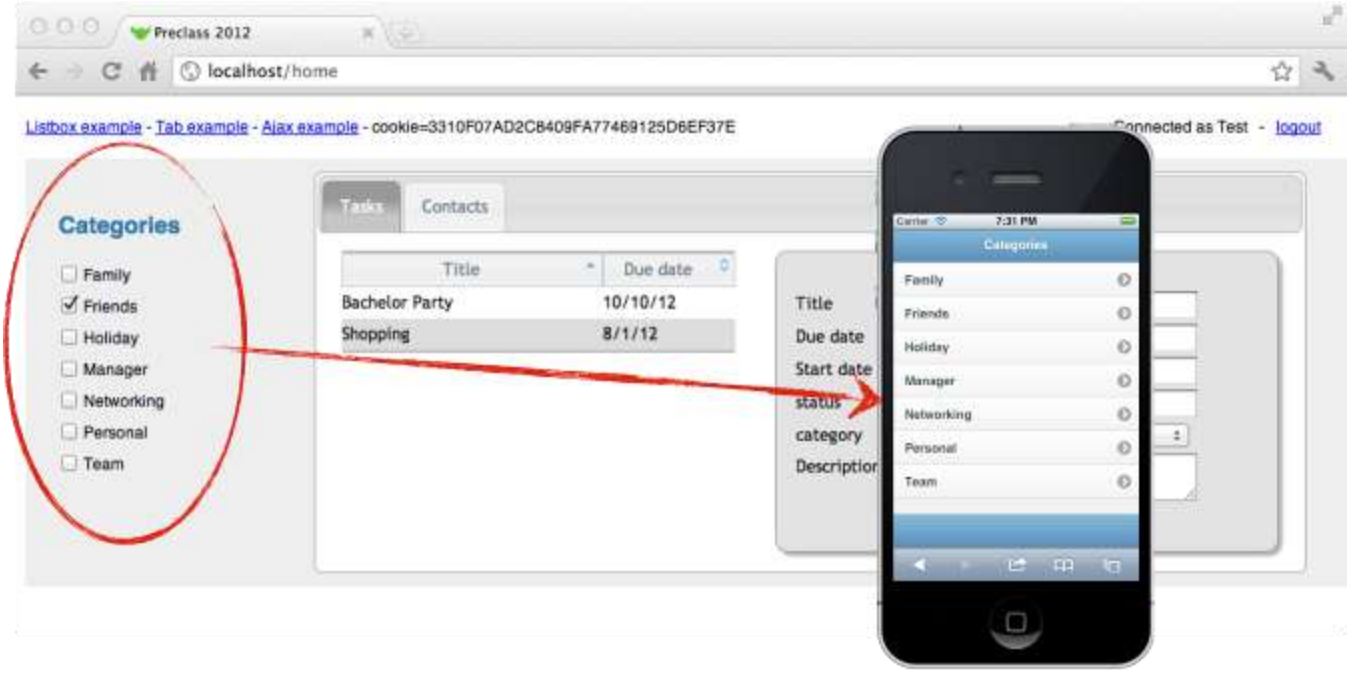


Pre-Class Demo to Mobile interface

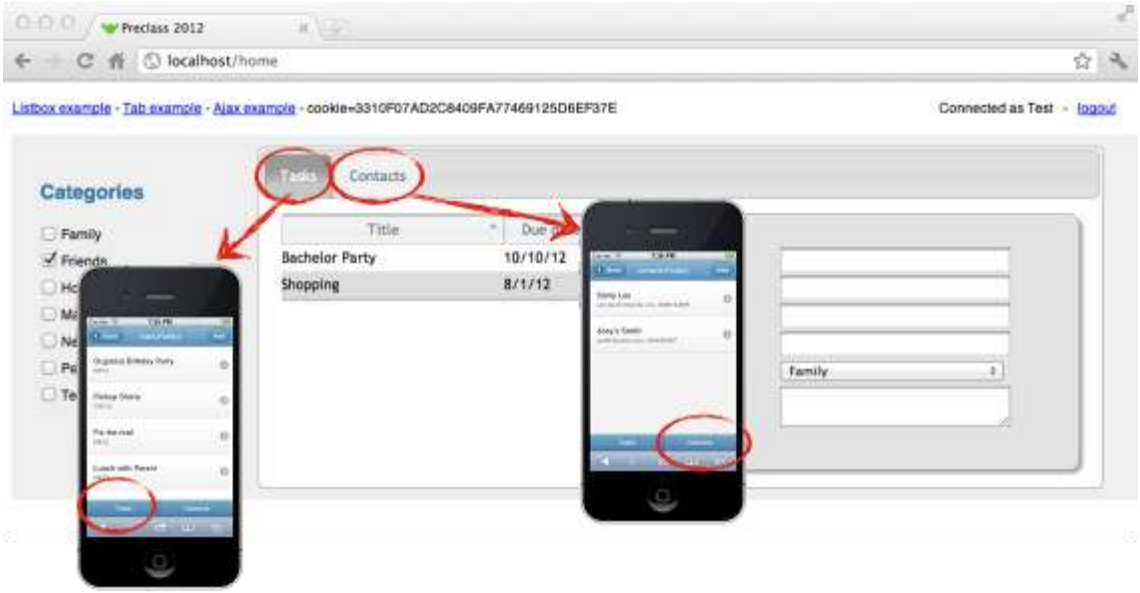
Now that we have learned the differences in mobile architectures and have reviewed some considerations for creating mobile interfaces, lets see what we can come up with for putting together a mobile interface for our Pre-Class Demo. Given the following screen, we need to try to organize the information in a meaningful way that can be displayed on a small mobile screen.



The first thing we can see is that the navigation starts with the selection of Categories, so our starting point will be to work up an interface starting with categories.



Next, whenever we touch a category, we would like to transition to show the related Tasks and Contacts. We will try to retain the fact that there is a tab to switch between Tasks and Contacts, so we could come up with a screen that looks something like:



From these screens, you can touch an item in the list to navigate to their respective entry screens. Also, the user can add Contacts and Tasks by clicking the add button above the list. To delete a Task or Contact, we chose to navigate to the entry screen and allow the user to click the Delete button within the entry screen.

You may have noticed that we have lost some functionality in or mobile design. In this implementation, we allow the user to touch one and only one category and when they do, the screen transitions to display the related Tasks and Contacts for the one selected category. This demonstrates that sometimes, in order to maintain an intuitive mobile interface, you may have to alter the functionality a bit. We could have designed a similar kind of interface to that used in the web example where the user can select checkboxes next to categories, then allow the user to click a button entitled “Display Results”. This would work, but is not very intuitive for a mobile design.

Under the Hood of our Mobile Demo

OK, now that we have reviewed the interface and looked at the design, look and feel, lets expose the code and see how we accomplished this design.

The architecture of choice was to develop a pure web-based mobile application using JQuery-Mobile frameworks.

I must explain that our goal was to make minimal modifications to the existing 4D database and attempt to use all the existing web calls that were developed for the web interface. We were able to accomplish this with the exception of two areas.

The first addition was to add code so that 4D could detect whether a request was coming from a desktop web browser or a mobile web browser. To accomplish this, we used the http header field “user-agent”. In the ON WEB CONNECTION method in 4D, we added a call to a new method entitled “w_isMobile”. This method returns TRUE if the connection was made from a mobile browser.

```
On Web Connection
1   C_TEXT($1;$2;$3;$4;$5;$6)
2   w_preprocess ($1;$2;$3;$4;$5;$6)  // fil
3   C_BOOLEAN(w_connected)  // False by defa
4   C_BOOLEAN(fIsMobile)  //Phil, Added new
5
6   fIsMobile:=w_isMobile
7
8   Case of
9     : (False)  // ************ simple exam
11    : (w_page="test_tabs")  ...
18    : (w_page="test_listbox")  ...
22    : (w_page="test_ajax")  ...
25    : (w_page="ajaxGetTime")  ...
30    : (False)  // ************ Home page *
32    : ((w_page="") & (w_resource=""))
39    : (w_page="login")  ...
42    : (Not(w_connected))  ...
48    : (w_page="logout")  ...
51    : (w_page="home")
Ln 7   Col 0   Ch 1
```

```
Method: w_isMobile
1   //Phil, Added new method
2   C_BOOLEAN($0)
3
4   ARRAY TEXT($sttHeaders;0)
5   ARRAY TEXT($sttValues;0)
6   WEB GET HTTP HEADER($sttHeaders;$sttValues)
7   $0:=True
8   $ttUserAgent:=""
9   $xlPos:=Find in array($sttHeaders;"user-agent")
10  If ($xlPos>0)
11    $ttUserAgent:=$sttValues{$xlPos}
12  End if
13  Case of
14    : ($ttUserAgent="@iPhone@")
15    : ($ttUserAgent="@iPad@")
16    : ($ttUserAgent="@iPod@")
17
18    : ($ttUserAgent="@Android@")
19      // ..etc
20
21    Else
22      $0:=False
23  End case
Ln 1   Col 0   Ch 1
```

In this example, we test for the iOS devices and Android. In order to expand your support, you could add additional lines in your case statement for Blackberry, Windows Mobile, etc…

Now that we have the ability to discern mobile from desktop requests, we simply add a test when serving the “home.shtml” screen and if we are responding to a mobile request, we need to server the “mbl_home.shtml” page instead.

```
50    : (w_page="home")
51      ALL RECORDS([Category])
52      If (fIsMobile)  //Phil, Added check
53        w_sendFile ("mbl_home.shtml")
54      Else
55        w_sendFile ("home.shtml")
56      End if
57      QUERY WITH ARRAY([Category]UID;_selected_categories)
58
```

The second modification that we made was in the REST call for “contacts” whic returns the list of contacts. In the web example, Simil chose to use the technique for embedding 4D Tags to return the raw HTML for an unordered list.

```
131     : (w_resource="contacts")  // list
132       RELATE MANY SELECTION([Contacts]CategoryUID)
133       QUERY SELECTION([Contacts];[Contacts]UserID=[Users]id)
134       ORDER BY([Contacts];[Contacts]Lastname)
135       If (fIsMobile)  //Phil, for jQuery-Mobile transitions, we
136         w_sendFile ("contactListJSON.shtml")  //Phil, added page
137       Else
138         w_sendFile ("contactList.shtml")
139       End if
140
```

In the Mobile example, we decided to return built JSON so that we could rebuild the JQuery-Mobile unordered list. Soon we will discuss why we did this and some of the side effects with working with JQuery-Mobile.

contactList.shtml – returned for desktop web interface

```
<!--4dloop [Contacts]-->
<li uid="<!--4dtext [Contacts]UID-->">
    <b><!--4dtext [Contacts]Firstname+" "+[Contacts]Lastname--></b>
    <br>
    <!--4dtext [Contacts]eMail-->  <!--4dtext [Contacts]workPhone-->
</li>
<!--4dendloop-->
```

contactListJSON.shtml – returned for mobile web interface

```
{
  "sEcho": 1,
  "iTotalRecords": "<!--4dtext nbRecords-->",
  "iTotalDisplayRecords": "<!--4dtext nbRecords-->",
  "aaData": [
  <!--4dloop [Contacts]-->
    [
      "<!--4dtext [Contacts]UID-->",
       "<!--4dtext [Contacts]Firstname-->",
       "<!--4dtext [Contacts]Lastname-->",
       "<!--4dtext [Contacts]Email-->",
       "<!--4dtext [Contacts]mobilePhone-->"
    ]<!--4dtext w_comma(->[Contacts])-->
  <!--4dendloop-->
  ]
}
```

Structure of the Mobile Web Interface

As I noted before, we decided to use a pure web-based solution for our mobile interface. There were a couple of reasons why we made this decision. First, we wanted to leverage as much of the existing javascript techniques that were already developed for the desktop web interface. Secondly, we wanted to make minimal changes to the 4D web server code. We accomplished our goal by adopting the use of JQuery-Mobile to give our application the look and feel of a mobile app. There is no particular real reason why we chose JQuery-Mobile other than the fact that Simil was already using JQuery extensively and we wanted to follow suit.

We followed the JQuery-Mobile examples and created a single web page entitled "mbl_home.shtml". To include the JQuery-Mobile framework add the following code in the <head> section of the page:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet"  href="/js/jquery-mobile/jquery.mobile-1.2.0-pre.css" />
<link rel="stylesheet" href="/js/jquery-mobile/jqm-docs.css"/>

<script src="/js/jquery-mobile/jquery-1.7.1.min.js"></script>
<script src="/js/jquery-mobile/jqm-docs.js"></script>
<script src="/js/jquery-mobile/jquery.mobile-1.2.0-pre.js"></script>
```

The structure of the page simply contains a separate DIV for each page that we implemented. The content for the page consists of three DIV structures, one for the Header, one for the Content and one for the Footer. The first page will be the list of categories. Since this listing should be initially populated with all categories, we will simply use 4D Tags when the page is served. In the code below, see if you can pick out the Header, Content and Footer sections.

```
<div data-role="page" id="page">
    <div data-role="header" data-theme="b">
        <h1>Categories</h1>
    </div>

    <div data-role="content">
        <ul data-role="listview">
            <!--4dloop [Category]-->

            <li onClick="selectedCategory('<!--4dtext [Category]UID-->', '<!--4dtext [Category]name-->')">
            <a href="#taskListView" data-transition="slide"><!--4dtext [Category]name--></a>
            </li>

            <!--4dendloop-->
        </ul>
    </div>

    <div data-role="footer" data-theme="b" data-position="fixed">
        <h4> </h4>
    </div>
</div>
```

Notice that in the unordered list definition, we set an "unclick" event that calls the "selectedCategory()" function. We pass this function the Category UID and Name. We also configure the list item inner html as a hyperlink. This is required if you want the list item to be selectable. The "<a href=" value for the list item is set to the ID of the DIV defining the page we want to transition to when this item is clicked. In this case, we will navigate to the DIV with an ID of "taskListView". There is no content in the Footer section, but the existence of the Footer DIV tells JQuery-Mobile to draw an empty footer. The html code for the "taskListView" looks like this:

```
<!--  Task Listing  -->
<div data-role="page" id="taskListView">
    <div data-role="header" data-theme="b">
        <a href="/" data-icon="arrow-l" data-transition="slide" data-direction="reverse">Back</a>
        <h1 id="taskListingHeader">Tasks</h1>
        <a href="#taskInfo" data-transition="slide" class="ui-btn-right" onClick="addTask()">Add</a>
    </div>

    <div data-role="content" >
        <ul data-role="listview" id="taskListing">
              <!-- the content is programatically build using AJAX -->
        </ul>
    </div>

    <div data-role="footer" data-id="foo1" data-position="fixed" data-theme="b">
            <div data-role="navbar">
                <ul>
                    <li onClick="loadtasks()">
                    <a href="#taskListView" data-prefetch="true" data-transition="fade">Tasks</a>
                    </li>
                    |
                    <li onClick="loadcontacts()">
                    <a href="#contactListView" data-prefetch="true" data-transition="fade">Contacts</a>
                    </li>
                </ul>
            </div><!-- /navbar -->
    </div>
</div>
```

In the Task list view, notice that we have added two buttons in the header section of the page, a “back” button to navigate back to the previous screen and an “Add” button to add a new task. The content section simply contains an “ ” The reason for this is because the content of this screen is loaded programmatically in the “selectedCategory()” function whenever a category item is selected on the category listing screen. The footer on this page contains a navigation tab bar containing tabs for “Tasks” and “Contacts”. Notice that when the user touches the Tasks navigation bar item, we call the function “loadtasks()” and then navigate back to its self. Clicking on the “Contacts” navigation item calls “loadcontacts()” and navigates to the page that has a DIV ID of “contactListView”.

Lets look at the page definition for the Contact Entry screen to see how we can define entry fields using JQuery-Mobile.

```
<!--  Contact Detail Screen  -->
<div data-role="page" id="contactInfo"  data-add-back-btn="true">
    <div data-role="header" data-theme="b">
        <h1>Contact Entry</h1>
        <a href="#contactListView" data-transition="slide"
        data-direction="reverse" class="ui-btn-right" onClick="saveContact()">Save</a>
    </div>
    <div data-role="content">
    <form id="contactForm" action="">
    <input type="hidden" id="contactUID" name="contactUID" />
        <ul data-role="listview">
            <li data-role="fieldcontain">
                <label for="firstName">First Name</label>
                <input type="text" name="firstName" id="firstName" value=""  />
            </li>
            <li data-role="fieldcontain">
                <label for="lastName">Last Name</label>
                <input type="text" name="lastName" id="lastName" value=""  />
            </li>
            <li data-role="fieldcontain">
                <label for="mail">e-mail</label>
                <input type="text" name="mail" id="mail" value=""  />
            </li>
            <li data-role="fieldcontain">
                <label for="MobilePhone">Mobile</label>
            <input type="text" name="MobilePhone" id="MobilePhone"></input>
            </li>
         </ul>
     </form>
    </div>
    <div data-role="footer" data-theme="b" data-position="fixed">
        <a href="#contactListView" data-role="button" data-icon="minus"
        data-transition="slide" data-direction="reverse" onClick="deleteContact()">Delete Contact</a>

    </div>
</div>
```

The content area defines the enterable fields for First name, Last Name, E-Mail and Mobile. The header section contains a “Save” button for saving the modified contact info. The footer section contains a button entitled “Delete Contact”. Notice that we did NOT specifically put a “Back” button on this page. This is because we set the “data-add-back-btn” property equal to “true”. JQuery-Mobile automatically manages a back button for this page.

How JQuery-Mobile works

As you have probably noticed by now, there is nothing in the definition of our page DIVs that hint we are using JQuery-Mobile. There are, however, interesting properties like “data-role”, “data-transition”, “data-direction”, etc… These are in fact, jQuery-Mobile properties that tell JQuery-Mobile how to interpret and display these objects. Lets look at just a few that we have used within our page.

- *data-role* - defines what type of object your defining, like” fieldcontain”, “navbar”, “header”, “content”, “footer”, “listview”, etc…
- *data-theme* – specifies what type of skin to use when drawing the object.
- *data-icon* – allows you to control the icon used when rendering objects.
- *data-transition* – describes the transition style when navigating between pages.
- *data-direction* – specifies the direction of the transition. The default is a slide to the right.
- *data-position* – sets the object as fixed or relative.
- *data-add-back-button* – this property is usually used in the page DIV and tells jQuery-Mobile to add a back button to the page that automatically handles the navigation when

pressed. You can manually add a back button just as we did in the "taskListView" example above.

Whenever the page is finished loading, JQuery-Mobile scans the page and rebuilds each element in the DOM so that the elements have the look and feel of a mobile interface. As you can see, our HTML code is not complex and we did not have to do very much to allow this to happen.

Loading Content from the Demo Database

We will now, look at an example of how we request data from the demo database. I think you will find that the code looks very similar if not identical in places to the code found in the web version of the interface. As we stated before, our goal was to reuse as much of the existing page logic as we could so that we could bring our mobile application to market much faster than if we were to start from scratch.

Whenever we touch a category in our category listing, we need to make a call to 4D to request a list of related Tasks for the selected category. We do this by setting the onClick event function to "selectedCategory(ttID, ttName)" which in turn calls "loadtasks()".

```
function loadtasks ()
{
    var ttID = ttSelectedCatUID;
    var ttName = ttSelectedCatName;

    $.post('/data/selectCategories', 'id='+ttID, function(data){
            // on success :
            $.getJSON("/data/tasks", function(getdata){
                var ttNumRecs = getdata.iTotalRecords;
                $("#taskListingHeader").html('<font style="white-space:normal; font-size: small" >Tasks ('+ttName+')</font>');

                var sttArray = getdata.aaData;
                var ttTotalRecs = '';
                for(var i=0; i< sttArray.length; i++){
                    var sttRecordArray = sttArray[i];


                    var ttTitle = sttRecordArray[0];
                    var ttDate = sttRecordArray[1];
                    var ttID = sttRecordArray[2];
                    ttTotalRecs = ttTotalRecs+'<li onClick="loadTaskDetail(\''+ttID+'\', \''+ttTitle+'\')">'
                    ttTotalRecs = ttTotalRecs+'<a href="#taskInfo" data-transition="slide">'
                    ttTotalRecs = ttTotalRecs+'<h3>'+ttTitle+'</h3>'
                    ttTotalRecs = ttTotalRecs+'<p>'+ttDate+'</p>'
                    ttTotalRecs = ttTotalRecs+'</a></li>'

                }

                $("#taskListing").html(ttTotalRecs);
                $('#taskListing').listview('refresh');// reload the contacts list
                //$('#taskListingNavBar').navbar('refresh');
            })

        });
    return true;
}
```

We use the same sequence of ajax requests that the web interface uses. We first, we make a request for "`/data/selectCategories?id=600B203CEC85496EB946F1297522DB0B`" to have 4D query for the [Tasks] records and build our current selection of tasks. Next, we make a request for "/data/tasks" which asks 4D to return the list of Tasks in JSON format. Once we get the response, we can walk though the JSON and rebuild our list of tasks.

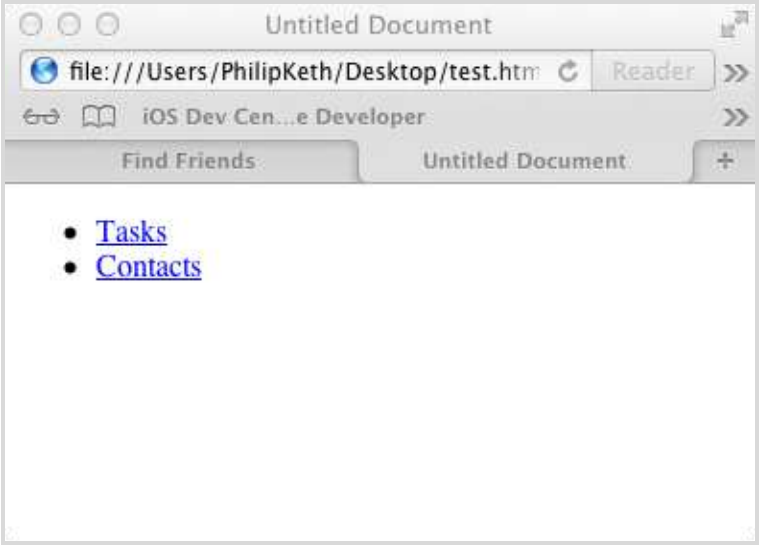
Manipulating Objects in the DOM

The key to remember here is that whenever the page has finished loading and JQuery-Mobile has done its job, all the elements in the DOM are very different in structure. This makes it difficult to manipulate certain aspects of an element with javascript. Lets look at an example.

Here we have a block of HTML code for the footer that defines our navigation bar on the task listing page.

```
<div data-role="footer" data-id="foo1" data-position="fixed" data-theme="b">
        <div data-role="navbar">
            <ul>
                <li onClick="loadtasks()">
                <a href="#taskListView" data-prefetch="true" data-transition="fade">Tasks</a>
                </li>

                <li onClick="loadcontacts()">
                <a href="#contactListView" data-prefetch="true" data-transition="fade">Contacts</a>
                </li>
            </ul>
        </div><!-- /navbar -->
</div>
```

For those web developers, this is pretty basic, we have one unordered list with two list items, one for Tasks and the other for Contacts. When viewed in a browser, this list would render like this:



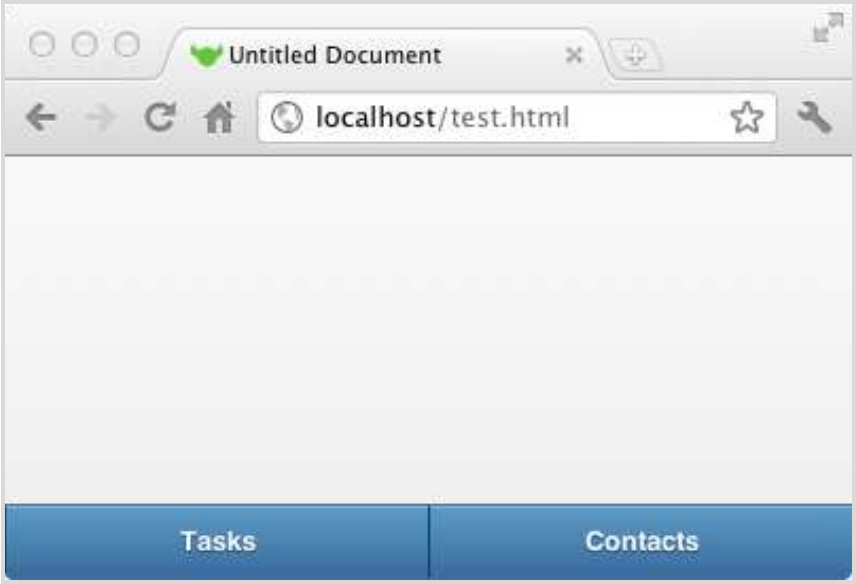
and when you inspect the element in the DOM, it retains our expected structure.

```
▼<div data-role="footer" data-id="foo1" data-position="fixed" data-theme="b">
  ▼<div data-role="navbar">
    ▼<ul>
      ▼<li onclick="loadtasks()">
         <a href="#taskListView" data-prefetch="true" data-transition="fade">Tasks</a>
       </li>
      ▼<li onclick="loadcontacts()">
         <a href="#contactListView" data-prefetch="true" data-transition="fade">Contacts</a>
       </li>
     </ul>
   </div>
   <!-- /navbar -->
 </div>
```

Now, lets add the following lines to the <head> section of our page:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet"  href="/js/jquery-mobile/jquery.mobile-1.2.0-pre.css" />
<link rel="stylesheet" href="/js/jquery-mobile/jqm-docs.css"/>

<script src="/js/jquery-mobile/jquery-1.7.1.min.js"></script>
<script src="/js/jquery-mobile/jqm-docs.js"></script>
<script src="/js/jquery-mobile/jquery.mobile-1.2.0-pre.js"></script>
```

Whenever we refresh the page, now our list is rendered as follows:



To confuse matters now, lets inspect our footer bar element to see what is really in the DOM at this point.

```
▼<div data-role="footer" data-id="foo1" data-position="fixed" data-theme="b" class="ui-footer ui-
  bar-b ui-footer-fixed slideup" role="contentinfo">
  ▼<div data-role="navbar" class="ui-navbar ui-mini" role="navigation">
    ▼<ul class="ui-grid-a">
      ▼<li onclick="loadtasks()" class="ui-block-a">
        ▼<a href="#taskListView" data-prefetch="true" data-transition="fade" data-corners="false"
          data-shadow="false" data-iconshadow="true" data-wrapperels="span" data-theme="b" data-
          inline="true" class="ui-btn ui-btn-inline ui-btn-up-b">
          ▼<span class="ui-btn-inner">
             <span class="ui-btn-text">Tasks</span>
           </span>
         </a>
       </li>
      ▼<li onclick="loadcontacts()" class="ui-block-b">
        ▼<a href="#contactListView" data-prefetch="true" data-transition="fade" data-corners="false"
          data-shadow="false" data-iconshadow="true" data-wrapperels="span" data-theme="b" data-
          inline="true" class="ui-btn ui-btn-inline ui-btn-up-b">
          ▼<span class="ui-btn-inner">
             <span class="ui-btn-text">Contacts</span>
           </span>
         </a>
       </li>
     </ul>
   </div>
   <!-- /navbar -->
 </div>
```

You can see that our unordered list and its items have been revised. So now, lets say that you want to programmatically update the list of items by making an AJAX call to the server, refer to the

section entitled "Loading Content form the Demo Database" for an example of how we repopulate the taskListing screen.

Notice that after we have rebuilt our HTML code, we set the html for the list as follows:

```
$("#taskListing").html(ttTotalRecs);
```

Once we reset our HTML the list returns to a standard, unordered list and you have broken the look of the JQuery-Mobile navigation bar. In order to force JQuery-Mobile to reformat the list for the new content, you must call the "refresh" object function:

```
$('#taskListing').listview('refresh');
```

The fact that you can no longer manipulate individual list items makes it a bit difficult. You basically have to rebuild your entire list and then call the "refresh" function. Some objects, such as buttons, have other functions like "enable" or "disable" so that you can programmatically disable a button on the fly without having to call "refresh". Knowing this limitation will save you hours of debugging down the road.

Conclusion on Mobile Development

We have now been able to create a mobile interface for our database. The advantage that we had was that we had already created a web interface and all the ON WEB CONNECTION logic existed to support web technologies. It is easy to make the assumption that since we already created hooks in 4D to respond to a web interface, that we must use a web architecture for a mobile application. This is not true, in fact, you can design a native approach and still make the REST calls with no change to the 4D Database.

Remember to define your requirements first, then brush up on the advantages and disadvantages of each mobile architecture. Only then, can you decide which architecture will give you the greatest return for your efforts.

For more information on Mobile Development and communicating with 4D using other architectures, refer to the Session entitled "iOS, Android & 4D".

To learn more about JQuery-Mobile, go to their website http://jquerymobile.com/demos/1.2.0/

REFERENCES

- Overview of the 4D Web Server, http://doc.4d.com/4Dv13.1/help/Title/en/page46.html
- On Web Connection method, http://doc.4d.com/4Dv13.1/help/Title/en/page745.html
- 4D Tags, http://doc.4d.com/4Dv13.1/help/Title/en/page2850.html
- 4D Tech Note, Static Web Serving, http://kb.4d.com/assetid=76547
- 4D Tech Note, Dynamic Web Serving, http://kb.4d.com/assetid=76611
- 4D Tech Note, Multiple Web Servers on a Single OS, http://kb.4d.com/assetid=76631
- HTTP Session Management, http://kb.4d.com/assetid=76521
- Mastering 4D Tags, http://kb.4d.com/assetid=76526
- W3 Schools, excellent tutorials for HTML, CSS, JavaScript and jQuery, http://www.w3schools.com/
- JavaScript Guide from Mozilla Developer Network, https://developer.mozilla.org/en-US/docs/JavaScript/Guide
- JQuery, http://jquery.com/
- CSS3 Maker, http://www.css3maker.com/index.html